

Report ISW2 – Software Testing

Flavio Simonelli - 0365333

Sommario

1	Introduzione.....	1
2	Bookkeeper	1
2.1	Descrizione del sistema.....	1
2.2	BufferedChannel.....	1
2.2.1	Test Manuali.....	1
2.2.2	Risultati dei test.....	3
2.2.3	Prima iterazione con Jacoco.....	4
2.2.4	Seconda iterazione con Pitest	4
2.2.5	Generazione automatica dei Test	5
2.2.6	Conclusione	5
2.3	JournalChannel.....	6
2.3.1	Test Manuali.....	6
2.3.2	Risultati dei Test.....	6
2.3.3	Prima iterazione con Jacoco.....	7
2.3.4	Seconda iterazione con Pitest	7
2.3.5	Generazione automatica dei Test	7
2.3.6	Conclusione	8
3	OpenJPA	8
3.1	Descrizione del sistema.....	8
3.2	CacheMap	8
3.2.1	Test Manuali.....	9
3.2.2	Risultati dei Test.....	11
3.2.3	Prima iterazione con Jacoco.....	12
3.2.4	Seconda iterazione con Pit Test.....	13
3.2.5	Generazione automatica dei Test	13
3.2.6	Conclusioni	14
3.3	ProxyManagerImpl.....	14
3.3.1	Test Manuali.....	14
3.3.2	Risultati dei test manuali	15
3.3.3	Prima iterazione Coverage	16
3.3.4	Seconda iterazione con PitTest	16
3.3.5	Generazione automatica dei Test	16
3.3.6	Conclusione	17
4	Appendice.....	18
4.1	Tabelle	18
4.1.1	BufferedChannel	18
4.1.2	JournalChannel.....	29

4.1.3	CacheMap.....	34
4.1.4	ProxyManagerImpl.....	39
4.2	Immagini	44
4.2.1	BufferedChannel	44
4.2.2	JournalChannel.....	50
4.2.3	CacheMap.....	62
4.2.4	ProxyManagerImpl.....	72
4.3	Prompt LLM	83

1 Introduzione

Il presente elaborato illustra le metodologie e i risultati dell'attività di verifica del software condotta su quattro classi, selezionate dai progetti open source Apache **Bookkeeper** e Apache **OpenJPA**. L'indagine ha previsto l'applicazione comparata di strategie di testing eterogenee, spaziando dalla definizione manuale dei casi di test (basata su Category Partition e Boundary Value Analysis) alla generazione assistita da Large Language Models, fino all'impiego di tool per la generazione automatica quali Randoop ed EvoSuite. Al fine di garantire la modularità dell'ambiente di lavoro, l'infrastruttura di build è stata organizzata definendo specifici profili Maven nel file pom dei moduli in esame. Attraverso l'uso del build-helper-maven-plugin sono state isolate le directory sorgenti, per ciascuna tecnica, in profili dedicati (*tests-manual*, *tests-llm*, *tests-randoop*, *tests-evosuite*). Questi sono stati affiancati da un profilo aggregatore *tests-all*, per l'esecuzione congiunta, attivo di default. Tale architettura è supportata in entrambi i progetti da un workflow di Continuous Integration implementato tramite **GitHub Actions**, che integra anche l'analisi della qualità del codice via **SonarCloud**, assicurando l'automazione dei processi di Continuous Testing e la riproducibilità delle suite generate. I repository sono disponibili ai seguenti link Bookkeeper¹ e OpenJPA².

2 Bookkeeper

2.1 Descrizione del sistema

Apache BookKeeper è un sistema di storage distribuito progettato per garantire bassa latenza e alta affidabilità nella gestione di log sequenziali. All'interno della sua architettura, ogni nodo di storage (Bookie) assicura la durabilità dei dati tramite un meccanismo di Write-Ahead Logging su un registro persistente chiamato Journal. È in questo contesto critico che operano le classi oggetto di questa analisi: **JournalChannel** e **BufferedChannel**.

2.2 BufferedChannel

All'interno dell'ecosistema Apache BookKeeper, la classe BufferedChannel svolge un ruolo critico nella gestione dell'I/O, agendo come wrapper funzionale per i FileChannel utilizzati nell'archiviazione di Ledger e Journal.

Come documentato³, questa classe implementa un layer di buffering in memoria RAM posizionato logicamente al di sopra della page cache del sistema operativo e del sottostante file. Questa configurazione definisce un'architettura a tre livelli (Application Buffer → OS Cache → Disk) progettata per ottimizzare le performance di scrittura. Il principale vantaggio di questa architettura risiede nell'abilitazione dell'I/O asincrono, che realizza un efficace disaccoppiamento tra l'operazione di scrittura logica e la persistenza fisica su disco, traducendosi in un significativo incremento delle prestazioni. Parallelamente, la resilienza del dato è garantita da un modello a consistenza causale governato da una logica a soglia. Il sistema monitora il volume di byte accumulati nel buffer e nella cache del file che non sono ancora persistiti: al superamento del limite predefinito, viene attivato un flag di stato che innesca mandatoriamente l'operazione di flush su disco, assicurando la durabilità dei dati.

Le funzionalità principali individuate per il testing sono:

- Inizializzazione: creazione del layer di buffering e associazione al FileChannel sottostante.
- Scrittura: scrittura nel buffer e utilizzo del meccanismo asincrono prima descritto per la persistenza dei dati su disco.
- Lettura: Verifica della capacità della classe di astrarre la posizione fisica del dato, accertandosi che restituisca una vista contigua e coerente dei dati indipendentemente dal layer in cui risiedono.

2.2.1 Test Manuali

INIZIALIZZAZIONE DEL LAYER IN MEMORIA

La fase di istanziazione della classe BufferedChannel è responsabile dell'allocazione dei buffer in RAM essenziali per l'esecuzione di tutte le successive operazioni di I/O. L'implementazione di tale logica risiede nel costruttore della classe:

```
public BufferedChannel(ByteBufAllocator allocator, FileChannel fc, int writeCapacity, int readCapacity, long unpersistedBytesBound) throws IOException
```

Esso ha la responsabilità primaria di validare l'integrità del FileChannel sottostante e, successivamente, di procedere alla creazione dei buffer in memoria secondo i parametri di configurazione forniti.

Identificazione dei parametri

Come indicato nella documentazione alla sezione Reference>Configuration>Journal Settings⁴:

- writeCapacity: Definisce la dimensione (in byte) del buffer dedicato alle operazioni di scrittura in memoria RAM.
- readCapacity: Definisce la dimensione (in byte) del buffer dedicato alle operazioni di lettura.
- ByteBufferAllocator: Allocatore responsabile della creazione dei buffer in memoria.
- FileChannel: Il canale file nativo su cui il BufferedChannel si appoggia per la persistenza fisica e il recupero dei dati.

Dall'analisi corrente è stato volutamente escluso il parametro del costruttore unpersistedBytesBound, poiché non ha un ruolo attivo all'interno della funzionalità descritta. Il suo valore, infatti, non modifica alcun comportamento in questa fase. Tale parametro sarà oggetto della Category Partition relativa alla funzionalità di Write, dove il suo valore influenzerà in modo determinante il comportamento della classe.

Category Partition

Per i parametri selezionati e in base alla funzionalità descritta sono state distinte le seguenti classi di equivalenza in *Tabella 1*.

¹ <https://github.com/flavio-simonelli/bookkeeper/tree/flavio>

² <https://github.com/flavio-simonelli/openjpa/tree/flavio>

³ <https://bookkeeper.apache.org/docs/latest/api/javadoc/org/apache/bookkeeper/bookie/BufferedChannel.html>

⁴ <https://bookkeeper.apache.org/docs/reference/config/>

Si potrebbe ipotizzare un'analisi delle partizioni sulle dimensioni combinate per writeCapacity e readCapacity; tuttavia, indipendentemente dall'implementazione specifica, sono necessari due buffer distinti: uno per contenere i dati in lettura e l'altro per i dati in scrittura, poiché tali operazioni possono avvenire in posizioni differenti. Al contempo, si è prestata particolare attenzione al valore 0 per questi due parametri. Si presuppone che una richiesta di un buffer con capacità pari a 0 sia una richiesta valida; tuttavia, la configurazione si considera non corretta nel momento in cui sia il buffer di lettura che quello di scrittura siano impostati a 0, poiché ciò renderebbe privo di senso l'utilizzo della classe.

Boundary Analysis

Una volta stabilite le partizioni per ogni parametro in test, identifichiamo un valore rappresentativo di ogni categoria utilizzando la tecnica delle Boundary Values, ovvero cercando i valori rappresentativi ai limiti di ogni categoria. I valori scelti, e la loro motivazione è descritta in *Tabella 2*. Una particolare attenzione è stata rivolta alla gestione del componente ByteBufAllocator. Sebbene quest'ultimo sia intrinsecamente stateless agendo come una factory, si è ritenuto necessario definire una specifica classe di istanza invalida per prevenire potenziali regressioni funzionali. In questo scenario, l'integrità del ByteBuf o il rispetto delle specifiche richieste vengono compromessi intenzionalmente attraverso l'uso di mock (Mockito), al fine di testare la capacità del sistema di gestire correttamente errori nell'allocazione o nel rilascio delle risorse. Le medesime considerazioni metodologiche applicate al FileChannel e ai restanti parametri sono descritte nel dettaglio all'interno della *Tabella 2*.

Test

La metodologia adottata evolve da un approccio teorico monodimensionale verso un'analisi combinatoria, testando le permutazioni critiche dei parametri. L'elenco completo di questi test è disponibile nella *Tabella 3* in Appendice.

FUNZIONALITÀ DI READ

La funzionalità di lettura esposta dalla classe BufferedChannel ha come requisito primario l'astrazione della dislocazione fisica del dato. Il componente deve mascherare la complessità dell'architettura a tre livelli (Buffer Applicativo, Cache, Disco), fornendo al chiamante una vista logica unificata e contigua dei dati, indipendentemente dal fatto che essi risiedano ancora nella memoria volatile o siano già stati persistiti sul file system.

Il punto di ingresso per la verifica di questa funzionalità è il seguente metodo:

```
public synchronized int read(ByteBuf dest, long pos, int length) throws IOException
```

Identificazione dei parametri

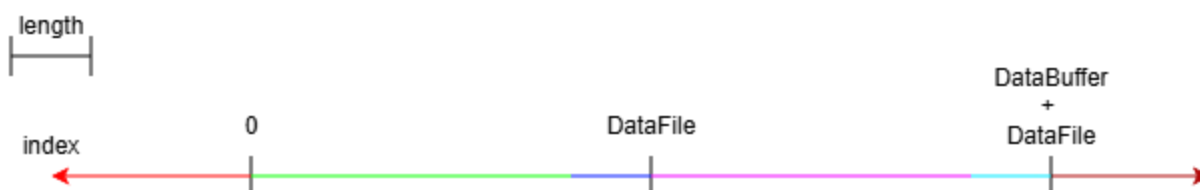
Ai fini della definizione della strategia di test, il dominio degli input non si limita ai soli argomenti passati alla firma del metodo. È necessario considerare lo stato interno dell'istanza come un input implicito determinante per l'esito dell'operazione. Di seguito vengono analizzati i parametri identificati:

- FileChannel: Il canale file su cui il BufferedChannel si appoggia per la persistenza fisica e il recupero dei dati.
- readCapacity: la capacità di lettura del buffer istanziato in fase di inizializzazione.
- length: la lunghezza dei dati da leggere.
- dest: buffer di destinazione dei dati letti.
- pos: posizione di lettura che può indicizzare un dato nel file o nel buffer.

Category Partition

Una volta individuati i parametri da analizzare, per ciascuno di essi sono state definite le partizioni di equivalenza, come riportato nella *Tabella 4*. Tra gli scenari analizzati, particolare rilevanza assume la verifica del FileChannel in stato di chiusura. Tale controllo si rende necessario poiché il ciclo di vita del canale può essere gestito esternamente: è infatti possibile che l'entità responsabile dell'istanziamento del BufferedChannel chiuda la risorsa sottostante prematuramente, rendendo il wrapper inutilizzabile. Per quanto riguarda invece il parametro pos, la derivazione delle classi di equivalenza ha seguito lo schema riportato di seguito:

Immagine 1 : Category Partition pos (Read)



Boundary Analysis

A partire dalle classi di equivalenza definite, è stata applicata la Boundary Value Analysis così come nella *Tabella 5*.

Test

La preconditione per l'esecuzione di ogni test richiede il popolamento con dati predefiniti: writeBuffer con il vettore wbData = {5, 6, 7, 8} e FileChannel (quando aperto) con il vettore fcData = {1, 2, 3, 4}.

I Test sono visionabili nella *Tabella 6* in appendice.

FUNZIONALITÀ DI WRITE

L'operazione di scrittura gestisce il payload in ingresso inserendo i dati nel buffer interno (writeBuffer) fino al raggiungimento della sua capacità massima. Nel caso in cui la dimensione dei dati in ingresso ecceda lo spazio disponibile, il sistema opera iterativamente:

1. Esegue un flush del contenuto attuale del buffer verso la cache del sistema operativo (tramite il FileChannel).
2. Libera il buffer e prosegue con la scrittura della porzione residua dei dati.

Questo ciclo si ripete fino al completo esaurimento dei dati in input.

Conformemente alla documentazione, la sincronizzazione fisica su disco è governata da una logica condizionale basata sul parametro unpersistedBytesBound. Viene specificato che la verifica di tale condizione avviene esclusivamente al

termine dell'intera operazione di scrittura logica (ovvero, dopo che tutti i dati sono stati trasferiti nel buffer o nella cache). Se in quel momento il contatore dei byte non persistenti (unpersistedBytes) raggiunge la soglia definita, viene invocata l'operazione di force. Il meccanismo di persistenza è implementato con un approccio best-effort. Operativamente, ciò significa che il contatore dei byte non persistenti viene azzerato, in via ottimistica, immediatamente prima dell'effettiva esecuzione della chiamata force sul FileChannel. Il metodo individuato per questa funzionalità è:

```
public void write(ByteBuf src) throws IOException
```

Identificazione dei parametri

- writeCapacity: Definisce la dimensione (in byte) del buffer dedicato alle operazioni di scrittura in memoria RAM.
- FileChannel: Il canale file nativo su cui il BufferedChannel si appoggia per la persistenza fisica.
- UnpersistedBytesBound: soglia descritta precedentemente.
- Src: buffer sorgente dei dati.

Category Partition

La scomposizione completa secondo il metodo Category Partition è consultabile nella *Tabella 7* dell'Appendice.

Boundary Values

L'identificazione dei Boundary values e la loro motivazione è consultabile nella *Tabella 8* in Appendice.

La tabella include la configurazione del parametro UnpersistedBytes al fine di accertare la corretta valutazione della soglia di flushing (UBB). Si intende verificare che la condizione di scatto tenga conto del backlog preesistente di byte non persistiti, oltre che del payload della richiesta di scrittura attuale.

Test

In fase di inizializzazione, il buffer sorgente acquisisce i valori dal vettore costante SRC_DATA = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}. La quantità di elementi trasferiti varia dinamicamente in funzione del parametro di lunghezza (src) specificato nel test. Per l'implementazione di questi test sfrutteremo uno spy che ci permette di verificare se viene chiamata l'operazione di force su file e quali dati erano presenti in quel momento in modo da verificare l'operazione di force su disco persistente. I test progettati sono nella *Tabella 9* in Appendice.

2.2.2 Risultati dei test

Conclusa la codifica dei test case ingegnerizzati, sono stati esaminati gli esiti ponendoci nell'ottica dell'esecutore tecnico, simulando la comunicazione dei risultati al progettista.

Inizializzazione del layer in memoria

Su un totale di 27 test eseguiti, si registrano 10 fallimenti:

Test 1: Mancata validazione della capacità. La classe viene istanziata correttamente anche se il buffer fornito dall'allocatore ha dimensioni minori di quelle richieste.

Test 2: Mancato controllo su buffer nulli. L'inizializzazione avviene senza lanciare alcuna eccezione anche se l'allocatore restituisce un buffer null.

Test 3: Utilizzo di risorse non valide. La classe viene istanziata senza eccezioni anche utilizzando un buffer già deallocato (released).

Test 6 e 7: Incoerenza operativa (Read-Only). Nonostante il FileChannel sia configurato in sola lettura e la richiesta del writeBuffer non sia nulla la classe viene istanziata senza lanciare eccezioni.

Test 13 e 15: Incoerenza operativa (Write-Only). Analogamente, viene istanziata correttamente senza lanciare eccezioni la classe BufferedChannel anche se la richiesta dimensionale del buffer di lettura è maggiore di 0.

Test 9, 16 e 25: Incoerenza operativa (entrambi i buffer =0). Il valore atteso in queste condizioni è sbagliato dal risultato ottenuto. Anche se entrambi le richieste di buffer sono di capacità nulle la classe viene correttamente istanziata.

In assenza di specifiche dettagliate ed impossibilità nel contattare il progettista del sistema per ottenere informazioni aggiuntive, i test 1, 6, 7, 13 e 15 sono stati adattati (*Tabella 10*) ipotizzando una logica di sanitizzazione interna. Si assume che l'istanziamento di un buffer di lettura venga inibita qualora il FileChannel operi in modalità di sola scrittura (writeOnly). Per supportare questa verifica, è stato implementato un meccanismo di ispezione Gray Box che consente il monitoraggio della capacità allocata per i buffer interni (writeBuffer e readBuffer).

Nonostante questa modifica i test falliscono perché la classe alloca erroneamente i buffer corrispondenti con dimensione > 0. Non si considera comunque un errore di implementazione poiché non lascia il BufferedChannel in uno stato inconsistente ma rende obbligatorio l'inserimento, nelle funzionalità di read e write, di test che verifichino le corrispettive operazioni in questi determinati stati iniziali che prima erano considerati impossibili.

Per quanto riguarda i test 9, 16 e 25, anche se supponiamo un'operazione priva di senso, consideriamo questi errori un errore nella progettazione dei test poiché comunque non lasciano lo stato del bufferedChannel in uno stato inconsistente. I test vengono quindi modificati (*Tabella 11*) per aderire a questa nuova informazione.

Dopo queste considerazioni, si provvederà ad aggiungere test per le successive funzionalità con queste caratteristiche per osservarne il funzionamento.

Read

Dopo aver codificato i test ideati, la loro esecuzione ha portato ai seguenti risultati di 3 fallimenti su 29 test.

Test 5: Il test ha esito negativo in quanto, nonostante il file sia aperto in modalità di sola scrittura, il sistema consente l'accesso in lettura ai dati presenti nel writeBuffer. Viene eseguito un test con risultato atteso l'ottenimento delle informazioni richieste e il test passa. Tali dati vengono quindi restituiti al chiamante, violando il vincolo direzionale imposto dalla configurazione.

Test 17: L'operazione, seppur richiedendo una length nulla, lancia una eccezione poiché il buffer source è nullo. Questo non è un errore implementativo. Ci fa capire che il controllo sul buffer source avviene prima del controllo della dimensione di lettura richiesta.

Test 19: Il test ha esito negativo a causa del mancato sollevamento dell'eccezione attesa. A fronte di un parametro di lunghezza negativo, il sistema gestisce l'errore non leggendo alcun byte, come se la dimensione richiesta fosse 0. Anche questo non viene considerato come errore implementativo ma una mancata documentazione. Dopo queste osservazioni si è proceduto ad allineare i casi di test al comportamento osservato, assumendo la logica implementata come specifica di riferimento. Al contrario, il fallimento del test 5 è una condizione Border-line che dovrebbe essere specificatamente documentata. Essendo il controllo della modalità di apertura un controllo avanzato, spesso non richiesto in queste implementazioni interne, è stato classificato come un difetto di implementazione dei Test (*Tabella 12*). Inoltre, per le considerazioni riguardo i fallimenti dell'inizializzazione, testiamo anche le seguenti configurazioni (*Tabella 13*). Da specificare che, osservando la documentazione⁵, l'allocatore viene utilizzato per istanziare solamente il buffer di scrittura poiché il costruttore del `BufferedReadChannel` accetta come parametri solo il `fileChannel` di riferimento e la dimensione del Buffer di lettura richiesta. Quindi ci aspettiamo che, in questo caso, utilizzando allocatori invalidi l'operazione di lettura rimanga consistente. Tutti i test prodotti in *Tabella 13* hanno un esito positivo.

Write

Per quanto riguarda l'operazione di write abbiamo ottenuto i seguenti risultati con 13 failure su 48 test eseguiti:

Test 4: Non genera eccezione, piuttosto i dati vengono scritti correttamente sul `writeBuffer`. Questo, in mancanza di documentazione specifica, viene valutato come una considerazione del test sbagliata.

Test 5: Nonostante venga correttamente sollevata l'eccezione per il file chiuso (impossibilità di effettuare il flush dei dati) lo stato del `writeBuffer` non è inalterato. C'è una footprint. Tutti i dati che sono stati scritti all'interno del buffer prima di effettuare la chiamata al flush su file, rimangono all'interno del buffer. Questo è un chiaro errore implementativo. Come abbiamo visto nei test della funzionalità read si può leggere direttamente dal `writeBuffer`. Inoltre, quei dati non potranno mai essere persistiti.

Test 46: Non genera una eccezione, piuttosto internamente viene applicata una sanitizzazione del valore di `unpersistedBytesBound` in cui viene assunto come valore 0 (senza meccanismo di threshold automatico).

Test 7, 9, 10, 11, 24, 26, 27, 28, 40, 42: Entrano in un loop infinito. Tutti i test sono accomunati dalla scrittura su un `writeBuffer` di dimensione 0. Questo è un problema di implementazione che vediamo all'interno del ciclo while del codice della write. Se il buffer di scrittura ha dimensione 0 e ci sono dati da scrivere, `bytesToCopy` è sempre pari a 0 e il ciclo while non termina.

Vengono di conseguenza adattati i test 4 e 46 (*Tabella 14*). Vengono integrati i test derivanti dall'inizializzazione (*Tabella 15*) dove anche nel test 50 otteniamo lo stesso errore del test 5. Viene ritornata una eccezione per l'impossibilità di scrivere sul file ma il `writeBuffer` rimane sporco.

2.2.3 Prima iterazione con Jacoco

L'esecuzione della suite di test iniziale e la successiva analisi tramite il tool JaCoCo hanno evidenziato una copertura dello statement dell'89% e dei branch dell'82% (*Immagine 2*). Sebbene la funzionalità di inizializzazione risultasse interamente coperta (*Immagine 3*), l'analisi ha rivelato che alcune combinazioni di valori di input e stati del sistema non erano state incluse nella definizione iniziale delle classi di equivalenza. Utilizzando i dati di copertura come feedback per il raffinamento del test design, sono state identificate e testate le seguenti partizioni mancanti. Nella category partition dell'operazione di lettura (*Immagine 5*) il parametro `position` era stato partizionato senza la considerazione dello stato del buffer di lettura. Tutti i test che abbiamo progettato considerano un'unica operazione di lettura, senza considerare possibili letture sequenziali. L'analisi ha suggerito l'introduzione di una nuova classe di equivalenza per le letture "all'indietro", ovvero quando il valore di `pos` è inferiore all'indice di inizio del buffer di lettura corrente. Questo scenario serve ad invalidare il contenuto del buffer e a rileggere dal disco. Un altro scenario limite che non è stato considerato è la lettura a fine file in presenza di un allocatore di memoria nullo, che porta ad avere un `writeBuffer` assente. Per quanto riguarda le operazioni di scrittura (*Immagine 4*), sebbene il metodo `write` presenti una copertura completa, l'analisi ha rivelato lacune negli altri metodi coordinati. Nello specifico, per il metodo `forceWrite` (*Immagine 6*) è emerso che la partizione relativa al valore 0 per il parametro `unpersistedBytesBound` non era stata isolata correttamente nel contesto dell'invocazione esplicita. Sebbene questo valore disabiliti tecnicamente il buffering automatico, il metodo `force()` pubblico può essere invocato e deve comunque garantire il passaggio del comando al canale sottostante, indipendentemente dallo stato del conteggio dei byte. L'analisi del metodo `flush` (*Immagine 7*) ha evidenziato una lacuna nella modellazione del comportamento del `fileChannel`. Fino a quel momento, si era assunto implicitamente che il `FileChannel` scrivesse sempre tutti i byte in un'unica operazione. È stata quindi introdotta una nuova partizione per simulare il comportamento di Scrittura Parziale da parte del sistema operativo, costringendo il `BufferedChannel` a iterare il ciclo di scrittura fino al completo svuotamento del buffer. L'integrazione di questi scenari nella classe dedicata `BufferedChannelJacocoTest` descritti in *Tabella 16* ha permesso di validare la robustezza del sistema su partizioni di input e condizioni ambientali precedentemente trascurate, elevando le metriche finali al 90% di statement coverage e al 92% di branch coverage (*Immagine 8*).

2.2.4 Seconda iterazione con Pitest

L'esecuzione dell'analisi delle mutazioni tramite il framework PIT ha evidenziato un livello di copertura del 74% (*Immagine 9*). Questo risultato, sebbene positivo, ha offerto l'opportunità di indagare l'efficacia della suite di test e verificare la solidità del lavoro svolto in relazione alla capacità di rilevamento dei difetti. Dall'analisi dettagliata dei mutanti sopravvissuti è emerso un pattern interessante legato ai valori di input utilizzati nei test funzionali standard. In particolare un mutante relativo al campo `writeBufferStartPosition` nella funzione di inizializzazione (*Immagine 10*) sopravviveva perché, nei test precedenti, il canale veniva inizializzato alla posizione 0. Poiché il valore di default di un `AtomicLong` è

⁵ <https://bookkeeper.apache.org/docs/latest/api/javadoc/org/apache/bookkeeper/bookie/BufferedReadChannel.html>

proprio 0, la rimozione dell'istruzione di assegnazione risultava invisibile ai test (il valore atteso e quello effettivo coincidevano "per coincidenza"). Per indirizzare queste specificità e incrementare il mutation score, è stata adottata una duplice strategia. Da un lato è stata introdotta la classe dedicata `BufferedChannelPitestIncreaseTest`, progettata per coprire scenari specifici che richiedono offset non nulli e configurazioni ad-hoc descritti in *Tabella 17*. Dall'altro, si è intervenuti direttamente sulla suite di regressione esistente `BufferedChannelWriteTest`: i test parametrizzati sono stati estesi introducendo rigide verifiche sugli invarianti di stato (controllo dei puntatori `writeBufferStartPosition` e `unpersistedBytes`), permettendo così di rilevare e uccidere sistematicamente i mutanti legati all'aggiornamento dei puntatori post-flush che in precedenza sopravvivevano silenziosamente.

2.2.5 Generazione automatica dei Test

Randoop: test randomici

Nonostante non sia stato impostato alcun limite di tempo, così da permettere al tool un'esplorazione approfondita, la generazione è risultata problematica a causa dell'incapacità del framework di istanziare autonomamente le classi utilizzate come parametri nei metodi di `BufferedChannel`. Per ovviare a tale limite, sono stati implementati diversi metodi helper all'interno della classe `RandoopDataFactory` (nel package `org.apache.bookkeeper.testutils`) per facilitare l'istanziamento dei parametri necessari. Sebbene Randoop sia riuscito a utilizzare correttamente gli helper relativi a `ByteBufAllocator` e ai buffer di dati, l'integrazione di quelli preposti alla fornitura di `FileChannel` ha causato uno stallone sistematico nel processo di generazione. Inizialmente, si è ipotizzato che tale criticità derivasse dal numero limitato di descrittori di file associabili a ciascun processo, data l'elevata frequenza di test eseguiti al secondo dal tool. Per risolvere il problema, è stata sviluppata e fornita a Randoop un'implementazione in memoria del canale, denominata `InMemoryFileChannel`. Tuttavia, anche in questo scenario lo strumento è incorso in uno stallone, riconducibile con ogni probabilità a un consumo eccessivo di memoria RAM. Di conseguenza, i test generati con successo, sebbene numerosi, hanno dovuto escludere l'utilizzo degli helper per i filechannel; ciò ha comportato la creazione di test focalizzati prevalentemente sul costruttore e sulla gestione di parametri nulli, risultando di fatto poco rilevanti.

Evosuite

Allo stesso modo, per evosuite sono stati forniti gli stessi helper in maniera tale da agevolare la creazione di classi parametri di `bufferedChannel`. A valle dell'esecuzione, il tool ha prodotto una suite composta da 30 casi di test. L'analisi delle metriche, effettuata tramite JaCoCo, riporta uno Statement Coverage del 62% e un Branch Coverage del 40% (*Immagine 15*), mentre il punteggio di Mutation Coverage si attesta al 30% (*Immagine 16*). Sebbene i risultati siano leggermente migliori rispetto ai precedenti, l'efficacia qualitativa della suite rimane limitata. Dall'analisi dettagliata della copertura emerge infatti una distribuzione sbilanciata: i test generati si concentrano prevalentemente su metodi accessori e di configurazione. Al contrario, la *core logic* della classe, rappresentata dalle operazioni critiche di I/O, evidenzia una copertura marginale che si aggira intorno al 16%, lasciando di fatto non verificate le parti più complesse e rilevanti dell'implementazione.

LLM: Guided ToT Approach

Per l'attività di generazione automatica mediante LLM (Gemini Pro), è stato adottato il pattern di prompting Guided Tree-of-Thoughts (ToT) (*Prompt LLM 1*). Questa scelta architetturale, che simula la collaborazione tra molteplici esperti di dominio, si è rivelata determinante per gestire la complessità del task e raggiungere una cardinalità di test comparabile a quella manuale. Il workflow è stato ottimizzato suddividendo i passaggi logici previsti dal pattern in due macro-fasi distinte all'interno della stessa sessione. Nella fase preliminare di analisi, anziché procedere alla codifica immediata, è stata generata una lista strutturata di cento scenari di test, focalizzandosi sull'identificazione di edge cases e scenari di eccezione. Successivamente, si è proceduto alla generazione incrementale del codice. Dopo la quinta esecuzione di questo approccio, i raw generated tests prodotti hanno presentato un tasso di errore minimo (7 errori sintattici), che sono stati risolti applicando un meccanismo di validation e repair interattivo utilizzando la stessa sessione. I risultati ottenuti sono Statement Coverage e Branch Coverage al 100% (*Immagine 17*). Per quanto concerne il Mutation Testing, è stato raggiunto un mutation score del 92% (*Immagine 18*); l'analisi dei mutanti sopravvissuti evidenzia tuttavia problematiche per le funzionalità più complesse, come il metodo `read` (*Immagine 19*). Una valutazione a posteriori evidenzia come la strategia di fornire l'intero codice sorgente in input, unita alla richiesta esplicita di cento casi di test, sia stata determinante per saturare la copertura strutturale. Tuttavia, questo approccio mostra un limite metodologico: il modello trascura la documentazione e le specifiche logiche applicative, effettuando un'analisi puramente White Box guidata interamente dall'implementazione visibile. Questo comporta il rischio che, in presenza di un'implementazione errata, i test generati si limitino a specchiare il comportamento del codice invece di rilevarne la discrepanza rispetto ai requisiti attesi.

2.2.6 Conclusione

L'aggregazione delle diverse suite di test ha permesso di raggiungere la saturazione delle metriche di copertura, ottenendo un 100% di Statement Coverage e Branch Coverage (*Immagine 20*). Anche l'analisi della robustezza ha prodotto risultati eccellenti, con un Mutation Coverage del 98% (*Immagine 21*); l'unico mutante sopravvissuto riguarda l'operazione `clear` (*Immagine 22*), un caso marginale che non inficia la stabilità complessiva del componente.

Per quanto riguarda la stima della Reliability, il calcolo è stato effettuato su un campione significativo di 220 casi di test (120 manuali e 100 generati tramite LLM). Dal computo sono stati esclusi i test prodotti da EvoSuite e Randoop, in quanto la loro scarsa focalizzazione sulla logica di business non offriva un contributo rilevante alla validazione funzionale. Adottando un profilo operativo con probabilità di utilizzo uniforme e registrando 13 test falliti sul totale considerato, il valore finale di reliability si attesta allo 0.94.

2.3 JournalChannel

La classe JournalChannel ricopre un ruolo fondamentale nella gestione dell'Input/Output per i file di Journal, noti anche come Write Ahead Log. In linea con la documentazione ufficiale⁶, questa classe agisce come un wrapper intorno al BufferedChannel, arricchendolo con logiche specifiche per la gestione degli header e il controllo delle versioni del formato dei file. Operativamente, la classe gestisce due flussi principali: l'inizializzazione per la scrittura, durante la normale attività del Bookie per la creazione di nuovi log, e l'apertura in modalità di recupero, necessaria per garantire l'accesso in lettura ai file storici durante le procedure di recovery e scansione. Di conseguenza, l'attività di testing si focalizzerà sulla verifica della corretta apertura di un JournalChannel, sia nel caso di creazione di un nuovo file, sia nel caso di recupero di un file esistente.

2.3.1 Test Manuali

APERTURA DI UN JOURNALCHANNEL

L'operazione che implementa la funzionalità di apertura di un Journal Channel è il costruttore della classe stessa. Viene considerato quindi il costruttore che prevede più parametri per proseguire con l'approccio di category partition.

Identificazione dei Parametri

Come riportato in Reference>Configuration>Journal Settings⁷ :

- File journalDirectory: Rappresenta il percorso della directory nel file system locale dove vengono fisicamente persistiti i file di log.
- long logId: Identificativo numerico univoco assegnato al file di log corrente; viene utilizzato per generare il nome del file.
- long preAllocSize: Dimensione in byte da pre-allocare in caso di necessità per la scrittura.
- int writeBufferSize: Definisce la capacità del buffer in memoria (gestito internamente dal BufferedChannel) utilizzato per accumulare le entry prima dell'effettivo flush su disco.
- int journalAlignSize: Specifica l'allineamento in byte per le operazioni di I/O (padding).
- long position: Indica l'offset assoluto (in byte) all'interno del file da cui il canale deve iniziare le operazioni di lettura o scrittura.
- Boolean fRemoveFromPageCache: Flag che, se abilitato, istruisce il sistema operativo a rimuovere le pagine appena scritte dalla Page Cache.
- int formatVersionToWrite: Determina la versione del protocollo di formattazione del Journal.
- Journal.BufferedChannelBuilder bcBuilder: Istanza del builder necessaria per configurare e costruire l'oggetto BufferedChannel sottostante.
- FileChannelProvider provider: Provider per l'apertura del file journal.
- Long toReplaceLogId: Identificativo di un eventuale file di log preesistente che deve essere sostituito dal nuovo canale.

Category Partition

La modalità operativa di JournalChannel è determinata in modo implicito dalla combinazione tra lo stato di persistenza del file system e la configurazione dei parametri logId e toReplaceLogId. Si definisce una apertura di sola lettura qualora il logId corrisponda a un file già esistente; al contrario, si ricade nella apertura in scrittura sia in caso di creazione ex-novo (quando il logId non esiste), sia in caso di riuso (quando un file esistente viene rinominato tramite toReplaceLogId). Tale distinzione influenza direttamente il dominio del parametro position, il cui valore assume una validità differente. Questi valori permettono di isolare le classi di equivalenza e renderle mutuamente esclusive. La category partition in esame è consultabile alla *Tabella 18* in Appendice.

Boundary Analysis

Successivamente si è passati alla Boundary Analysis per le categorie prima descritte. Il risultato è consultabile in Appendice alla *Tabella 19*.

Test

Per la progettazione della suite di test è stato adottato un approccio monodimensionale, motivato dall'elevato numero di parametri coinvolti. Il dettaglio dei casi di test definiti è riportato nella *Tabella 20*, mentre i corrispondenti risultati attesi sono documentati nella *Tabella 21*.

2.3.2 Risultati dei Test

L'analisi dei test manuali sulla classe JournalChannel ha portato alla luce diverse difformità significative rispetto ai requisiti funzionali attesi. In prima istanza, i test 3, 4 e 5 hanno evidenziato l'impossibilità di inizializzare file di log utilizzando versioni del protocollo legacy (v1, v2, v3). Sebbene l'analisi del codice sorgente abbia confermato la presenza di una costante che impone la versione v4 come soglia minima per la scrittura, tale vincolo non trova riscontro nella documentazione ufficiale, configurando quindi un disallineamento informativo piuttosto che un difetto software in senso stretto. Di natura diversa è invece il fallimento registrato nel test 6, classificabile come un chiaro errore implementativo: il sistema consente la creazione di file con numeri di versione non ancora supportati (es. v7), pur mantenendo la struttura dell'header della v6, esponendo così il componente a rischi di compatibilità futura.

Sul fronte della logica di persistenza, il test 9 ha rilevato un difetto critico nella gestione delle configurazioni con allineamento coincidente alla preAlloc/alignSize (512/512 byte), scenario che causa erroneamente la sovrascrittura dell'header anziché garantirne la persistenza. Parallelamente, è emersa una grave mancanza di atomicità nel processo di creazione del canale: come evidenziato dai test 10, 11, 18, 19 e 20, il file viene generato fisicamente nel file system

⁶ <https://bookkeeper.apache.org/docs/latest/api/javadoc/org/apache/bookkeeper/bookie/JournalChannel.html>

⁷ <https://bookkeeper.apache.org/docs/reference/config/>

anche qualora l'operazione venga interrotta da eccezioni dovute a parametri invalidi (quali `preAllocSize`, `fileChannelProvider` o `bufferedChannelBuilder`), lasciando potenzialmente artefatti corrotti su disco. Una simile carenza nella validazione degli input è confermata dal test 17, dove il passaggio di un `journalDirectory` nullo non solleva l'eccezione attesa, ma provoca la creazione imprevista del file nella directory di lavoro corrente. Per quanto concerne le politiche di riutilizzo delle risorse, analizzate nel test 24, si riscontra che il file preesistente non viene troncato qualora la nuova dimensione di pre-allocazione risulti inferiore a quella attuale, introducendo rischi di inconsistenza dei dati; test supplementari hanno confermato che il riuso avviene correttamente solo in caso di perfetta coincidenza dimensionale. Infine, il test 33 sancisce una violazione del principio di incapsulamento durante le operazioni di lettura: il sistema espone erroneamente la sezione riservata ai metadati, compromettendo la trasparenza dell'accesso e mescolando header e payload utente.

2.3.3 Prima iterazione con Jacoco

L'applicazione della strategia di Category Partition ha fornito una solida base di partenza, permettendo di raggiungere una copertura dello statement del 71% e dei branch del 75% (*Immagine 23*). Tuttavia, l'analisi della copertura del codice condotta tramite JaCoCo ha evidenziato alcune caratteristiche dei parametri che non erano state considerate nella fase iniziale e, di conseguenza, non rientravano nella Category Partition definita. Osservando i dati nel dettaglio, mentre la copertura per la funzionalità `writeHeader` risultava già completa (*Immagine 26*), quella relativa al costruttore (*Immagine 24 e Immagine 25*) richiedeva l'introduzione di ulteriori casi di test. In particolare, non era stata valutata la condizione esposta dal `FileChannelProvider` tramite il metodo `supportReuseFile`, il che ha portato all'identificazione di una nuova categoria per il Provider. È stato quindi aggiunto un test in cui la richiesta di sostituzione di un log (`toReplaceLogId`), pur essendo formalmente corretta, viene rifiutata dalla logica interna del provider, permettendo così di validare il corretto flusso logico (fallback) che impedisce la sovrascrittura o rinomina non supportata. Un'altra casistica non considerata riguarda il `FileChannel` restituito dal provider, che potrebbe riscontrare problemi durante il posizionamento (seek) generando una `IOException`; è stato perciò implementato un test per verificare la corretta propagazione di tale eccezione. Inoltre, poiché i test iniziali assumevano condizioni isolate, è stato introdotto uno scenario per simulare una *race condition* in cui due istanze competono per il medesimo logId. Attraverso l'uso di mock, si è riprodotto il caso in cui il file appare nel sistema nell'istante critico tra il controllo di esistenza e la creazione, verificando la resilienza del costruttore agli stati transitori del file system. Infine, per colmare una lacuna nella progettazione, sono stati inseriti test specifici sull'apertura di un `JournalChannel` con versioni di protocollo inferiori o superiori a quelle supportate (distinguendo questo caso dall'errore di Magic Word errato), garantendo che la logica di validazione degli header risponda correttamente agli edge cases. Grazie a questo approccio complementare il costruttore ha raggiunto la piena copertura (100%) per le metriche di Statement e Branch Coverage. L'intera suite di test è disponibile all'interno del file `JournalChannelJacocoTest.java`, situato nella directory `test-manual`. I risultati finali sono riepilogati nell'immagine seguente (*Immagine 27*). I test aggiunti sono consultabili in *Tabella 22*.

2.3.4 Seconda iterazione con Pitest

L'analisi della mutation coverage, condotta tramite il tool Pitest, ha evidenziato inizialmente un indice di copertura totale pari al 66% (*Immagine 28*). Un esame più approfondito delle funzionalità testate ha rivelato la presenza di tre mutanti sopravvissuti localizzati all'interno del costruttore (*Immagine 29 e Immagine 30*) e nel metodo `writeHeader` (*Immagine 31*). Il primo mutante, identificato alla riga 168, riguardava l'invocazione del metodo `notifyRename`, una callback interna al provider precedentemente non verificata; l'eliminazione di questo mutante è stata ottenuta introducendo la verifica esplicita di tale chiamata tramite uno `Spy`, precedentemente assente nel Test 24. Il secondo caso critico, situato alla riga 252, persisteva poiché l'eventuale errato dimensionamento dell'header veniva mascherato dal meccanismo di pre-allocazione del costruttore, il quale inseriva comunque il padding di zeri nel file fisico. La strategia risolutiva ha previsto la verifica della posizione del cursore di scrittura nell'istante esatto della creazione del `BufferedChannel`; per garantire l'isolamento del test, è stato introdotto un Mock del builder che intercetta il `FileChannel` passato come parametro (altrimenti inaccessibile poiché interno al costruttore) verificando che la sua posizione corrisponda esattamente alla dimensione attesa dell'header. Infine, il mutante alla riga 254 è stato classificato come "equivalente", in quanto l'istruzione `ZeroBuffer.put(bb)` risulta ridondante rispetto alla garanzia di inizializzazione a zero già offerta nativamente dal metodo `ByteBuffer.allocate`. L'integrazione di questo nuovo scenario di test, consultabile nella classe `JournalChannelPitestIncreaseTest.java` e in *Tabella 23*, ha permesso di incrementare la mutation coverage finale al 70% (*Immagine 32*) con l'uccisione dei precedenti mutanti (*Immagine 33 e Immagine 34*).

2.3.5 Generazione automatica dei Test

Randoop

L'applicazione dello strumento di test generation Randoop ha prodotto risultati complessivamente modesti. Anche in questo caso è stata integrata come classe di utility `JournalTestUtility` per la corretta istanziazione delle risorse necessarie ai test, analogamente a quanto implementato per la componente `BufferedChannel`. La suite generata di 3097 test ha raggiunto una statement coverage del 58% e una branch coverage del 46% (*Immagine 35*). Tale copertura, nonostante la mole di test generati, risulta tuttavia poco significativa, in quanto le lacune principali si concentrano proprio nei metodi core della classe, quali il costruttore (*Immagine 36 e Immagine 37*) e la funzione `writeHeader` (*Immagine 38*). Ancora più critici sono i dati relativi alla mutation coverage, attestatasi a un deludente 32% (*Immagine 39*) con una capacità di rilevamento mutanti pressoché nulla nelle logiche del costruttore (*Immagine 40 e Immagine 41*). Una criticità strutturale emersa durante l'esecuzione riguarda inoltre la gestione del file system: la suite ha prodotto numerose "footprint" residue nella directory di lavoro sotto forma di file `.txn`, evidenziando un'inadeguata gestione del ciclo di vita delle risorse generate. Questo comportamento non solo compromette la stabilità dell'ambiente di prova, ma rende i test prodotti

incompatibili con i flussi di Continuous Testing, dove la pulizia e l'isolamento dell'ambiente di esecuzione sono requisiti fondamentali.

EvoSuite

EvoSuite ha portato alla generazione di una suite composta da 43 test, evidenziando inizialmente un'incongruenza logica nel caso di test 21; quest'ultimo, infatti, falliva nel tentativo di asserire la creazione di un `JournalChannel` con versione dell'header pari a 2, configurazione non supportata dalla logica interna della classe (come identificato nei test manuali). A seguito di una correzione manuale per allineare il test alle specifiche, i risultati metrici hanno mostrato un sensibile miglioramento rispetto alle altre classi testate, attestando la statement coverage all'80% e la branch coverage al 74% (*Immagine 42*). Sul fronte della mutation coverage è stato raggiunto un valore complessivo dell'85% (*Immagine 43*), un dato certamente positivo che tuttavia maschera una criticità localizzata: un'analisi granulare dei report rivela infatti che le mutazioni residenti nel costruttore rimangono per la maggior parte non identificate (*Immagine 44*). Questo suggerisce che, nonostante l'elevata copertura strutturale, la capacità del tool di verificare rigorosamente le delicate fasi di inizializzazione e validazione dei metadati rimanga parziale, rendendo la suite non adeguata al testing della classe.

LLM: Zero-shot

La metodologia di test generation applicata a `JournalChannel` ha previsto l'impiego del modello Gemini Pro attraverso una tecnica di prompting *zero-shot*. Il protocollo sperimentale ha previsto l'esecuzione di dieci interazioni indipendenti, ciascuna basata su una singola richiesta diretta a generare una suite composta da 40 tests (*Prompt LLM 2*). Si è riscontrata una costante divergenza quantitativa rispetto al prompt, con la produzione sistematica di un numero inferiore di casi di test rispetto a quanto richiesto; nello specifico, l'ottava iterazione analizzata ha restituito una suite di 20 test. Nonostante la riduzione numerica, i risultati strutturali hanno mostrato una statement coverage del 90% e una branch coverage del 75% (*Immagine 45*). Tuttavia, permane una criticità qualitativa già riscontrata con altri strumenti: le lacune di copertura si concentrano prevalentemente nelle sezioni critiche del costruttore (*Immagine 46* e *Immagine 47*) e della funzione `writeHeader` (*Immagine 48*). Tale limite si riflette anche nei dati relativi alla mutation coverage, attestatasi al 60% (*Immagine 49*), dove i mutanti sopravvissuti confermano la difficoltà del modello nel validare rigorosamente le logiche di inizializzazione (*Immagine 50* e *Immagine 51*) e di gestione dell'header del journal (*Immagine 52*).

2.3.6 Conclusione

L'integrazione delle diverse suite di test analizzate ha permesso di raggiungere una Statement coverage del 97% e una Branch coverage del 92% (*Immagine 53*). L'analisi complessiva della mutation coverage si è attestata all'81% (*Immagine 54*). Per quanto concerne la stima della Reliability della classe `JournalChannel`, la valutazione è stata condotta sull'intera suite di test prodotta (composta da 52 test manuali, 20 generati tramite LLM e 43 prodotti da EvoSuite), adottando un profilo operativo con probabilità di utilizzo uniforme. Su un totale di 115 casi di test, 105 sono stati eseguiti con successo, portando a una Reliability stimata di 0.9130 (91,30%). Le 10 rilevazioni negative (pari all'8,7% del totale) sono riconducibili ai fallimenti sistematici discussi nell'analisi dei risultati dei test manuali, in particolare alle difformità implementative nella gestione del versioning del protocollo, alla mancata atomicità nelle operazioni di creazione del file e agli errori di sovrascrittura in configurazioni specifiche di allineamento.

3 OpenJPA

3.1 Descrizione del sistema

Apache OpenJPA⁸ è un middleware Java che implementa la specifica Jakarta Persistence API (JPA), agendo come strato di astrazione Object-Relational Mapping (ORM). Tra le sue funzionalità architetturali, risultano determinanti per la presente attività di testing: la gestione automatizzata del ciclo di vita delle entità, supportata da meccanismi di proxying per il monitoraggio degli oggetti, e l'integrazione di sistemi di caching avanzati per l'ottimizzazione delle performance. In questo contesto, l'attività di testing si è focalizzata sulle classi **CacheMap** e **ProxyManagerImpl**, in quanto componenti critici deputati rispettivamente alla gestione delle cache e alla generazione trasparente dei proxy per le entità persistenti.

3.2 CacheMap

La classe `CacheMap`⁹, residente nel modulo `openjpa-kernel`, estende `java.lang.Object`¹⁰ e implementa l'interfaccia `java.util.Map`¹¹. Essa fornisce una mappa dati ibrida progettata per la gestione della memoria sensibile al carico.

La struttura della `CacheMap` non è monolitica, ma orchestra tre distinti livelli di memorizzazione:

- **Pinned Map (Livello Prioritario):** Contiene le entry marcate esplicitamente come "pinned". Queste entry mantengono strong references, garantendo che non vengano mai rimosse né dalle politiche di eviction interne né dal Garbage Collector (GC) della JVM, indipendentemente dal carico di memoria o dalle dimensioni della cache.
- **Hard Map (Cache Primaria):** Una mappa a dimensione fissa (configurabile) che mantiene strong references agli oggetti. Rappresenta il working set attivo. Le entry in questo livello sono protette dalla Garbage Collection.
- **Soft Map (Cache di Overflow):** Una struttura di backup che utilizza `java.lang.ref.SoftReference`. Funge da destinazione per le entry escluse dalla Hard Map. Il contenuto di questo livello è volatile: può essere reclamato dal Garbage Collector in caso di scarsità di memoria.

La politica di eviction predefinita è casuale (Random), come da cap. 10¹² della documentazione, con supporto opzionale per l'algoritmo LRU (Least Recently Used).

⁸ <https://openjpa.apache.org/documentation.html>

⁹ <https://openjpa.apache.org/builds/4.1.1/apidocs/org/apache/openjpa/util/CacheMap.html>

¹⁰ <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Object.html>

¹¹ <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/Map.html?is-external=true>

¹² https://openjpa.apache.org/builds/3.1.2/apache-openjpa/docs/ref_guide_caching.html#ref_guide_data_cache

Le funzionalità principali individuate per il testing sono:

- Inizializzazione: meccanismo di istanziazione dei tre livelli descritti precedentemente.
- Put: operazione di inserimento di una entry all'interno della cacheMap.
- Pin: "Pinning" di una chiave.
- Remove: rimozione di una coppia chiave valore all'interno della cache.

3.2.1 Test Manuali

INIZIALIZZAZIONE DELLA CACHEMAP

La prima fase di analisi è dedicata alla verifica dei meccanismi di istanziazione della classe CacheMap, con specifico riferimento alla configurazione della struttura interna a tre livelli. In questo contesto, l'attività di testing prescinde dalla correttezza operativa dei metodi successivi (es. operazioni di lettura/scrittura), focalizzandosi esclusivamente sulla logica di inizializzazione. L'obiettivo è validare i confini di ammissibilità delle configurazioni, accertando che:

- Le configurazioni conformi alle specifiche portino alla corretta allocazione delle strutture in memoria.
- Le configurazioni definite come non ammissibili non diano luogo all'istanziazione dell'oggetto o vengano gestite tramite le opportune eccezioni/stati di errore.

Tale approccio permette di isolare la verifica strutturale da quella funzionale, garantendo che i vincoli architetturici di Apache OpenJPA siano rispettati ancor prima dell'utilizzo operativo della cache.

Descrizione dei parametri

Per la definizione delle variabili di test tramite Category Partition e Boundary Value Analysis, si è assunto come riferimento il costruttore canonico, in quanto espone il sovrainsieme completo dei parametri configurabili:

```
public CacheMap(boolean lru, int max, int size, float load, int concurrencyLevel)
```

Sebbene la completa analisi richiederebbe la validazione di ogni overload, si è adottata una strategia di prioritizzazione basata sul rischio, considerando che le analisi automatiche andranno a coprire queste lacune. Poiché i costruttori secondari agiscono tipicamente come wrapper di convenienza delegando l'inizializzazione a quello principale, l'analisi manuale si concentra esclusivamente su quest'ultimo. In assenza di specifiche dettagliate nella documentazione diretta di CacheMap, la semantica di alcuni parametri di configurazione è stata inferita analizzando la classe correlata ConcurrentHashMap¹³, sulla quale si basa l'implementazione sottostante. Di seguito si riporta la definizione funzionale delle variabili identificate:

- boolean lru: Flag di configurazione della politica di espulsione (eviction policy). Determina se la gestione della cache debba seguire un algoritmo LRU (Least Recently Used) o una politica di rimozione casuale (Random).
- int max: Definisce la capacità massima (upper bound) della mappa HardMap.
- int size: Stabilisce la capacità iniziale allocata per la HardMap all'atto dell'istanziazione.
- float load: Fattore di carico (load factor). Rappresenta la soglia di riempimento che, una volta superata, innesca il ridimensionamento automatico della struttura (re-hashing), analogamente a quanto avviene nelle implementazioni standard di HashMap.

L'analisi della documentazione ufficiale non ha evidenziato riferimenti espliciti relativi al funzionamento o al dominio di validità del parametro concurrencyLevel. Di conseguenza, la definizione delle partizioni per questo input (Category Partition) non si baserà su specifiche formali, bensì sulle proprietà intrinseche del tipo di dato (int) e su inferenze semantiche derivanti dalla sua nomenclatura.

Category Partition

I parametri relativi alla dimensione iniziale size e alla capacità massima max sono stati trattati in maniera congiunta. Ciò è dettato dal vincolo di consistenza interna ipotizzato che impone che la dimensione iniziale di allocazione non possa eccedere la soglia massima consentita. Configurazioni che violano tale disuguaglianza sono considerate come non istanziabili. Il parametro load è stato interpretato semanticamente come un coefficiente percentuale di riempimento. Di conseguenza, il dominio di validità è stato ristretto all'intervallo matematico (0, 1]. In mancanza di specifiche dettagliate, si assume che concurrencyLevel rappresenti il grado di parallelismo supportato (numero stimato di thread concorrenti). Pertanto, si considerano invalide le configurazioni che presentano valori non positivi, in quanto privi di significato logico. La tabella della category partition è visionabile in Appendice alla *Tabella 24*.

Boundary Analysis

Il dettaglio dell'analisi dei Boundary Values, comprensivo delle motivazioni tecniche per la selezione di ciascun valore di frontiera, è riportato nella *Tabella 25*.

Test

Per la progettazione dei test si è adottata una strategia monodimensionale per la riduzione della complessità combinatoria. Tale approccio si fonda sull'ipotesi di indipendenza funzionale tra parametri che regolano aspetti distinti del sistema. Ad esempio, si assume che i meccanismi di dimensionamento (es. max/size) operino in modo disaccoppiato rispetto alla gestione della concorrenza (concurrencyLevel). Di conseguenza, le variabili sono state analizzate separatamente o per gruppi logici, escludendo la verifica di interazioni incrociate ritenute non significative così come possiamo consultare nella *Tabella 26* in Appendice.

PUT

Il metodo put è deputato all'inserimento o all'aggiornamento delle associazioni chiave-valore all'interno della cache. La strategia di testing è stata definita isolando le responsabilità funzionali della classe CacheMap rispetto all'implementazione sottostante. A tal fine, è fondamentale operare una distinzione architetturale tra due meccanismi:

¹³ <http://openjpa.apache.org/builds/4.1.1/apidocs/org/apache/openjpa/lib/util/concurrent/ConcurrentHashMap.html>

- Ridimensionamento Dinamico (Rehashing): L'espansione della capacità basata sul fattore di carico è gestita nativamente dalle librerie standard di Java (HashMap). Tale comportamento, essendo trasparente e consolidato, è considerato fuori dal perimetro della presente analisi.
- Politiche di Eviction: La gestione della saturazione della cache, governata dagli algoritmi LRU (Least Recently Used) o Random, costituisce una logica proprietaria implementata specificamente in CacheMap.

Di conseguenza, la progettazione dei test case si concentra sulla validazione di quest'ultimo aspetto, verificando che il meccanismo di rimozione delle entry obsolete intervenga correttamente al raggiungimento dei limiti configurati.

Descrizione dei parametri

L'analisi funzionale fa riferimento alla firma del metodo esposto dall'API pubblica:

```
public Object put(Object key, Object value)
```

Tuttavia, poiché la logica di inserimento è strettamente accoppiata alle condizioni di saturazione della memoria, il dominio di input è stato esteso includendo, oltre agli argomenti espliciti, le variabili di stato della CacheMap. Di seguito si riporta la definizione delle grandezze considerate nel modello di test:

- Object Key: Identificatore univoco dell'entry. Svolge un ruolo critico nei meccanismi di indicizzazione, calcolo dell'hash code e successivo recupero del dato.
- Object Value: Il payload informativo da memorizzare nella cache e da associare alla chiave specificata.
- Stato della HardMap: Rappresenta il livello di riempimento attuale (saturazione) della HardMap al momento dell'invocazione. È la variabile determinante per l'attivazione dei meccanismi di eviction.
- boolean LRU: Flag che definisce la politica di sostituzione attiva (Eviction Policy). Determina l'algoritmo utilizzato per selezionare l'elemento da rimuovere in caso di saturazione (Random vs LRU).
- size della SoftMap: Parametro che indica la capacità massima (o lo stato di occupazione) del buffer a Soft Reference, influenzando le logiche di migrazione delle entry tra i livelli della cache.

Category Partition

Un appunto su cui soffermarci è il parametro "size della SoftMap", questo ci indica la capacità della softMap. La documentazione ci riporta che impostando questo valore a -1 la sua size viene impostata senza alcun limite e quindi le chiavi rimarranno al suo interno fin quando il GC non le eliminerà per assenza di memoria. Questo si presta particolarmente bene a testare la funzionalità di Put tramite metodo Black Box. Infatti, se impostato a 0 la sezione delle soft Map non esisterà e potremmo testare quale delle entry è stata evicted. Le partizioni relative alla Key non si limitano a validare l'input, ma verificano la consistenza posizionale dell'entry. L'obiettivo è confermare che le operazioni di put gestiscano correttamente l'aggiornamento o la promozione di chiavi già esistenti nei diversi livelli della gerarchia di cache (es. promozione da Soft a Hard in seguito a un aggiornamento). La distinzione tra valori Immutable e Mutable per il parametro Value è finalizzata a verificare la strategia di memorizzazione adottata da CacheMap. Il test mira a discernere se il salvataggio avvenga by-reference o tramite deep copy. Verificare questo comportamento è cruciale per garantire che eventuali modifiche apportate dall'utente all'oggetto originale *dopo* l'inserimento mantengano lo stato della cache coerente. Le classi di equivalenza identificate per la validazione funzionale del meccanismo di inserimento e le relative partizioni di test sono riepilogate nella *Tabella 27*.

Boundary Analysis

Per la Chiave, oltre agli stati posizionali standard, è stato introdotto lo scenario hashCodeException (tramite mock) per testare la resilienza del sistema a errori di runtime durante l'indicizzazione.

Per quanto concerne il parametro Object Value, la strategia di test mira a verificare la semantica di memorizzazione adottata da CacheMap, discriminando tra persistenza per copia e persistenza per riferimento. A tale scopo, sono state selezionate due tipologie di dati campione:

Oggetti Immutabili: L'uso di stringhe garantisce l'invarianza di stato (operazioni come concat generano nuove istanze senza alterare l'originale).

Oggetti Mutabili: È stata implementata una classe helper ad-hoc denominata MutableObject, dotata di uno stato interno modificabile (nello specifico, un attributo "nome" aggiornabile tramite metodi mutatori). L'impiego di questo oggetto è fondamentale per verificare l'integrità referenziale: il test controlla se le mutazioni applicate all'istanza esternamente alla cache si riflettono sull'entry memorizzata, chiarendo così se la struttura mantiene un puntatore all'oggetto originale o ne crea una copia difensiva. La *Tabella 28* illustra la selezione dei Boundary Values adottati, evidenziando per ciascuno la specifica motivazione tecnica e l'obiettivo del test.

Test

Ai fini della parametrizzazione dei test, il valore della capacità massima (Max) è stato fissato alla costante 10. Tale dimensione, pur mantenendo un basso impatto sulle risorse di memoria, risulta sufficientemente rappresentativa per validare le logiche di saturazione; gli stati della HardMap descritti in tabella sono pertanto calcolati in funzione di tale soglia.

Si evidenzia, tuttavia, una limitazione strutturale nell'approccio Black Box combinato con la politica di eviction Random. A causa della natura non deterministica dell'algoritmo, risulta impossibile prevedere quale specifica entry venga espulsa dalla HardMap. Di conseguenza, non è fattibile riprodurre in modo deterministico la precondizione {Existing Soft} per una chiave specifica, poiché l'assenza di visibilità interna impedisce di stabilire a priori quale elemento transiterà nella struttura a riferimenti deboli. La *Tabella 29* fornisce un riepilogo esaustivo dei test identificati, descrivendo per ciascuno le precondizioni, i valori di input e il comportamento atteso della cache.

PIN

Il metodo pin implementa la logica di permanenza forzata di una specifica associazione chiave-valore. L'invocazione di tale funzionalità altera il ciclo di vita standard dell'entry, conferendole uno stato di immunità rispetto alle politiche di rimpiazzo. Le entry sottoposte a pinning vengono escluse permanentemente dagli algoritmi di rimozione automatica

(sia LRU che Random). Architetturealmente, ciò implica che tali oggetti non saranno mai degradati verso la cache a Soft Reference, né eliminati per liberare risorse in caso di saturazione.

Gli oggetti "pinnati" non contribuiscono al calcolo della dimensione corrente (size) rispetto al limite massimo configurato (max). In pratica, occupano uno spazio "extra" rispetto alla capacità dichiarata della cache.

Il metodo restituisce un valore booleano:

- true: La chiave è stata identificata all'interno della gerarchia di cache (sia essa nella HardMap o recuperata dalla SoftMap) ed è stata correttamente marcata come inamovibile.
- false: La chiave non è presente in alcuna struttura. L'operazione non produce effetti collaterali, evidenziando l'impossibilità strutturale di applicare il pinning preventivo su chiavi non ancora indicizzate.

Descrizione dei parametri

La verifica funzionale si concentra sulla seguente interfaccia pubblica:

```
public boolean pin(Object key)
```

La definizione delle variabili di test per questo metodo è strettamente legata alla proprietà di invarianza della capacità discussa in precedenza. Poiché le chiavi pinned sono escluse dal calcolo della dimensione corrente della HardMap, si configura uno scenario limite critico: la promozione di una entry dalla SoftMap alla HardMap in condizioni di saturazione. In tale circostanza, il sistema deve garantire il re-inserimento dell'oggetto nella memoria hard senza innescare alcuna procedura di eviction, consentendo un temporaneo overflow rispetto al limite configurato.

Le variabili considerate nel modello di test sono le seguenti:

- Object Key: L'identificatore univoco dell'entità da ancorare. La sua collocazione iniziale in memoria (assente, Hard, Soft) determina il flusso di esecuzione e l'esito dell'operazione.
- Stato della Hard Map: Il livello di saturazione della mappa principale al momento dell'invocazione. Questa variabile di stato è determinante per validare che il meccanismo di esenzione dal conteggio operi correttamente (nessuna rimozione forzata) anche quando la struttura ha raggiunto la capacità massima.

Category Partition

La scomposizione del dominio di input e degli stati interni della cache nelle partizioni di equivalenza specifiche per la funzionalità di pin è consultabile nella Tabella 30 in Appendice.

Boundary Analysis

Il dettaglio della Boundary Value Analysis, comprensivo delle motivazioni tecniche che giustificano la scelta di ciascun valore limite per il metodo pin, è riportato nella *Tabella 31*.

Tests

I test case risultanti dal processo di progettazione sono riepilogati nella *Tabella 32*; ogni riga descrive una specifica combinazione di parametri volta a coprire le diverse ramificazioni logiche della funzionalità.

REMOVE

L'analisi funzionale prosegue con la verifica del metodo deputato alla cancellazione esplicita delle entry:

```
public Object remove(Object key)
```

L'obiettivo del test è validare la capacità del metodo di eliminare un'associazione chiave-valore in modo consistente, indipendentemente dalla sua collocazione topologica all'interno della gerarchia di memoria. L'operazione deve garantire la rimozione dell'elemento sia che esso risieda nella HardMap, nella SoftMap, o che sia stato precedentemente ancorato tramite pinning. Un aspetto critico specificato nella documentazione ufficiale riguarda la priorità dell'operazione di cancellazione. Qualora la chiave target risulti pinned all'interno della cache, l'invocazione del metodo remove comporta contestualmente la revoca del vincolo di ancoraggio e la conseguente eliminazione fisica dell'oggetto. In sintesi, l'azione di rimozione è distruttiva e prevale sullo stato di persistenza forzata.

Descrizione dei parametri

In questo caso, l'unico parametro interessante è la chiave da eliminare e il suo stato.

- Object Key: Rappresenta l'identificatore univoco dell'associazione da rimuovere. Svolge il ruolo di selettore di stato all'interno della gerarchia di memoria, poiché la sua posizione (HardMap, SoftMap o PinnedMap) determina il percorso di cancellazione e la necessità di revocare eventuali vincoli di persistenza.

Category Partition

La scomposizione del dominio di input nelle relative classi di equivalenza, mirata a isolare i diversi scenari di rimozione in base alla collocazione della chiave nella gerarchia di memoria, è riassunta nella *Tabella 33*.

Boundary Analysis

A valle della definizione delle categorie, si è proceduto all'individuazione dei valori di frontiera il cui dettaglio è consultabile nella *Tabella 34* in Appendice.

Test

Al fine di garantire che le verifiche riflettano un contesto operativo realistico, ogni scenario di test prevede una fase di pre-configurazione in cui la cache viene inizializzata con 5 entry. Tale accorgimento è fondamentale per validare la coerenza del metodo in presenza di dati preesistenti, evitando che stati di errore critici rimangano silenti a causa di una struttura dati inizialmente vuota. La suite finale dei casi di test per il metodo remove è riportata nella *Tabella 35* con i relativi risultati attesi e gli stati post-condizione.

3.2.2 Risultati dei Test

Inizializzazione della CacheMap

L'esecuzione dei test manuali ha evidenziato diverse discrepanze rispetto al comportamento atteso. Di seguito si riporta il dettaglio puntuale dei casi falliti:

- Test 7: Il parametro ConcurrencyLevel viene ignorato dall'implementazione corrente, rendendolo ininfluenza nella configurazione.

- Test 8: Analogamente al caso precedente, il valore di `ConcurrencyLevel` non viene preso in considerazione dalla logica interna.
- Test 9: A fronte di dimensioni (`size`) negative, interviene un meccanismo di sanitizzazione forzata che imposta arbitrariamente il valore a 500. Comportamento non tracciato in documentazione. Viene di conseguenza modificato il risultato atteso del test.
- Test 10: Il costruttore non solleva l'eccezione attesa nonostante entrambi i parametri di dimensionamento (`max` e `size`) siano impostati a 0. Viene considerata valida l'allocazione di una `cacheMap` con `maxSize` 0. Nelle successive funzionalità saranno aggiunti dei test che verificheranno il comportamento della classe in questa condizione.
- Test 11: Si rileva la mancata generazione di un'eccezione per una dimensione iniziale (`size`) pari a 0 quando la politica è `Random`, evidenziando un'incongruenza rispetto allo scenario `LRU`. Ciò indica che, con la politica `LRU`, la dimensione minima non può essere nulla. Tale divergenza deriva dalle strategie di ridimensionamento: a differenza di una mappa standard, la struttura `LRU` non prevede il ridimensionamento automatico. L'analisi dettagliata della capacità massima verrà approfondita nella sezione dedicata all'operazione di `put`.
- Test 12: Viene applicata una sanitizzazione implicita al valore `max` non specificata in documentazione, che viene impostato di default a `Integer.MAX_VALUE`.
- Test 13: La configurazione con capacità massima pari a 0 è accettata dal sistema come una funzionalità intenzionale anziché un errore. Tale impostazione permette infatti di disabilitare la cache, forzando l'applicazione alla lettura diretta dal database per ogni richiesta. Viene modificato il test di conseguenza.
- Test 14: Il sistema permette di istanziare una mappa con dimensione iniziale superiore alla capacità massima. Sebbene non generi un errore a runtime, tale configurazione rappresenta una *bad practice*.

Alla luce di queste evidenze, la strategia di test per le operazioni di `put` dovrà necessariamente includere e verificare questi stati limite accettati dall'implementazione. Nello specifico, verranno analizzati gli scenari in cui la politica `LRU` è disabilitata (`false`), la dimensione iniziale è nulla (`size = 0`) e la capacità massima è azzerata (`max = 0`).

Put

Durante l'esecuzione dei test funzionali sul metodo `put`, è stato registrato un fallimento nel Test 9, caratterizzato dalla perdita imprevista di un'entry.

L'analisi del codice sorgente ha permesso di ricostruire la dinamica dell'evento, causata da una collisione durante la fase di promozione:

1. Una entry residente nella `SoftMap` viene richiamata e promossa verso la `HardMap`.
2. Poiché la `HardMap` si trova in stato di saturazione, l'inserimento innesca l'eviction dell'elemento meno recente (`LRU`), che viene degradato verso la `SoftMap`.
3. Essendo la `SoftMap` configurata con capacità unitaria (`size=1`) ed essendo tale slot tecnicamente occupato durante la transazione dall'entry che sta risalendo, l'entry degradata non trova spazio disponibile e viene definitivamente scartata.

È stato verificato sperimentalmente che incrementando la capacità della `SoftMap` a 2, il problema di contesa viene risolto, prevenendo la perdita del dato. Tale comportamento non è classificato come errore implementativo. Le entry gestite tramite `Soft References` sono per definizione volatili e prive di vincoli stretti di persistenza (possono essere rimosse arbitrariamente dal Garbage Collector per esigenze di memoria). Tuttavia, il meccanismo di "drop per collisione" rappresenta un dettaglio implementativo rilevante che deve essere esplicitato nella documentazione tecnica, in quanto influenza la predicibilità della cache in configurazioni a bassa memoria. A seguito delle criticità emerse nell'analisi preliminare del costruttore, si è ritenuto necessario estendere la suite con i test in *Tabella 36*, i quali hanno fornito esiti positivi. Che hanno avuto esiti positivi. Tuttavia, in entrambi i casi (Test 10 e 11), l'invocazione dei metodi `containsKey` e `get`, finalizzata a verificare l'effettiva assenza della chiave, ha provocato il sollevamento di una `ArithmeticException`. Tale comportamento evidenzia un errore implementativo nella gestione dei casi in cui la dimensione della mappa è pari a 0.

Pin

L'esecuzione del Test 0, progettato per verificare la robustezza del metodo `pin` a fronte di input invalidi (`key = null`), ha prodotto un esito inatteso. Contrariamente all'aspettativa di un rifiuto esplicito (tramite eccezione), il sistema ha accettato la chiave nulla, memorizzandola con successo all'interno della mappa delle entry pinned associata a un valore placeholder. L'analisi del codice conferma che tale comportamento non è frutto di una scelta progettuale esplicita, bensì una conseguenza diretta della mancanza di controlli preventivi. L'implementazione delega interamente la gestione dell'inserimento alle classi standard delle collezioni Java sottostanti, le quali supportano nativamente l'uso di chiavi null, trasferendo tale permissività alla logica della `CacheMap` senza filtri di validazione.

Remove

L'esecuzione dei test sulla funzionalità `remove` ha portato alla luce una incongruenza critica tra le specifiche dichiarate e il comportamento a runtime. Mentre la documentazione ufficiale stabilisce esplicitamente che la rimozione di una chiave ancorata deve comportare la contestuale revoca del vincolo ("*the pin is cleared*") e l'eliminazione dell'oggetto, l'analisi del codice ha rivelato un'implementazione divergente. Nello specifico, l'operazione di rimozione si limita a settare a null il valore associato alla chiave, mantenendo tuttavia attivo lo stato di pinning. Tale comportamento genera uno stato inconsistente e rappresenta un chiaro errore implementativo, configurandosi come una violazione diretta del contratto funzionale descritto nella documentazione.

3.2.3 Prima iterazione con Jacoco

L'analisi quantitativa della copertura del codice, effettuata tramite il tool JaCoCo, ha fornito una valutazione oggettiva della copertura della suite di test manuali inizialmente progettata. I dati preliminari hanno evidenziato una *statement coverage* del 63% e una *Branch Coverage* del 58% (*Immagine 55*). L'esame dettagliato dei report ha permesso di

identificare alcune classi di equivalenza e stati interni della cache che erano stati inizialmente omessi nella fase di Category Partitioning. Per il costruttore abbiamo una copertura di entrambe le metriche al 100% (*Immagine 58*). Per la funzionalità di pin (*Immagine 59*) è emersa la mancanza di una partizione specifica: il caso in cui una chiave risulti già pinned ma associata a un valore null. È stato quindi introdotto un test aggiuntivo per verificare il comportamento del metodo in presenza di questa "entry ancorata vuota". Analogamente al metodo pin, in put (*Immagine 56*) è emersa la mancanza di una partizione in cui la entry era all'interno della PinnedMap ma con valore nullo. La suite è stata estesa simulando l'operazione di put su una chiave preesistente in questo specifico stato interno. Per la funzionalità di remove (*Immagine 57*) permane una copertura parziale dei rami. Tale scelta è stata deliberata: forzare la copertura del ramo logico affetto dall'errore implementativo discusso (la mancata revoca del pin) avrebbe significato validare un comportamento scorretto in violazione delle specifiche. I nuovi test sono descritti nella *Tabella 37*. A conclusione di questa fase di affinamento, le metriche globali della classe sono salite al 65% di Instruction Coverage e al 60% di Branch Coverage (*Immagine 60*). È importante sottolineare che, isolando l'analisi alle sole funzionalità oggetto di studio ed escludendo le porzioni di codice affette dai bug noti, la suite sviluppata raggiunge ora una copertura del 100% in entrambe le metriche.

3.2.4 Seconda iterazione con Pit Test

L'impiego della Mutation Analysis ha rappresentato la fase finale e più rigorosa della validazione, permettendo di misurare l'effettiva capacità di fault detection della suite di test. Inizialmente, la combinazione dei test manuali e di quelli derivanti dall'analisi della copertura ha prodotto un Mutation Score del 50%, con 66 mutanti uccisi su 133 generati (*Immagine 61*). Sebbene il costruttore abbia dimostrato una robustezza impeccabile raggiungendo il 100% di mutanti eliminati (*Immagine 62*), le altre funzionalità hanno evidenziato la necessità di un affinamento metodologico. Per quanto riguarda il metodo pin (*Immagine 63*), l'analisi ha permesso di isolare due mutanti equivalenti (falsi positivi generati dallo strumento sui valori di ritorno booleani) e di far emergere un limite strutturale della suite: la natura single-threaded dei test. La sopravvivenza dei mutanti legati alle istruzioni di writeUnlock ha evidenziato che i test attuali non erano in grado di sollecitare i meccanismi di sincronizzazione. Tale evidenza ha reso necessaria l'estensione della suite con test di concorrenza specifici, capaci di simulare accessi simultanei da thread distinti per validare l'integrità dei lock. La funzionalità put (*Immagine 64*) ha richiesto il cambio di paradigma passando da un approccio Black Box a uno White Box. I mutanti sopravvissuti, legati alle callback interne come entryAdded ed entryRemoved, non sono stati inizialmente eliminati poiché i loro effetti non erano direttamente osservabili tramite l'API pubblica. Per superare questa barriera, è stato introdotto l'uso di uno Spy sul SUT, permettendo di intercettare le invocazioni dei metodi interni e discriminare correttamente i percorsi di esecuzione. Parallelamente, si è proceduto al raffinamento di un test esistente, testPut_UpdatePinnedGhostEntry_incrementSize, dove è stata inserita un'asserzione esplicita sulla size per garantire l'uccisione del mutante precedentemente ignorato. Infine, anche per il metodo remove (*Immagine 65*) sono state riscontrate lacune analoghe relative alla gestione dei lock e alle chiamate di callback. L'implementazione di test supplementari focalizzati sul multi-threading e sull'intercettazione delle logiche interne consultabili nella *Tabella 38* ha permesso di elevare il risultato finale di PITest è salito al 61% di mutanti uccisi (81 su 133) (*Immagine 66*).

3.2.5 Generazione automatica dei Test

LLM: Zero-shot

Assumendo che il vincolo operativo principale nell'utilizzo degli LLM sia rappresentato dal consumo di token (limite della finestra di contesto), nel caso specifico tale problematica risulta mitigata dalle dimensioni ridotte della classe in esame (688 righe di codice). Tale caratteristica ha permesso di adottare un approccio Zero-Shot Prompting, fiduciosi che il codice venga analizzato interamente dal modello. La sperimentazione è stata condotta utilizzando il modello Gemini 3 Pro. Al fine di valutare la consistenza dell'output e mitigare la variabilità stocastica intrinseca agli LLM, la richiesta è stata iterata per dieci volte. Per garantire l'indipendenza statistica di ogni iterazione, ogni test è stato eseguito in una sessione di inferenza isolata, azzerando così la cronologia del contesto ed evitando che le generazioni precedenti influenzassero i risultati successivi. Il prompt utilizzato per l'esperimento è consultabile al seguente riferimento *Prompt LLM 3*. L'esame comparato delle iterazioni ha evidenziato una notevole uniformità quantitativa nel codice prodotto. Il numero di test case generati è risultato costante in tutte le sessioni, suggerendo che la lunghezza dell'output sia stata saturata dal limite tecnico dei token di risposta del modello. Tra le varianti ottenute, la terza iterazione è stata selezionata per l'analisi approfondita, in quanto ha presentato lo scenario di copertura più rilevante. L'analisi ha rivelato un errore sistematico presente in 7 versioni su 10, specificamente nel metodo generato testConstructors. Il modello ha manifestato un'incomprensione semantica della firma del costruttore, associando erroneamente il parametro size alla capacità della SoftMap. Di conseguenza, le asserzioni generate fallivano sistematicamente: il test prevedeva che la softReferenceSize assumesse il valore passato (es. 50), mentre l'implementazione reale della classe la inizializza di default a -1 (illimitata), destinando quel parametro esclusivamente al dimensionamento iniziale della HardMap. Questa versione si distingue per aver identificato autonomamente due criticità funzionali. La prima riguarda la gestione della rimozione in presenza di pinning (verificata tramite il test testPinningWithRemove): l'esecuzione ha evidenziato la stessa discrepanza identificata nei test manuali. La seconda criticità, individuata mediante il test testSoftReferencePromotion, concerne il difetto nell'inizializzazione della classe con LRU attivo e size pari a 0, passando per il costruttore a due parametri che, come valore di size, dimezza il valore di max. I risultati ottenuti sono statement coverage 92% e branch coverage 85% (*Immagine 67*) e mutation score 64% (*Immagine 68*).

Randoop

L'impiego di Randoop ha prodotto 21 test, tutti con esito positivo. Le metriche hanno registrato una statement coverage del 65% e una branch coverage del 44% (*Immagine 69*), con un mutation score del 34% (*Immagine 70*).

Evosuite

Tramite EvoSuite sono stati generati 72 test, anch'essi con esito positivo. Il tool ha ottenuto risultati superiori, raggiungendo il 99% di statement coverage e il 97% di branch coverage (*Immagine 71*), con un mutation score del 59% (*Immagine 72*).

3.2.6 Conclusioni

L'integrazione delle suite di test prodotte tramite le diverse metodologie ha permesso di validare la classe attraverso 167 test case. Considerando i 4 fallimenti rilevati, relativi alla rimozione del pinning e ai metodi containsKey e get con size nulla, si ottiene una reliability del 97,6%, calcolata adottando un profilo operativo con probabilità di utilizzo uniforme. Le metriche finali confermano l'efficacia del processo, con una statement coverage del 99%, una branch coverage del 97% (*Immagine 73*) e un mutation score dell'83% (*Immagine 74*).

3.3 ProxyManagerImpl

La classe ProxyManagerImpl è il componente responsabile della gestione del ciclo di vita dei tipi di dati mutabili (Collections, Maps, Dates, Calendars) all'interno del framework. Essa agisce come un servizio ausiliario di Factory per lo StateManager (il gestore dello stato dell'entità), fornendo due servizi essenziali:

- **Strumentazione:** Converte oggetti Java standard in istanze Proxy capaci di intercettare le modifiche e notificarle allo StateManager.
- **Destrumentazione:** Converte istanze Proxy in copie "pulite", prive di logica infrastrutturale, utilizzate per operazioni di detach, rollback e caching.

Sempre all'interno della documentazione¹⁴ troviamo alcune impostazioni specifiche della nostra classe:

- **TrackChanges:** Questo flag controlla l'intelligenza dei proxy generati. Determina se il proxy deve limitarsi a segnalare *che* è cambiato (dirty flag), o se deve tracciare *cosa* è cambiato. Quando il flag è impostato a true, il proxy generato deve implementare l'interfaccia org.apache.openjpa.util.ChangeTracker, mantenendo internamente dei set di dati che mantengono lo storico dei cambiamenti.
- **AssertAllowedType:** Questo flag controlla la sicurezza dei tipi a runtime all'interno dei proxy. Determina se i proxy devono fidarsi ciecamente del codice chiamante o se devono validare che gli oggetti inseriti corrispondano ai metadati. Quando il flag è impostato a true, il proxy generato inietterà controlli aggiuntivi nei metodi di scrittura (come add, put, set). Prima di modificare l'istanza, verificherà che il tipo dell'oggetto corrisponda a quello atteso. In caso contrario, lancerà immediatamente un'eccezione.
- **DelayCollectionLoading:** Questo flag controlla la strategia di inizializzazione dei dati all'interno dei proxy Collection. Se è true il caricamento dei dati deve avvenire solo ad una operazione che ne richiede il caricamento. In questo caso il Proxy deve implementare l'interfaccia org.apache.openjpa.util.DelayedProxy. Il caricamento ritardato permette di eseguire semplici operazioni di aggiunta o rimozione non indicizzate su una collezione di tipo "lazy" senza doverla caricare interamente. La collezione viene caricata solo quando necessario, ad esempio durante l'iterazione, operazioni indicizzate, oppure richiamando i metodi isEmpty o size.
- **Unproxyable:** Questa configurazione controlla le eccezioni alla regola di strumentazione. Determina una lista specifica di classi che il ProxyManager deve rifiutarsi categoricamente di trasformare in Proxy. Quando una classe è presente nella lista, il metodo factory non genererà alcun proxy, ma restituirà null. Quando invece la classe non è presente (default), il manager avvolgerà l'oggetto in una classe generata che implementa l'interfaccia Proxy.

Per il testing di queste funzionalità ci focalizzeremo sui metodi principali che implementano la logica della factory: newCustomProxy e copyCustom. Focalizzarsi su questi due metodi è una scelta strategica perché rappresentano i "punti di transito" cruciali dove avviene la trasformazione del dato: uno attiva l'intelligenza del proxy per tracciare i cambiamenti, mentre l'altro ne garantisce una copia pulita e sicura per le operazioni esterne. Concentrarsi su di essi permette di validare in un colpo solo tutte le logiche di configurazione e sicurezza del framework, testando l'intero ciclo di vita del componente nel modo più efficace e meno frammentato possibile.

3.3.1 Test Manuali

FUNZIONALITÀ DI STRUMENTAZIONE

L'obiettivo di questa funzionalità è quello di decidere quale tipo di proxy creare partendo da un oggetto esistente e assicurarsi che i dati di quest'ultimo vengano trasferiti all'interno del nuovo proxy. Questa funzionalità è implementata dal metodo:

```
public Proxy newCustomProxy(Object orig, boolean autoOff)
```

Descrizione dei parametri

- **Object orig:** Rappresenta l'istanza sorgente (collezione, mappa, data o bean) da cui il manager deve derivare il proxy, preservandone lo stato e i metadati.
- **boolean autoOff:** Nella documentazione non è specificato l'uso e il significato di questo parametro.
- **boolean TrackChanges:** Flag globale che determina se i proxy generati debbano integrare un meccanismo di ChangeTracker per registrare analiticamente ogni modifica apportata ai dati.
- **boolean AssertAllowedType:** Impone ai proxy l'esecuzione di controlli di validazione a runtime per garantire che gli elementi aggiunti siano conformi ai tipi definiti nei metadati.
- **boolean DelayCollectionLoading:** Abilita il caricamento differito per le collezioni, posticipando l'invocazione del recupero dati fino al momento necessario.

¹⁴ <https://openjpa.apache.org/builds/4.1.1/apidocs/org/apache/openjpa/util/ProxyManagerImpl.html>

- String Unproxyable: Definisce una lista di nomi di classi (separati da punto e virgola) per le quali il manager ha il divieto esplicito di generare o restituire istanze proxy.

Category Partition

Sulla base dell'analisi funzionale della classe precedentemente illustrata e dell'individuazione dei parametri critici per la metodologia Category Partition, si definiscono nella *Tabella 39* le partizioni individuate. Per ciascuna categoria, le partizioni sono corredate da una specifica motivazione tecnica, volta a giustificare la scelta.

Boundary Analysis

A seguito della scomposizione dei parametri in categorie, si è proceduto all'individuazione dei Boundary Value applicando i principi della Boundary Value Analysis. Tali valori, dettagliati nella *Tabella 40*, sono stati selezionati per sollecitare i limiti delle partizioni identificate; dove ritenuto opportuno, la scelta è supportata da una specifica motivazione tecnica.

Test

Adottando un approccio monodimensionale sono stati derivati i casi di test riportati nella *Tabella 41*. Per ciascun caso di test, la *Tabella 42* specifica i relativi risultati attesi, definendo il comportamento nominale del sistema a fronte degli input selezionati.

FUNZIONALITÀ DI DESTUMENTAZIONE

Il metodo copyCustom è il punto di ingresso per la de-strumentazione o la duplicazione sicura dei dati. Il suo scopo principale è produrre un'istanza indipendente dell'oggetto fornito, che mantenga lo stesso stato informativo ma sia scollegata dalle logiche di intercettazione. Se orig è una istanza Proxy il metodo esegue una "pulizia" del dato estraendo lo stato interno e creando un nuovo oggetto del tipo standard. Se l'oggetto è già standard effettua una semplice copia o restituisce null nel caso il manager non sia in grado di effettuare la copia:

```
Public Object copyCustom(Object orig)
```

Descrizione dei parametri

- Object orig: rappresenta l'istanza originale da de-strumentare o clonare.

In questo scenario, gli attributi di configurazione del Manager non influenzano l'esito dell'operazione di de-strumentazione o clonazione. Di conseguenza, tali parametri sono stati esclusi dal processo di Category Partition, focalizzando l'analisi esclusivamente sulle varianti applicabili all'oggetto orig.

Category Partition

La *Tabella 43* riporta la scomposizione del parametro orig in categorie distinte. Per ciascuna di esse viene fornita una giustificazione metodologica, esplicitando il motivo per cui la specifica variante è ritenuta significativa ai fini del testing.

Boundary Analysis

A seguito della definizione delle partizioni, la *Tabella 44* riporta i Boundary Value individuati per ciascuna categoria. Dove ritenuto necessario, i valori di frontiera sono corredate da una nota descrittiva che ne giustifica la selezione ai fini della verifica dei limiti del sistema.

Test

I casi di test derivati per questa funzionalità mediante la metodologia Category Partition sono riportati nella *Tabella 45*, corredate dal relativo risultato atteso per ciascuno scenario individuato.

3.3.2 Risultati dei test manuali

La fase di progettazione ed esecuzione dei test ha permesso di portare alla luce diverse criticità, distinguendo tra comportamenti attesi non documentati, limitazioni tecniche del linguaggio e veri e propri errori implementativi. Per quanto riguarda le operazioni di destrumentazione, l'analisi del Test 3 ha rivelato un vincolo architetturale significativo nell'implementazione di copyCustom. Non è sufficiente che la classe target implementi il pattern del Copy Constructor; essa deve necessariamente configurarsi come un Bean, ovvero esporre dei metodi setter. Analizzando il metodo proxySetters, è emerso che ProxyManagerImpl cerca attivamente questi metodi e, in loro assenza, rifiuta di generare il proxy interno necessario per orchestrare la copia, facendo fallire l'operazione anche su oggetti tecnicamente validi. Pur trattandosi di un comportamento coerente con la logica del framework (che necessita di proxy per operare), la mancanza di documentazione in merito ha portato a classificare l'evento come un errore nel test, risolto successivamente aggiungendo un metodo di setting alla classe CustomCopyConstructor. Più grave è quanto emerso nel Test 2: il manager fallisce la copia se non trova un costruttore nullo o un copy constructor, ignorando completamente l'eventuale implementazione dell'interfaccia standard Cloneable. Questa è una chiara lacuna implementativa: è inaccettabile che un metodo di copia non preveda un fallback sulla strategia di clonazione nativa di Java. Tuttavia, poiché la logica della classe è intrinsecamente legata al paradigma del Proxy, il fallimento riscontrato nel test è da attribuire ad una criticità nella progettazione del test stesso, derivante dalla carenza di documentazione tecnica. Passando alla strumentazione, i Test 20 e 21 relativi alla funzionalità assertAllowedTypeOn su ArrayList e HashMap hanno evidenziato i limiti imposti dalla Type Erasure di Java. Utilizzando il metodo generico newCustomProxy, il framework non può dedurre i tipi specifici degli elementi (o di chiave e valore), passandoli internamente come null e disabilitando di fatto i controlli di tipo. Non si tratta di un bug del software, ma di un vincolo tecnologico che obbliga l'utilizzo dei metodi specifici newMapProxy e newCollectionProxy per garantire la sicurezza dei tipi. Una distinzione che dovrebbe essere chiaramente documentata. Analogamente, il fallimento del Test 23 sulla strumentazione ritardata (delayCollectionLoading) ha evidenziato che l'attuale implementazione del manager esclude le ArrayDeque dai tipi supportati per i proxy delayed, configurando una mancata implementazione funzionale non documentata. Infine, sono stati isolati tre difetti implementativi concreti. Il Test 17 ha scovato una fragilità nella classe di utilità StringUtil, che va in crash con un errore di indice se la configurazione dei filtri contiene una stringa di soli separatori (es. ";;;"). Ancora più critico è l'errore logico emerso nei Test 14 e 15: i filtri per le classi unproxyable vengono applicati correttamente agli oggetti custom, ma vengono totalmente ignorati per le Collezioni, le Mappe, le Date e i Calendar. Questo accade perché il controllo di sicurezza è posizionato troppo tardi

nel flusso di esecuzione, permettendo al sistema di creare proxy anche per classi che l'utente aveva esplicitamente escluso.

3.3.3 Prima iterazione Coverage

Dopo l'esecuzione dei test manuali progettati, i risultati ottenuti tramite JaCoCo evidenziano uno statement coverage del 36% e un branch coverage del 38% (*Immagine 75* e *Immagine 76*). A seguito di un'approfondita analisi del report è emerso che diversi rami logici complessi non venivano esercitati dai test funzionali. Per garantire una maggiore copertura, la suite di test manuali è stata estesa (*Tabella 46* e *Tabella 47*). La prima area di intervento riguarda i metodi di clonazione (copyMap, copyDate, ecc.) (*Immagine 79*). Questi erano stati testati solo indirettamente tramite l'utilizzo di copyCustom; tuttavia, facendo parte dell'interfaccia pubblica, è necessario validarne la robustezza a fronte di input non validi (come null) o input di tipo Proxy. Per quanto riguarda la funzionalità di "destrumentazione" (copyCustom *Immagine 78*), la generazione dei test iniziale si è rivelata troppo restrittiva, limitandosi all'utilizzo di un singolo tipo di oggetto non-Proxy. Sono stati quindi inseriti test specifici per Map, Date e Calendar standard (non proxy), verificando per ciascuno che venga restituita correttamente un'istanza copiata dell'oggetto di input. Un altro ramo non completamente esplorato riguarda la creazione dei DelayedProxy. L'analisi dell'implementazione del metodo loadDelayedProxy (*Immagine 80*) mostra la presenza di diverse tipologie di oggetti che supportano la creazione ritardata. Per ciascuno di questi (HashSet, LinkedList, Vector, LinkedHashSet, PriorityQueue) è stato introdotto un test specifico al fine di verificare che, a fronte di una richiesta con configurazione *delayed*, il proxy restituito implementi correttamente l'interfaccia DelayedProxy.class. Infine, l'analisi della copertura ha evidenziato che la logica di generazione dinamica delle classi (tramite ASM) all'interno di ProxyManagerImpl era mascherata dal sistema di caching di OpenJPA. Utilizzando nei test solo classi standard (java.util.HashMap, java.util.Date), il manager riutilizzava proxy già generati, lasciando inesorabilmente il codice di generazione del bytecode. Per esercitare effettivamente queste linee di codice critiche, sono stati introdotti test che invocano newCustomProxy passando istanze di classi custom (non standard), definite specificamente per il test (es. FreshHashMapForTest). Questo approccio aggira la cache, costringendo il ProxyManager a eseguire l'intera pipeline di creazione del bytecode.

3.3.4 Seconda iterazione con PitTest

Una volta implementati i test derivanti dall'analisi della coverage, è stata valutata l'adeguatezza dell'intera suite per confermare la sufficienza del lavoro svolto. L'analisi effettuata tramite il tool PIT sui test manuali ha prodotto un risultato di mutation coverage del 64% (*Immagine 81*). Dall'esame del report generato dal tool, emerge che le principali funzioni presentano una copertura completa dei mutanti (es *Immagine 82*). Tuttavia, sono stati individuati alcuni mutanti sopravvissuti ritenuti critici, per i quali è stato necessario intervenire. Nello specifico, nella funzione copyCollection (*Immagine 84*) è sopravvissuto il mutante alla riga 214, relativo al ritorno di null in caso di input nullo. Poiché questo comportamento è parte integrante del contratto dell'interfaccia, è stato aggiunto un test case esplicito per verificare la corretta gestione dell'input null. Analizzando la funzionalità newCustomProxy (*Immagine 83*), è emersa una lacuna nella verifica dei dati: il test esistente sul proxy del GregorianCalendar controllava solo la corretta copia della TimeZone. Tuttavia, è fondamentale verificare anche la persistenza dei millisecondi, un altro dato complesso dell'oggetto. Di conseguenza, il Test 10 (strumentazione) è stato esteso per includere un'asserzione specifica sulla corretta copia del valore temporale. Infine, un fattore non precedentemente considerato riguarda la gestione della lista Unproxyable (*Immagine 85*). È stato quindi aggiunto un test funzionale che prevede l'inserimento effettivo di una classe nella lista delle esclusioni, verificando successivamente che per tale classe non venga generato alcun proxy.

3.3.5 Generazione automatica dei Test

Fallimento della Generazione Automatica dei Test con EvoSuite

Il primo tentativo di generazione diretta sulla classe target è fallito a causa della natura dinamica del componente: ProxyManagerImpl genera bytecode a runtime (classi proxy) che il motore di strumentazione di EvoSuite non è riuscito a gestire, sollevando eccezioni di tipo ClassNotFoundException. Per mitigare il problema, è stata sviluppata una classe Driver (ProxyManagerDriver) progettata per esporre solo i metodi deterministici (logica di copia e configurazione) e isolare le funzionalità di generazione dei proxy. Nonostante il refactoring e l'applicazione di filtri di esclusione espliciti per ignorare le classi generate dinamicamente, il processo è fallito nuovamente con errori critici di comunicazione RMI (java.rmi.UnmarshalException). L'analisi dello stack trace ha confermato che il ClassLoader di EvoSuite tenta invariabilmente di strumentare le classi effimere create da OpenJPA, causandone il crash. Di conseguenza, l'approccio basato su EvoSuite è stato considerato inapplicabile.

Randoop: Test Randomici

A differenza di EvoSuite, che richiede un'invasiva strumentazione del bytecode, l'approccio feedback-directed random testing di Randoop ha permesso di evitare le eccezioni di tipo ClassNotFoundException derivanti dalla generazione dinamica dei proxy. L'analisi strutturale evidenzia una statement coverage del 35% e una branch coverage del 28% (*Immagine 86* e *Immagine 87*). Tali metriche denotano i limiti del tool nell'esplorare la complessità logica della classe ProxyManagerImpl. Sebbene la copertura sia ottimale per i metodi di configurazione e stato, si registra un netto calo in corrispondenza della business logic principale. La bassa percentuale di rami coperti suggerisce che i metodi siano stati invocati prevalentemente con input banali o sequenze lineari ('happy path'), fallendo nell'attivare le diramazioni condizionali più profonde. Come atteso, i metodi deputati alla manipolazione del bytecode sono rimasti esclusi dall'esecuzione. Questa bassa copertura si riflette naturalmente nei risultati di Pitest: una mutation coverage del 21% (*Immagine 88*) dimostra che l'uso isolato di Randoop non è sufficiente a garantire una adeguatezza della suite di test e garantire una validazione robusta della classe.

LLM: Few-shot Strategy

Per l'attività di generazione automatica tramite LLM (Gemini Pro) dei test relativi alla classe `ProxyManagerImpl`, è stata adottata la strategia di Few-shot Prompting. Questa scelta è stata dettata non solo dalla necessità di superare i limiti riscontrati con gli approcci precedenti, ma anche dalla volontà di sperimentare e validare i differenti approcci metodologici presentati a lezione. L'obiettivo era fornire al Large Language Model delle informazioni mirate sulla classe per istruirlo correttamente sul contesto di riferimento specifico di Apache OpenJPA. A tal fine, sono stati passati in input sia una classe di test manuali sia la classe utility utilizzata in tali test.

Per consentire un confronto diretto, è stato richiesto all'LLM un numero di test equivalente a quelli manuali (circa 70). Sfruttando la capacità degli LLM di interagire tramite linguaggio naturale, come già effettuato con l'approccio Guided Tree-of-Thoughts per la classe `BufferedChannel` in Bookkeeper, nel prompt non è stato richiesto immediatamente il codice Java, ma è stato previsto un passaggio preliminare di strutturazione arbitraria. In questa fase, è stato chiesto all'LLM di descrivere a parole ognuno dei 70 test, procedendo successivamente con un'implementazione in batch per garantire che il limite di token non compromettesse la produzione dei raw generated tests (*Prompt LLM 4*). A differenza dell'approccio ToT, in tutte le 10 differenti iterazioni i test prodotti dall'LLM hanno presentato numerosi errori, principalmente legati al casting degli Object prodotti dal Manager. Questo è indice di una difficoltà maggiore per LLM di costruire test per classi più complesse come `ProxyManagerImpl`. Ciò ha richiesto diverse operazioni manuali di modifica prima della compilazione. Dopo la fase di testing, è emerso che l'LLM ha commesso un errore logico sistematico, aspettandosi che i metodi `copyCollection`, `copyMap`, `copyDate`, `copyCalendar` e `copyCustom` restituissero dei proxy. Gli errori derivanti dai risultati del testing sono stati quindi sottoposti all'LLM per una fase di Validator and Repair, fino a ottenere i selected and repaired tests finali. La suite completa ha ottenuto una statement coverage del 50%, una branch coverage del 48% (*Immagine 89 e Immagine 90*) e una mutation coverage del 32% (*Immagine 91*). Questi risultati confermano quanto emerso dalle risposte del modello durante la pianificazione: l'LLM si è affidato eccessivamente agli esempi forniti (*bias* del context), producendo una suite molto simile a quella manuale data in input, ma con input semplificati non derivanti da un approccio di category partition. Il risultato è stato una scarsa efficacia sia nella branch coverage e, soprattutto, nella mutation coverage. In conclusione, questa esperienza conferma che la generazione automatica di test tramite LLM non può prescindere da una supervisione umana critica. Metodologicamente, il Few-shot prompting è risultato il meno efficace: l'elevata complessità nel gestire strutture di prompt articolate e contesti voluminosi non ha prodotto un valore aggiunto proporzionale allo sforzo. Al contrario, ha generato un bias di ridondanza, dove il modello si è limitato a replicare gli esempi forniti senza esplorare nuovi scenari. Paradossalmente, la semplicità dello Zero-shot prompting offre una maggiore "creatività" potenziale, permettendo all'LLM di suggerire rami logici e opzioni inedite che l'approccio basato su esempi finisce inevitabilmente per vincolare.

3.3.6 Conclusione

L'integrazione dei differenti approcci di testing precedentemente analizzati ha permesso di consolidare una suite di test completa, i cui risultati riflettono un discreto grado di accuratezza nell'analisi del codice. In termini di copertura strutturale, si è raggiunta una Statement Coverage del 91% e una Branch Coverage del 75% (*Immagine 92 e Immagine 93*). La Mutation Coverage si attesta al 69% (*Immagine 94*) con la corretta identificazione e uccisione di 474 mutanti su un totale di 684. Per quanto concerne la stima della Reliability, la valutazione è stata condotta sull'intera suite di test prodotta, adottando un profilo operativo con probabilità di utilizzo uniforme. Su un totale di 163 casi di test, 160 sono stati eseguiti con successo, portando a una Reliability stimata di 0.9816 (98,16%). Le tre rilevazioni negative (pari all'1,84% del totale) sono riconducibili a fallimenti sistematici nei test relativi al meccanismo di vincolo di `UnProxyable` (nello specifico, i test 14, 15 e 17 riportati in *Tabella 41*). Tali esiti sono stati classificati come errori di implementazione nel codice sorgente, evidenziando una criticità specifica nella gestione dei vincoli di proxying che la suite di test è stata in grado di isolare con precisione.

4 Appendice

4.1 Tabelle

4.1.1 BufferedChannel

Tabella 1 : Category Partition BufferedChannel Inizializzazione del layer in Memoria

parametro	partizione	descrizione
writecapacity	< 0	L'allocazione di una capacità minore di zero è sempre invalida.
	> 0	Valido nel caso in cui il fileChannel è aperto in scrittura.
	= readCapacity = 0	Si considera l'istanziamento di un BufferedChannel, con readCapacity e writeCapacity nulle, un'operazione non corretta poiché priva di significato.
	= 0 && ≠ readCapacity	Il valore 0 associato a writeCapacity assume un senso nel caso in cui il corrispettivo readCapacity è maggiore di 0: si vuole usare il canale in sola lettura.
readcapacity	< 0	L'allocazione di una capacità minore di zero è sempre invalida.
	> 0	Valido nel caso in cui il fileChannel è aperto in lettura.
	= writeCapacity = 0	Si considera l'istanziamento di un BufferedChannel, con readCapacity e writeCapacity nulle, un'operazione non corretta poiché priva di significato.
	= 0 && ≠ writeCapacity	Il valore 0 associato a readCapacity assume un senso nel caso in cui il corrispettivo writeCapacity è maggiore di 0: si vuole usare il canale in sola scrittura.
bytebufferallocator	Istanza valida	Tutte le implementazioni dell'interfaccia di ByteBuffer Allocator che restituiscono correttamente un buffer della dimensione richiesta.
	Istanza invalida	Essendo un'interfaccia vi è la possibilità che venga utilizzato un allocatore invalido, cioè (essendo una factory) che non restituisca un buffer della dimensione richiesta.
filechannel	Null	Parametro nullo.
	Istanza valida	Il canale è correttamente inizializzato, aperto e associato a un file esistente con permessi coerenti alle capacità di lettura/scrittura richieste.
	Istanza invalida	Il canale è chiuso (isOpen() == false) oppure non possiede i privilegi necessari per completare l'operazione (es. tentativo di scrittura su un canale aperto in sola lettura). Oppure un fileChannel che è
	Null	Il riferimento al canale è nullo, rendendo impossibile qualsiasi operazione di delega verso il File System sottostante.

Tabella 2 : Boundary Analysis BufferedChannel Inizializzazione del Layer in Memoria

parametro	categoria	boundary value	descrizione
writecapacity	< 0	-1	Ci aspettiamo che ogni richiesta di allocazione con writeCapacity -1 lanci una eccezione
	> 0	1	Valore minimo di capacità positiva. Testa l'allocazione del buffer di scrittura più piccolo possibile.
	= readCapacity = 0	0	Unico valore possibile per la categoria.
readcapacity	= 0 && != readCapacity	0	Unico valore possibile per la categoria.
	< 0	-1	Ci aspettiamo che ogni richiesta di allocazione con readCapacity -1 lanci una eccezione
	> 0	1	Valore minimo di capacità positiva. Testa l'allocazione del buffer di lettura più piccolo possibile.
bytebufferallocator	= writeCapacity = 0	0	Unico valore possibile per la categoria.
	= 0 && != writeCapacity	0	Unico valore possibile per la categoria.
	Istanza valida	Netty UnpooledByteBufAllocator	L'allocatore della libreria Netty che risponde correttamente alla richiesta, allocando un buffer della dimensione corretta.
	Istanza invalida	Ritorno null	L'istanza di un allocatore che non alloca un buffer e restituisce null.
		Capacità Insufficiente	L'allocatore ritorna un buffer la cui dimensione effettiva (buffer.capacity()) è minore della dimensione richiesta. Questo testa la capacità del BufferedChannel di validare la capacità fisica del buffer allocato.
		Reference Count Invalido	L'allocatore ritorna un buffer ma con il contatore di riferimenti (refCnt) pari a zero. Questo scenario simula un buffer già rilasciato/deallocato che non può essere utilizzato.
	Null	null	Ci aspettiamo che ogni richiesta di allocazione con byteBufferAllocator null lanci una eccezione perché non è possibile istanziare i buffer richiesti
filechannel	Istanza valida	La validità del FileChannel è condizionata dalla capacità richiesta in lettura e scrittura. La funzionalità di inizializzazione deve validare i permessi del canale contro le capacità dei buffer. - FileChannel chiuso - FileChannel aperto in scrittura - FileChannel aperto in lettura - FileChannel aperto in lettura e scrittura Le istanze, oltre al Filechannel chiuso, diventano invalide in base alla capacità dei buffer richiesti. Un Canale aperto solo in scrittura è considerato invalido se readCapacity > 0, un Canale aperto solo in lettura è considerato invalido se writeCapacity > 0.	
	Istanza invalida		
	Null	null	Ci aspettiamo che ogni richiesta di istanziazione di bufferChannel con fileChannel nullo lanci una eccezione

Tabella 3 : Test Manuali "Inizializzazione BufferedChannel"

#	write capacity	read capacity	bytebuffer allocator	file channel	risultato atteso
0	1	1	Null	Open ReadWrite	Eccezione: non è possibile allocare i ByteBuf
1	1	1	Less Return	Open ReadWrite	Eccezione: l'allocatore restituisce un ByteBuf valido ma non corretto
2	1	1	ByteBuf nullo	Open ReadWrite	Eccezione: l'allocatore non restituisce un ByteBuf valido
3	1	1	ByteBuf released	Open ReadWrite	Eccezione: l'allocatore restituisce un ByteBuf valido ma non corretto
4	1	1	Valido	Closed	Eccezione: FileChannel closed
5	1	1	Valido	Null	Eccezione: NullPointerException filechannel
6	1	1	Valido	Open ReadOnly	Eccezione: richiesto buffer di scrittura ma il fc è readOnly
7	1	0	Valido	Open ReadOnly	Eccezione: richiesto buffer di scrittura ma il fc è readOnly
8	0	1	Valido	Open ReadOnly	Valido: nessuna eccezione sollevata
9	0	0	Valido	Open ReadOnly	Eccezione: non ha senso l'istanziamento della classe
10	0	-1	Valido	Open ReadOnly	Eccezione: richiesto buffer di dimensione negativa
11	-1	0	Valido	Open ReadOnly	Eccezione: richiesto buffer di dimensione negativa
12	-1	-1	Valido	Open ReadOnly	Eccezione: richiesto buffer di dimensione negativa
13	1	1	Valido	Open WriteOnly	Eccezione: richiesto buffer di lettura ma il fc è writeOnly
14	1	0	Valido	Open WriteOnly	Valido: nessuna eccezione sollevata
15	0	1	Valido	Open WriteOnly	Eccezione: richiesto buffer di lettura ma il fc è writeOnly
16	0	0	Valido	Open WriteOnly	Eccezione: non ha senso l'istanziamento della classe
17	0	-1	Valido	Open WriteOnly	Eccezione: richiesto buffer di dimensione negativa
18	-1	0	Valido	Open WriteOnly	Eccezione: richiesto buffer di dimensione negativa
19	-1	-1	Valido	Open WriteOnly	Eccezione: richiesto buffer di dimensione negativa
20	1	1	Valido	Open ReadWrite	Valido: nessuna eccezione sollevata
21	1	0	Valido	Open ReadWrite	Valido: nessuna eccezione sollevata
22	0	1	Valido	Open ReadWrite	Valido: nessuna eccezione sollevata
23	0	-1	Valido	Open ReadWrite	Eccezione: richiesto buffer di dimensione negativa
24	-1	0	Valido	Open ReadWrite	Eccezione: richiesto buffer di dimensione negativa
25	0	0	Valido	Open ReadWrite	Eccezione: non ha senso l'istanziamento della classe
26	-1	-1	Valido	Open ReadWrite	Eccezione: richiesto buffer di dimensione negativa

Tabella 4 : Category Partition BufferedChannel Read

parametro	partizione	descrizione
filechannel	Chiuso	Anche se verificato nella funzionalità di inizializzazione, essendo passato come riferimento dall'esterno la classe che istanzia il bufferedChannel potrebbe in una fase successiva chiudere la risorsa.
	Aperto non in lettura	L'operazione di read su un file aperto non in lettura dovrebbe essere sempre invalida indipendentemente da dove risiedono i dati.
	Aperto in lettura	Il file può essere letto.
readcapacity	= 0	La classe buffered Channel potrebbe essere stata istanziata con readCapacity pari a 0, quindi bisogna testare la resistenza dell'operazione in questi casi.
	$< length$	Valore di readCapacity maggiore di 0 ma minore della dimensione dei dati da leggere. Testiamo la capacità della funzionalità nel caso in cui i dati da leggere non entrano tutti insieme nel buffer.
	$\geq length$	Valore di readCapacity maggiore di 0 e maggiore anche della lunghezza dei dati che vogliamo leggere: tutti i dati entrano insieme all'interno del buffer.
length	< 0	La lunghezza dei dati da leggere non può essere negativa. Ci aspettiamo un'eccezione.
	≥ 0	Lunghezza dei dati da leggere valida. Si considera, come nell'implementazione standard in Java che la richiesta di lettura con length 0 sia lecita e semplicemente non restituisce alcun dato letto.
dest	null	Un buffer nullo è invalido.
	Rilasciato	Un buffer rilasciato (con Ref=0) è invalido nel caso in cui la richiesta length è maggiore di 0.
	$ByteScrivibili < length$	La dimensione disponibile per la scrittura dei dati è minore della lunghezza richiesta.
pos	$ByteScrivibili \geq length$	La dimensione disponibile per la scrittura dei dati è maggiore o uguale della lunghezza richiesta.
	< 0	Una posizione minore di 0 è sempre considerata invalida.
	$0 \leq pos \leq Datafile - length$	I dati da leggere sono tutti all'interno del file.
	$Datafile - length < pos < Datafile$	I dati da leggere sono parte nel file e parte nel buffer di scrittura.
	$Datafile \leq pos \leq Datafile + Databuffer - length$	I dati da leggere sono tutti nel buffer di scrittura.
	$Datafile + Databuffer - length < pos < DataFile + DataBuffer$	La posizione è valida ma i dati non sono sufficienti a soddisfare la richiesta di length.
	$pos \geq DataFile + DataBuffer$	La posizione è oltre l'ultima locazione scritta.

Tabella 5 : Boundary Analysis BufferedChannel Read

parametro	partizione	boundary value	descrizione
filechannel	Chiuso	Closed	Un FileChannel può essere chiuso dall'esterno dopo l'istanziamento
	Aperto non in lettura	Open writeOnly	Un file Channel aperto solo in scrittura. L'operazione di lettura non deve essere permessa
	Aperto in lettura	Open ReadWrite	Un file Channel aperto in lettura.
readcapacity	= 0	0	Una readCapacity pari a 0 indica che il bufferedChannel non è abilitato alla lettura, quindi deve lanciare una eccezione.
	$< length$	1	Rappresenta il valore minimo di capacità valida ed è minore del valore massimo della length scelto.
	$\geq length$	2	Rappresenta il valore minimo di capacità della categoria rispetto al valore massimo della length scelto.
length	< 0	-1	Valore invalido per la lunghezza di lettura.
	≥ 0	0	Caso limite: una richiesta di lettura di lunghezza zero non deve produrre effetti né sollevare eccezioni.
		2	Rappresenta un valore di lettura multi-byte che permette di verificare tutte le categorie del parametro pos (ovvero anche il caso in cui 1 byte si trova nel file e l'altro nel buffer).
dest	null	Null	Verifica della robustezza: il passaggio di un buffer di destinazione nullo deve produrre un errore in fase di scrittura.
	Rilasciato	Released(refCnt=0)	La funzionalità read deve essere resistente alla ricezione di un buffer già rilasciato e quindi possibilmente deallocato.
	$ByteScrivibili < length$	Cap = 1	In questo caso, con parametro length=2 testeremo se la classe gestisce bene il caso in cui i dati richiesti non entrano contemporaneamente nel buffer ma bisogna effettuare una lettura per batch.
pos	$ByteScrivibili \geq length$	Cap = 2	Caso nominale in cui la capacità del buffer di destinazione è esattamente uguale alla lunghezza richiesta.
	< 0	-1	Una posizione negativa è sempre invalida.
	$0 \leq pos \leq Datafile - length$	0	In questo caso sono stati semplicemente presi tutti i limiti delle categorie designate tramite l'utilizzo dell' <i>Immagine 1 : Category Partition</i>
	$Datafile - length < pos < Datafile$	DataFile - length	
		Datafile - length + 1	
	$Datafile \leq pos \leq Datafile + Databuffer - length$	DataFile	<i>pos (Read).</i>
		DataFile + DataBuffer - length	In questo modo testiamo in maniera approfondita che l'operazione sia astratta rispetto a dove sono posizionati fisicamente i dati.
	$Datafile + Databuffer - length < pos < DataFile + Databuffer$	DataFile + DataBuffer - length + 1	
		DataFile + DataBuffer - 1	
	$pos \geq DataFile + DataBuffer$	DataFile + DataBuffer	

Tabella 6 : Test Manuali BufferedChannel "Read"

#	file channel	read capacity	length	dest	pos	risultato atteso
0	Closed	1	2	2	0	Eccezione: i dati richiesti sono nel file ed è chiuso
1	Closed	1	2	2	fcDataS	Valido: leggiamo correttamente i primi due byte del buffer
2	Closed	1	0	2	0	Valido: il file è chiuso ma la richiesta di lettura è di 0 byte
3	Closed	1	0	2	fcDataS	Valido: il file è chiuso ma la richiesta di lettura è di 0 byte
4	Open WriteOnly	0	2	2	0	Eccezione: il file è aperto in sola scrittura
5	Open WriteOnly	0	2	2	fcDataS	Eccezione: il file è aperto in sola scrittura
6	Open WriteOnly	0	0	2	0	Valido: il file è aperto in sola scrittura ma la richiesta di lettura è di 0 byte
7	Open WriteOnly	0	0	2	fcDataS	Valido: il file è aperto in sola scrittura ma la richiesta di lettura è di 0 byte
8	Open ReadWrite	0	2	2	0	Eccezione: readBuffer ha capacità nulla
9	Open ReadWrite	0	0	2	0	Valido: non verrà letto nulla e il buffer di destinazione rimarrà vuoto
10	Open ReadWrite	1	2	1	0	Eccezione: buffer di destinazione sottodimensionato. Nel buffer di destinazione non ci sarà nessun dato
11	Open ReadWrite	1	2	null	0	Eccezione: buffer dest invalido
12	Open ReadWrite	1	2	released	0	Eccezione: buffer dest invalido
13	Open ReadWrite	1	0	null	0	Eccezione: buffer dest invalido
14	Open ReadWrite	1	0	released	0	Valido: avendo richiesto 0 byte di lettura l'operazione non deve restituire alcun dato
15	Open ReadWrite	1	2	null	fcDataS	Eccezione: buffer dest invalido
16	Open ReadWrite	1	2	released	fcDataS	Eccezione: buffer dest invalido
17	Open ReadWrite	1	0	null	fcDataS	Valido: la richiesta di lettura è nulla
18	Open ReadWrite	1	0	released	fcDataS	Valido: la richiesta di lettura è nulla
19	Open ReadWrite	1	-1	2	0	Eccezione: length invalida
20	Open ReadWrite	1	2	2	-1	Eccezione: posizione di lettura invalida
21	Open ReadWrite	1	2	2	0	Valido: il writeBuffer conterrà i primi due byte di fcData
22	Open ReadWrite	1	2	2	fcDataS - 2	Valido: il writeBuffer conterrà gli ultimi 2 bytes di fcData
23	Open ReadWrite	1	2	2	fcDataS - 1	Valido: il writeBuffer conterrà l'ultimo byte del file e il primo del writebuffer
24	Open ReadWrite	1	2	2	fcDataS	Valido: il writebuffer conterrà i primi 2 bytes di wbData
25	Open ReadWrite	1	2	2	fcDataS+wbDataS-2	Valido: il writeBuffer conterrà gli ultimi 2 bytes di wbData
26	Open ReadWrite	1	2	2	fcDataS+wbDataS-1	Eccezione: IOException e il writeBuffer conterrà l'ultimo byte di wbData
27	Open ReadWrite	1	2	2	fcDataS+wbDataS	Eccezione: posizione di lettura invalida
28	Open ReadWrite	2	2	2	0	Valido: il writeBuffer conterrà i primi due byte di fcData

Tabella 7 : Category Partition BufferedChannel Write

parametro	partizione	descrizione
writecapacity	= 0	La classe BufferedChannel potrebbe essere stata istanziata con writeCapacity pari a 0, quindi bisogna testare la resistenza dell'operazione in questi casi.
	$ByteScrivibili \leq DataSRC$	Questa partizione comprende i casi in cui i dati superano la capacità libera del buffer, rendendo necessaria una procedura iterativa di riempimento e svuotamento verso il canale sottostante.
	$ByteScrivibili > DataSRC$	In questa partizione rientrano i casi in cui la dimensione dei dati da scrivere è inferiore alla capacità residua del buffer, permettendo l'inserimento senza necessità di operazioni immediate di flushing.
filechannel	Chiuso	Anche se verificato nella funzionalità di inizializzazione, essendo passato come riferimento dall'esterno la classe che istanzia il bufferedChannel potrebbe in una fase successiva chiudere la risorsa.
	Aperto non in scrittura	Se il fileChannel di riferimento non è aperto in modalità write l'operazione omonima non deve essere permessa.
	Aperto in scrittura	Partizione in cui l'operazione di write è concessa.
unpersistedbytesbound	< 0	Una threshold negativa non è mai valida
	= 0	Impostando la soglia a 0 ci aspettiamo che il sistema non esegua mai la procedura automatica di force.
	> 0	Una soglia maggiore di 0 imposta il meccanismo di force automatico.
src	null	Un buffer nullo è da considerare invalido, la funzione deve essere resistente a questo tipo di input.
	empty	La partizione comprende il caso limite di un buffer vuoto, verificando che l'operazione di scrittura avvenga correttamente senza sollevare eccezioni o produrre stati inconsistenti.
	released	Questa partizione analizza il comportamento della classe nel caso in cui venga fornito un buffer con un reference count pari a zero. L'obiettivo è confermare che il wrapper intercetti lo stato di deallocazione della risorsa.
	$BytesLeggibili < UBBound - UB$	In questa partizione rientrano i casi in cui il buffer source ha dimensione inferiore allo spazio residuo disponibile al raggiungimento della soglia.
	$BytesLeggibili \geq UBBound - UB$	In questa partizione rientrano i casi in cui il buffer source ha dimensione superiore o uguale allo spazio residuo disponibile al raggiungimento della soglia.

Tabella 8 : Boundary Anlysis BufferedChannel Write

parametro	partizione	boundary value	descrizione
Write capacity	= 0	0	Serve a verificare come il sistema gestisce un buffer di capacità nulla.
	$ByteScrivibili \leq DataSRC$	$DataSRC - 1$	Testa il caso in cui il buffer sia appena più piccolo del dato da scrivere. Serve a forzare la logica di overflow e verificare che il ciclo di scrittura gestisca correttamente il residuo.
		$DataSRC$	Caso limite di perfetto allineamento. Verifica che il sistema non esegua inutilmente cicli extra ma esegua semplicemente il flush finale in modo da liberare il buffer per scritture successive.
	$ByteScrivibili > DataSRC$	$DataSRC + 1$	Verifica il comportamento quando il buffer ha spazio sufficiente per l'intero dato più un margine minimo, assicurando che non vengano innescate procedure di flush premature.
filechannel	Chiuso	Closed	Verifica la gestione delle eccezioni. È fondamentale per testare la resilienza del wrapper quando la risorsa sottostante viene chiusa esternamente.
	Aperto non in scrittura	Open readOnly	Testa la violazione dei permessi di accesso. Serve a confermare che il sistema segnali correttamente l'impossibilità di scrivere su canali aperti in sola lettura.
	Aperto in scrittura	Open ReadWrite	Rappresenta l'identità operativa standard. Serve come baseline per confermare il corretto funzionamento in condizioni nominali.
unpersisted bytes bound	≤ 0	-1	Valore di input non valido. Serve a testare la gestione degli errori di configurazione della soglia.
	= 0	0	Viene utilizzato per verificare il comportamento del sistema quando il limite di byte non persistiti è disattivato.
	> 0	10	Utilizzato per verificare che il meccanismo di accumulo funzioni correttamente fino al raggiungimento di una soglia prestabilita.
Unpersisted Bytes	= 0	0	Stato iniziale o post-force.
	> 0	5	Stato intermedio. Verifica che il calcolo della soglia di <i>force</i> sia cumulativo, includendo correttamente sia i dati dell'operazione corrente sia quelli già presenti nel buffer (o cache) e non ancora persistiti su disco. Il valore è rispetto a UBB uguale a 10.
src	null	Null	Test di robustezza contro i puntatori nulli.
	empty	$readIndex = writeIndex = 0$	Buffer con <code>readableBytes = 0</code> . Serve a verificare che un'operazione di scrittura "vuota" non alteri lo stato interno o la posizione del canale.
	released	$released (refCnt = 0)$	Buffer con reference count a zero (specifico per Netty). Verifica che il sistema sia resistente all'accesso a memoria deallocata.
	$BytesLeggibili < UBBound - UB$	$BytesLeggibili = UBBound - UB - 1$	Rappresenta il valore limite superiore della partizione in cui non avviene il flush. Verifica che, pur aggiungendo dati, se il totale cumulativo rimane esattamente un byte sotto la soglia critica, l'operazione di <code>force write</code> non venga innescata.
	$BytesLeggibili \geq UBBound - UB$	$BytesLeggibili = UBBound - UB$	Rappresenta il punto di saturazione. Testa che il sistema attivi correttamente la persistenza dei dati nell'istante esatto in cui la somma dei byte già presenti e di quelli nuovi eguaglia il limite impostato.
		$BytesLeggibili = UBBound - UB + 1$	Verifica la gestione dell'eccedenza oltre la soglia. Assicura che, quando la dimensione totale dei dati non ancora persistiti supera il limite, il sistema reagisca in modo deterministico forzando la scrittura su disco di tutti i dati.

Tabella 9 : Test Manuali BufferedChannel "write"

#	Write capacity	File channel	Ub bound	ub	src	risultato atteso
0	0	closed	0	0	null	Eccezione: Src è invalida non è possibile leggerla
1	0	closed	0	0	released	Eccezione: Src è invalida non è possibile leggerla
2	0	closed	0	0	0	Valido: non avviene alcuna scrittura, il wbbuffer rimane vuoto e la force non viene chiamata
3	0	closed	0	0	1	Eccezione: il buffer di scrittura ha capacità nulle
4	10	Closed	10	0	9	Eccezione: il fileChannel è chiuso
5	8	Closed	10	0	9	Eccezione: il fileChannel è chiuso, il writebuffer deve rimanere inalterato
6	0	OpenRW	10	0	null	Eccezione: Src è invalida non è possibile leggerla
7	0	OpenRW	10	0	released	Eccezione: Src è invalida non è possibile leggerla
8	0	OpenRW	10	0	0	Valido: non avviene alcuna scrittura e non viene chiamata la force. Sia il file che il wbbuffer rimangono vuoti
9	0	OpenRW	10	0	9	Eccezione: il buffer di scrittura ha capacità nulle
10	0	OpenRW	10	0	10	Eccezione: il buffer di scrittura ha capacità nulle
11	0	OpenRW	10	0	11	Eccezione: il buffer di scrittura ha capacità nulle
12	8	OpenRW	10	0	9	Valido: i primi 8 byte sono scritti nel file e l'ultimo è nel buffer. Non viene chiamata la force
13	9	OpenRW	10	0	10	Valido: i bytes sono nel file ed è stata chiamata la force. Il wb rimane vuoto
14	10	OpenRW	10	0	11	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wb rimane vuoto
15	0	OpenRW	10	0	0	Valido: non avviene alcuna scrittura. Sia il file che il wbbuffer rimangono vuoti e non viene chiamata la force
16	9	OpenRW	10	0	9	Valido: tutti i bytes sono scritti all'interno del file ma non è stata chiamata la force. Il wbbuffer è vuoto
17	10	OpenRW	10	0	10	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
18	11	OpenRW	10	0	11	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
19	1	OpenRW	10	0	0	Valido: non avviene alcuna scrittura. Sia il file che il wbbuffer rimangono vuoti e non viene chiamata la force
20	10	OpenRW	10	0	9	Valido: tutti i bytes sono scritti all'interno del buffer. Il file rimane vuoto e non viene chiamata la force
21	11	OpenRW	10	0	10	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
22	12	OpenRW	10	0	11	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
23	0	OpenRW	10	5	null	Eccezione: Src è invalida non è possibile leggerla
24	0	OpenRW	10	5	released	Eccezione: Src è invalida non è possibile leggerla
25	0	OpenRW	10	5	0	Valido: non avviene alcuna scrittura. Il file e il wbbuffer sono vuoti e non è stata chiamata la force
26	0	OpenRW	10	5	4	Eccezione: il buffer di scrittura ha capacità nulle
27	0	OpenRW	10	5	5	Eccezione: il buffer di scrittura ha capacità nulle
28	0	OpenRW	10	5	6	Eccezione: il buffer di scrittura ha capacità nulle
29	3	OpenRW	10	5	4	Valido: i primi 3 byte sono nel file, l'ultimo è nel buffer. Non è stata chiamata la force
30	4	OpenRW	10	5	5	Valido: tutti i bytes sono nel file ed è stata chiamata la force.
31	5	OpenRW	10	5	6	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
32	4	OpenRW	10	5	4	Valido: tutti i bytes sono nel file ma non è stata chiamata la force. Il wbbuffer è vuoto
33	5	OpenRW	10	5	5	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
34	6	OpenRW	10	5	6	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
35	1	OpenRW	10	5	0	Valido: non avviene alcuna scrittura.. Il file e il wbbuffer sono vuoti e non è stata chiamata la force
36	5	OpenRW	10	5	4	Valido: tutti i byte sono nel buffer. Non è stata chiamata la force e il file è vuoto
37	6	OpenRW	10	5	5	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
38	7	OpenRW	10	5	6	Valido: tutti i bytes sono nel file ed è stata chiamata la force. Il wbbuffer è vuoto
39	0	OpenRW	0	0	null	Eccezione: Src è invalida non è possibile leggerla
40	0	OpenRW	0	0	released	Eccezione: Src è invalida non è possibile leggerla
41	0	OpenRW	0	0	0	Valido: non avviene alcuna scrittura. La force non viene chiamata ed entrambi sono vuoti
42	0	OpenRW	0	0	1	Eccezione: il buffer di scrittura ha capacità nulle
43	1	OpenRW	0	0	1	Valido: il meccanismo di force è disabilitato, i dati sono sul file ma non è stata chiamata la force. Il wbbuffer rimane vuoto
44	1	OpenRW	0	0	0	Valido: non avviene alcuna scrittura. Il file e il wbbuffer sono vuoti e non viene chiamata la force
45	2	OpenRW	0	0	1	Valido: il byte è nel buffer. Il file è vuoto e non è stata chiamata la force
46	1	OpenRW	-1	0	1	Eccezione: impossibile avere UBB negativo
47	0	OpenRO	10	5	4	Eccezione: non è possibile scrivere in un file aperto in sola lettura
48	1	OpenRW	10	0	released	Eccezione: non può essere utilizzato un source buffer rilasciato.

Tabella 10 : Modifiche test manuali Inizializzazione BufferedChannel

#	write capacity	read capacity	bytebuffer allocator	file channel	risultato atteso
27 (ex1)	1	1	Less Return	Open ReadWrite	Valido: la dimensione dei buffer è conforme ai parametri
28 (ex6)	1	1	Valido	Open ReadOnly	Valido: il buffer di scrittura non viene allocato
29 (ex7)	1	0	Valido	Open ReadOnly	Valido: il buffer di scrittura non viene allocato
30 (ex13)	1	1	Valido	Open WriteOnly	Valido: il buffer di lettura non viene allocato
31 (ex15)	0	1	Valido	Open WriteOnly	Valido: il buffer di lettura non viene allocato

Tabella 11 : Modifiche Test manuali inizializzazione BufferedChannel Parte 2

#	write capacity	read capacity	bytebuffer allocator	file channel	risultato atteso
28 (ex6)	1	1	Valido	Open ReadOnly	Valido: vengono comunque allocati i buffer della dimensione richiesta
29 (ex7)	1	0	Valido	Open ReadOnly	Valido: vengono comunque allocati i buffer della dimensione richiesta
30 (ex13)	1	1	Valido	Open WriteOnly	Valido: vengono comunque allocati i buffer della dimensione richiesta
31 (ex15)	0	1	Valido	Open WriteOnly	Valido: vengono comunque allocati i buffer della dimensione richiesta
32 (ex9)	0	0	Valido	Open ReadOnly	Valido: l'istanziamento della classe avviene correttamente
33 (ex16)	0	0	Valido	Open WriteOnly	Valido: l'istanziamento della classe avviene correttamente
34 (ex25)	0	0	Valido	Open RW	Valido: l'istanziamento della classe avviene correttamente

Tabella 12 : Modifiche test Manuali BufferedChannel read

#	file channel	read capacity	length	dest	pos	risultato atteso
29 (ex5)	Open WriteOnly	0	2	Valid	fcDataS	Valido: l'operazione legge correttamente i primi due byte del wbuffer direttamente da quest'ultimo
30 (ex17)	Open ReadWrite	1	0	Null	fcDataS	Eccezione: l'operazione controlla immediatamente che il buffer di destinazione è nullo e lancia una nullException
31 (ex19)	Open ReadWrite	1	-1	2	0	Valido: viene applicata una sanitizzazione della length portandola a 0. Il buffer di destinazione rimane vuoto

Tabella 13 : Test Manuali aggiunti a read dopo fallimenti costruttore

#	file channel	read capacity	length	dest	pos	byte buffer allocator	risultato atteso
32	Open ReadWrite	1	2	2	0	Buffer released	Eccezione: il buffer di lettura non è utilizzabile perché rilasciato al sistema Netty
33	Open ReadWrite	1	2	2	0	Buffer null	Valido: il buffer di lettura non è intaccato dall'allocatore, che, come unico utilizzo, è quello di allocare un writeBuffer
34	Open ReadWrite	1	2	2	0	Buffer LessReturn	Valido: il buffer di lettura non è intaccato dall'allocatore, che, come unico utilizzo, è quello di allocare un writeBuffer
35	Open WriteOnly	1	2	2	0	Valid	Eccezione: anche se presente un Buffer per la lettura il file è aperto in modalità solo scrittura e i dati richiesti risiedono nel file

Tabella 14 : Modifiche Test Manuali write BufferedChannel

#	writecapacity	filechannel	ubbound	ub	src	risultato atteso
48 (4)	10	Closed	10	0	9	Valido: il contenuto viene scritto nel buffer, il file rimane vuoto e non viene chiamata la force
49 (46)	1	Open ReadWrite	-1	0	1	Valido: il contenuto viene scritto correttamente nel file senza chiamare la force. Il buffer alla fine dell'operazione è vuoto.

Tabella 15 : Aggiunta Test Manuali write BufferedChannel dopo fallimenti costruttore

#	writecapacity	filechannel	ubbound	ub	src	bytebuffer allocator	risultato atteso
50	9	Open ReadOnly	10	0	9	Valid	Eccezione: il File è aperto solo per la lettura
51	10	Open ReadOnly	10	0	9	Valid	Valido: il contenuto viene correttamente scritto all'interno del buffer
52	10	Open ReadWrite	10	0	10	Buffer released	Eccezione: il buffer di scrittura non è utilizzabile perché rilasciato
53	10	Open ReadWrite	10	0	10	Buffer null	Eccezione: il buffer di scrittura non è utilizzabile perché nullo
54	10	Open ReadWrite	10	0	9	Buffer LessReturn	Valido: l'operazione di scrittura deve considerare la vera dimensione del buffer (9) andando a scrivere tutto il contenuto nel buffer, poi spostarlo nel file ma senza chiamare la force

Tabella 16 : BufferedChannel Test Manuali Aggiunti dopo Iterazione con Jacoco

configurazione	azione	risultato atteso
Bufferedchannel con mock di allocatore che restituisce null e filechannel inizializzato con 4 byte.	read(destBuffer, 4, 1)	L'operazione deve restituire 0 (bytesRead == 0)
Readcapacity = 2. Esecuzione di read(dest, 3, 1) per popolare il buffer lontano dall'inizio.	read(dest, 0, 1)	L'operazione deve restituire 1 e il byte letto deve essere uguale a DISK_DATA[0]
Utilizzo di spy su filechannel e unpersistedbytesbound = 0.	Chiamata esplicita a forceWrite(false)	Verifica tramite lo spy che il metodo force(false) sia stato invocato 1 volta.
Utilizzo di mock su filechannel con doanswer per avanzare la position del buffer di soli 2 byte su 5 e ritornare 2.	write(5 byte) e successiva chiamata a flush().	Verifica tramite il mock che il metodo write() sia stato chiamato 2 volte.

Tabella 17 : BufferedChannel Test Aggiunti seconda iterazione

configurazione	azione	risultato atteso
Inizializzazione di un filechannel fisico con cursore esplicitamente pre-impostato alla posizione 100L.	Istanziamento di un nuovo oggetto BufferedChannel utilizzando il canale pre-configurato.	getFileChannelPosition() deve restituire 100L.
Bufferedchannel con il buffer di lettura già popolato (warm-up) a partire dall'offset 40.	Esecuzione di una richiesta di read() mirata all'offset assoluto 42.	Il byte letto deve corrispondere esattamente al valore 42. Questo garantisce la correttezza aritmetica del calcolo dell'indice relativo all'interno del buffer in memoria.
Bufferedchannel configurato con soglia unpersistedbytesbound = 0 (tracking disabilitato) e contenente dati scritti nel buffer.	Invocazione esplicita del metodo forceWrite(false).	Il contatore getUnpersistedBytes() deve rimanere invariato a 0, verificando che la logica di aggiornamento dei byte non persistiti venga correttamente saltata quando la funzionalità è disabilitata.

4.1.2 JournalChannel

Tabella 18 : Category Partition JournalChannel Apertura

parametro	partizione	motivazione
file journal directory	Corretto	Di questa categoria fanno parte tutti quei FilePath che corrispondono ad una directory esistente e scrivibile.
	Valido non corretto	Di questa categoria fanno parte tutti quei filePath che corrispondono a directory non esistenti o non scrivibili.
	Invalido	FilePath sintatticamente validi per il sistema operativo, ma semanticamente errato per l'applicazione (es. il percorso punta a un file esistente anziché a una directory).
long logid	nullo	Test di robustezza per garantire che il costruttore gestisca il puntatore null.
	$\neq$ logID esistente nella Directory	Scenario standard di creazione di un nuovo Journal. Verifica che il sistema generi correttamente un nuovo file univoco senza interferire con la cronologia esistente.
	= ad un logID esistente	Scenario in cui il logID specificato appartiene ad un file già presente nella directory. Questa suddivisione si basa sul concetto semantico dell'identificativo univoco.
long prealloc size	$< HeaderSize$	Boundary value inferiore. Verifica il comportamento quando lo spazio pre-allocato è insufficiente persino per contenere l'header obbligatorio del file.
	$HeaderSize \leq preAllocSize < HeaderSize + AlignSize$	Verifica la gestione dello spazio quando l'allocazione è sufficiente per l'header ma non abbastanza grande per formare un blocco di allineamento completo successivo. Testa la logica di calcolo del residuo.
	$\geq HeaderSize + AlignSize \&\& preAllocSize \% AlignSize \neq 0$	Scenario in cui si verifica la logica di allineamento automatico (padding). Poiché la dimensione richiesta non è un multiplo esatto della dimensione di allineamento (AlignSize), il sistema deve essere in grado di arrotondare.
	$\geq HeaderSize + AlignSize \&\& preAllocSize \% AlignSize = 0$	Scenario standard in cui la dimensione di preAlloc è maggiore della dimensione dell'header e perfettamente allineata.
int write buffersize	≤ 0	Verifica che il sistema gestisca input non validi per la dimensione del buffer di scrittura.
int journalalign size	> 0	Caso standard in cui deve essere allocato un writeBuffer.
	≤ 0	Un allineamento nullo o negativo non ha senso fisico nel contesto del file system.
long position (apertura in lettura)	> 0	Scenario Standard in cui le scritture devono essere allineate ad un determinato valore positivo.
	$< header\ size$	Posizione all'interno dell'header. Il sistema dovrebbe gestire con cautela l'accesso a questa sezione.
	$header\ size \leq pos \leq file\ size \&\& pos \% AlignSize \neq 0$	Puntatori a posizioni valide all'interno del fileChannel ma disallineate.
	$header\ size \leq pos \leq file\ size \&\& pos \% AlignSize = 0$	Puntatori a posizioni valide e allineate all'interno del fileChannel
long position (creazione o riuso)	$> fileSize$	Limite superiore (EOF)
	$\neq header\ size$	In caso di logica di creazione o riuso, l'unica posizione corretta deve essere alla fine dell'header.
	$= header\ size$	Stato ideale di inizio scrittura.
boolean fremove frompagecache	true	
int formatversion towrite	false	
	V1	Classi di equivalenza per le versioni supportate.
	V2	
	V3	
	V4	
	V5	
	V6	
	< 1	Verifica la robustezza del validatore di input: il sistema deve rifiutare versioni di protocollo inesistenti.
journal.buffere dchannelbuilder bcbuilder	> 6	Builder configurato correttamente e pronto a istanziare il canale sottostante.
	Istanza valida	Data la natura di Factory del componente, è fondamentale validare la robustezza del sistema a fronte di builder malconfigurati, per evitare che regressioni nelle dipendenze compromettano l'integrità del JournalChannel.
	Istanza invalida	Verifica gestione di input nullo.
filechannelprovider provider	Null	Istanza valida di un provider che restituisce correttamente il fileChannel richiesto.
	Istanza valida	Per le stesse considerazioni sul bcBuilder valutiamo istanze di provider malconfigurati.
	Istanza invalida	Verifica gestione di input nullo.
long toreplacelogid	Null	Scenario di creazione standard. Non è richiesta alcuna sostituzione di log precedenti; il sistema deve procedere alla creazione di un nuovo file se logID non appartiene ad un file esistente.
	Null	Verifica la procedura corretta di ridenominazione del log. Il sistema deve individuare il file corrispondente a toReplaceLogId, troncarlo, e rinominarlo fisicamente sul disco con il nuovo nome derivato da logId.
	$= ID\ esistente \&\& (logID \neq replaceLogID)$	Scenario in cui il target e l'origine coincidono.
	$= ID\ esistente \&\& (logID = replaceLogID)$	Se il file indicato da toReplaceLogId non esiste nella directory, il sistema deve gestire l'impossibilità di effettuare il rename andando ad eseguire l'operazione corretta in base al valore del parametro logID.
	$\neq ID\ esistente$	

Tabella 19 : Boundary Analysis JournalChannel Apertura

parametro	partizione	boundary value	descrizione
file journaldirectory	Corretto	Directory con file a.txn	Rappresenta la pre-condizione ideale. Verifica che il sistema riconosca una directory valida contenente già dei dati, permettendo l'inizializzazione corretta senza errori.
	Valido non corretto	Directory non esistente	Testa la gestione del path. Il percorso è sintatticamente valido (potrebbe esistere), ma fisicamente manca.
	Invalido	File	Verifica il controllo di tipo sul file system. Si passa un percorso che punta a un file regolare invece che a una directory.
	nullo	null	Test di robustezza. Verifica che il costruttore gestisca difensivamente i puntatori nulli.
long logid	≠ logID esistente nella Directory	5	Caso standard di creazione.
	= ad un logID esistente	10	Si utilizza il numero 10 per controllare anche la traduzione in esadecimale.
long preallocsize	< HeaderSize	4	Posizione interna agli headerSize (tranne che per la V1 in cui non c'è header)
	$HeaderSize \leq preAllocSize < HeaderSize + AlignSize$	512 e 1024	A seconda della versione è la dimensione esatta degli header
	$\geq HeaderSize + AlignSize \ \&\& \ preAllocSize \% AlignSize \neq 0$	1025	Si considera l'utilizzo di AlignSize 512 quindi 1025 non è allineato
	$\geq HeaderSize + AlignSize \ \&\& \ preAllocSize \% AlignSize = 0$	1024	Considerando AlignSize 512 la dimensione è perfettamente allineata
int writebuffersize	< 0	-1	Dimensioni negativa del writeBuffer è sempre invalida
	= 0	0	Dimensione nulla del writeBuffer è valida in contesto di apertura in recovery.
	> 0	1	Dimensione minima funzionale.
int journalalignsize	≤ 0	-1 e 0	Valori non validi per l'allineamento fisico.
	> 0	512	Valore standard di settore disco.
long position (apertura in lettura)	< header size	-1	Indice negativo invalido.
	header size ≤ pos ≤ file size && pos%AlignSize ≠ 0	headerSize-1	Tentativo di lettura <i>dentro</i> l'header.
	header size ≤ pos ≤ file size && pos%AlignSize = 0	HeaderSize	È il primo byte valido per il payload utente.
	> fileSize	HeaderSize+1	Secondo byte utile per l'utente da considerare in caso di disallineamento
long position (creazione o riuso)	≠ header size	FileSize	End of File (EOF).
		FileSize+1	Oltre l'EOF.
		-1	Posizioni invalide, sia negativa che ai limiti dell'header.
	= header size	HeaderSize-1	
		HeaderSize+1	
		HeaderSize	L'unica posizione corretta in fase di creazione o riuso.
boolean removefrompagecache	true	True	
	false	False	
int formatversiontowrite	V1	V1	Versione legacy.
	V2	V2	Versione legacy.
	V3	V3	Versione legacy.
	V4	V4	Versione legacy.
	V5	V5	Versione legacy.
	V6	V6	L'ultima versione, utilizzata di default.
	< 1	0	Versione subito dopo la versione meno recente supportata.
	> 6	7	Versione non ancora creata.
journal.buffered channelbuilder bcbuilder	Istanza valida	Restituisce implementazione mock	Simula il comportamento corretto delle dipendenze per isolare il test sulla classe JournalChannel
	Istanza invalida	restituisce null	Fault Injection. Verifica che JournalChannel fallisca in modo controllato
	Null	Null	Test di robustezza. Verifica che il costruttore gestisca difensivamente i puntatori nulli.
filechannelprovider provider	Istanza valida	restituisce un BookieFileChannel	Mock del provider per evitare dipendenze che possano avere regressioni, restituisce un vero BookieFileChannel perché si vuole testare il componente e la sua scrittura su disco.
	Istanza invalida	restituisce null o FileChannelProvider chiuso	Fault Injection. Verifica che JournalChannel fallisca in modo controllato. Inoltre FileChannelProvider implementa l'interfaccia Closable quindi si verifica il comportamento di JournalChannel in caso di provider chiuso.
	Null	Null	Test di robustezza. Verifica che il costruttore gestisca difensivamente i puntatori nulli.

parametro	partizione	boundary value	descrizione
long toreplacelogid	Null	Null	Nessuna sostituzione. Flusso standard di creazione nuovo file o apertura di un file esistente.
	= ID esistente && (logID ≠ replaceLogID)	10	LogID esistente all'interno della directory corretta
	= ID esistente && (logID = replaceLogID)	10	
	≠ ID esistente	1	LogID non esistente.
contenuto del file a.txn	Vuoto	Empty	Differenti opzioni per validare la capacità di lettura di un JournalChannel in una delle versioni supportate. Inoltre considerando che ogni header è composto da MagicWord+VersionNumber, si considerano i due casi in cui gli header sono malformati.
	File versione 1	Hv1 + contenuto	
	File versione 2	Hv2 + contenuto	
	File versione 3	Hv3 + contenuto	
	File versione 4	Hv4 + contenuto	
	File versione 5	Hv5 + contenuto	
	File versione 6	Hv6 + contenuto	
	Header errato (numero di versione 7)	Header con numberVersion 7	
	MagicWorld errata	Header con BLKG "FAIL"	

Tabella 20 : Test Manuali Journal Channel

#	journal directory	logid	prealloc	wbuffer	align	pos	rfpc	version	Bc builder	fc provider	replace logid
creazione (apertura in scrittura): variazione sulla versione											
0	corretto	5	1024	1	512	HS	false	6	valido	valido	null
1	corretto	5	1024	1	512	HS	false	5	valido	valido	null
2	corretto	5	1024	1	512	HS	false	4	valido	valido	null
3	corretto	5	1024	1	512	HS	false	3	valido	valido	null
4	corretto	5	1024	1	512	HS	false	2	valido	valido	null
5	corretto	5	1024	1	512	HS	false	1	valido	valido	null
6	corretto	5	1024	1	512	HS	false	7	valido	valido	null
7	corretto	5	1024	1	512	HS	false	0	valido	valido	null
creazione (apertura in scrittura): variazione su alignment e alloc											
8	corretto	5	1025	1	512	HS	false	6	valido	valido	null
9	corretto	5	512	1	512	HS	false	6	valido	valido	null
10	corretto	5	4	1	512	HS	false	6	valido	valido	null
11	corretto	5	-1	1	512	HS	false	6	valido	valido	null
12	corretto	5	1024	1	-1	HS	false	6	valido	valido	null
13	corretto	5	1024	1	0	HS	false	6	valido	valido	null
creazione (apertura in scrittura): variazione sull'environment (journaldirectory e fremovefrompagecache)											
14	corretto	5	1024	1	512	HS	true	6	valido	valido	null
15	non corretto	5	1024	1	512	HS	false	6	valido	valido	null
16	invalido	5	1024	1	512	HS	false	6	valido	valido	null
17	null	5	1024	1	512	HS	false	6	valido	valido	null
creazione (apertura in scrittura): variazione su builder e provider											
18	corretto	5	1024	1	512	HS	false	6	invalido	valido	null
19	corretto	5	1024	1	512	HS	false	6	null	valido	null
20	corretto	5	1024	1	512	HS	false	6	valido	invalido	null
21	corretto	5	1024	1	512	HS	false	6	valido	null	null
22	corretto	5	1024	1	512	HS	false	6	valido	IOExc	null
riuso (apertura in scrittura)											
23	corretto	10	1024	1	512	HS	false	6	valido	valido	10
24	corretto	1	1024	1	512	HS	false	6	valido	valido	10
25	corretto	5	1024	1	512	HS	false	6	valido	valido	1
esistente (apertura in sola lettura) : variazione contenuto del file											
26	Corretto Hv1 + c	10	1024	1	512	HS	false	6	valido	valido	null
27	Corretto Hv2 + c	10	1024	1	512	HS	false	6	valido	valido	null
28	Corretto Hv3 + c	10	1024	1	512	HS	false	6	valido	valido	null
29	Corretto Hv4 + c	10	1024	1	512	HS	false	6	valido	valido	null
30	Corretto Hv5 + c	10	1024	1	512	HS	false	6	valido	valido	null
31	Corretto Hv6 + c	10	1024	1	512	HS	false	6	valido	valido	null
32	Corretto badMW	10	1024	1	512	HS	false	6	valido	valido	null
esistente (apertura in sola lettura) : variazione sulla posizione											
33	Corretto Hv6 + c	10	1024	1	512	HS-1	false	6	valido	valido	null
34	Corretto Hv6 + c	10	1024	1	512	HS+1	false	6	valido	valido	null
35	Corretto Hv6 + c	10	1024	1	512	FS	false	6	valido	valido	null
36	Corretto Hv6 + c	10	1024	1	512	FS+1	false	6	valido	valido	null
creazione (apertura in scrittura): variazione su buffered channel											
37	corretto	5	1024	-1	512	HS	false	6	valido	valido	null
38	corretto	5	1024	0	512	HS	false	6	valido	valido	null

Tabella 21 : Risultati Attesi Test Manuali JournalChannel

#	risultato atteso
0	Valido: Viene correttamente creato il file 5.txn di dimensione preAllocByte con all'interno l'header V6
1	Valido: Viene correttamente creato il file 5.txn di dimensione preAllocByte con all'interno l'header V5
2	Valido: Viene correttamente creato il file 5.txn di dimensione preAllocByte con all'interno l'header V4
3	Valido: Viene correttamente creato il file 5.txn di dimensione preAllocByte con all'interno l'header V3
4	Valido: Viene correttamente creato il file 5.txn di dimensione preAllocByte con all'interno l'header V2
5	Valido: Viene correttamente creato il file 5.txn di dimensione preAllocByte con all'interno l'header V1
6	Eccezione: versione non supportata. Il file non deve essere creato
7	Eccezione: versione non supportata. Il file non deve essere creato
8	Valido: Viene correttamente creato il file 5.txn di dimensione 1024 con all'interno l'header V6
9	Valido: Viene correttamente creato il file 5.txn di dimensione 512 con all'interno l'header V6
10	Eccezione: il parametro di Preallocazione deve essere maggiore dell'headerSize
11	Eccezione: il parametro di Preallocazione deve essere maggiore dell'headerSize
12	Eccezione: il parametro AlignSize deve essere maggiore di 0
13	Eccezione: il parametro AlignSize deve essere maggiore di 0
14	Valido: viene correttamente creato il file 5.txn di dimensione 1024 con header V6
15	Eccezione: la cartella non esiste
16	Eccezione: la journalDirectory non è una cartella
17	Eccezione: la journalDirectory è nulla
18	Eccezione: il bcBuilder restituisce null
19	Eccezione: il bcBuilder è nullo
20	Eccezione: il fileChannelProvider restituisce null
21	Eccezione: il fileChannelProvider è nullo
22	Eccezione: l'eccezione del fileChannelProvider viene propagata
23	Valido: viene ignorato il valore di toReplaceLogID e il file viene aperto in lettura
24	Valido: il file viene rinominato a 1.txn, troncato a preAllocSize e viene popolato con HeaderV6 e padding
25	Valido: viene ignorato il valore di toReplaceLogID e viene creato un nuovo journalChannel 5.txn
26	Valido: viene aperto il file a.txn e la lettura parte dalla posizione 0
27	Valido: viene aperto il file a.txn e la lettura parte dalla posizione 8
28	Valido: viene aperto il file a.txn e la lettura parte dalla posizione 8
29	Valido: viene aperto il file a.txn e la lettura parte dalla posizione 8
30	Valido: viene aperto il file a.txn e la lettura parte dalla posizione 512
31	Valido: viene aperto il file a.txn e la lettura parte dalla posizione 512
32	Valido: l'header non viene riconosciuto e avviene un fallback alla versione 1
33	Eccezione: la posizione è invalida perché andremmo a leggere l'header
34	Valido: il journalchannel viene correttamente aperto nella posizione richiesta
35	Valido: il journalchannel viene correttamente aperto con posizione l'ultimo byte successiva lettura ritorna EOF (-1)
36	Valido: il journalchannel viene correttamente aperto, successiva lettura ritorna EOF (-1)
37	Eccezione: il writeBuffer non può essere negativo
38	Eccezione: il writeBuffer non può essere 0

Tabella 22 : JournalChannel test Aggiuntivi Prima Iterazione

configurazione	azione	risultato atteso
Creazione fisica su disco di un file "fantasma" corrispondente al logid e configurazione di un mock (bookiefilechannel) per restituire false alla chiamata fileexists(), simulando un ritardo nell'aggiornamento dello stato.	Invocazione del costruttore new JournalChannel(...)	Il metodo deve lanciare una IOException
Creazione di un vecchio file di log (oldlogid) con dati reali. Creazione di uno spy su defaultfilechannelprovider con override doreturn(false).when(spy).supportreusefile().	Invocazione del costruttore passando oldLogId come parametro per richiedere il riuso, ma con il provider che lo nega.	Il file vecchio (oldFile) deve esistere e mantenere la dimensione originale (nessuna rinomina) e il nuovo file viene creato ex-novo.
Creazione manuale su disco di un file .txn avente magic word valida ma con header version impostato a 0.	Invocazione del costruttore new JournalChannel(...) puntando al file artefatto con versione obsoleta.	Il metodo deve lanciare IOException confermando il rifiuto da parte del sistema di elaborare versioni inesistenti.
Creazione manuale su disco di un file .txn avente magic word valida ma con header version impostato a 7.	Invocazione del costruttore new JournalChannel(...) puntando al file artefatto con versione futura.	Il metodo deve lanciare IOException confermando il rifiuto da parte del sistema di elaborare versioni inesistenti
Configurazione di un mock filechannel programmato per lanciare ioexception("simulated seek failure") all'invocazione di position(), unitamente a un mock bookiefilechannel che forza l'apertura in lettura dichiarando che il file esiste.	Invocazione del costruttore new JournalChannel(...)	L'eccezione IOException simulata deve essere catturata e propagata fino al test

Tabella 23 : JournalChannel Test Aggiuntivi Seconda Iterazione

configurazione	azione	risultato atteso
Creazione di un mock del bufferedchannelbuilder configurato con una callback (answer) che ispeziona la posizione del filechannel nel momento esatto in cui viene passato al builder. Versione impostata a 6.	Inizializzazione del costruttore di JournalChannel passando il builder mockato.	All'interno della callback del Mock, il cursore del file (fc.position()) deve trovarsi esattamente all'offset 512.

4.1.3 CacheMap

Tabella 24 : Category Partition CacheMap Inizializzazione

parametro	partizione	descrizione
boolean lru	True	Cache con politica di eviction Least Recently Used.
	False	Cache con politica di eviction Random.
int max	≤ 0	Una capacità massima nulla o negativa è semanticamente invalida per una cache.
	$0 < max < size$	Verifica la gestione del conflitto logico in cui la dimensione iniziale richiesta (size) è superiore alla capacità massima consentita (max).
	$size \leq max$	Scenario standard.
int size	≤ 0	Input invalido.
	> 0	Allocazione valida.
float load	≤ 0	Input invalido. Il fattore di carico, essendo un coefficiente di riempimento, deve essere matematicamente positivo.
	$0 < load \leq 1$	Intervallo di validità standard.
	$load > 1$	Configurazione invalida. Un fattore di carico superiore a 1 è semanticamente sbagliato.
int concurrencylevel	≤ 0	Si considera il parametro di concurrency level il livello di parallelismo atteso negli accessi alla CacheMap per stabilire il numero di strutture di sincronizzazione necessarie. Quindi un livello di parallelismo minore o uguale a 0 è semanticamente errato.
	> 0	Configurazione valida. La mappa deve essere inizializzata con il numero corretto di segmenti di locking per gestire l'accesso concorrente.

Tabella 25 : Boundary Analysis CacheMap Inizializzazione

parametro	partizione	boundary value	descrizione
boolean lru	True	True	Attiva la logica di eviction Least Recently Used.
	False	False	Attiva la logica di eviction Random.
int max	≤ 0	-1	Valore negativo. Verifica che il costruttore rifiuti capacità insensate.
		0	Una cache di dimensione zero è invalida perché non avrebbe senso utilizzare il meccanismo a tre livelli offerto da CacheMap.
	$0 < max < size$	1	Minimo valore positivo minore della dimensione size 2.
	$size \leq max$	2	Valore pari alla size 2. In questo caso non si utilizza il meccanismo di ridimensionamento.
		3	Valore appena superiore alla size. In questo caso si usufruisce del meccanismo di ridimensionamento.
int size	≤ 0	-1	Allocazione negativa invalida.
		0	Allocazione nulla senza senso per la creazione di una CacheMap.
	> 0	2	Valore operativo. Scelto strategicamente per testare le relazioni con max.
float load	≤ 0	-0.0001f	Appena sotto lo zero.
		0f	Zero esatto.
	$0 < load \leq 1$	0.0001f	Appena sopra lo zero.
		1f	Limite superiore.
	$load > 1$	1.0001f	Appena sopra l'uno.
Int concurrencylevel	≤ 0	-1	Concorrenza negativa, valore semanticamente non corretto.
		0	Concorrenza nulla. Non ha senso logico avere 0 thread concorrenti.
	> 0	1	Minimo parallelismo valido. Simula un comportamento single-threaded.

Tabella 26 : Test Manuali CacheMap Inizializzazione

#	lru	max	size	load	concurrency level	risultato atteso
0	false	3	2	0.0001f	1	Valido: la classe viene istanziata correttamente, la dimensione della cacheMap massima è 3 e la cacheMap è vuota
1	true	3	2	0.0001f	1	Valido: la classe viene istanziata correttamente, la dimensione della cacheMap massima è 3 e la cacheMap è vuota.
2	true	1	0	0.0001f	1	Eccezione: la dimensione iniziale non può essere nulla
3	false	3	2	-0.0001f	1	Eccezione: il load non può essere negativo
4	false	3	2	0f	1	Eccezione: il load non può essere 0
5	false	3	2	1f	1	Valido: la classe viene istanziata correttamente, , la dimensione della cacheMap massima è 3 e la cacheMap è vuota
6	false	3	2	1.0001f	1	Eccezione: il load non può essere maggiore di 1
7	false	3	2	0.0001f	-1	Eccezione: il livello di concorrenza atteso non può essere negativo
8	false	3	2	0.0001f	0	Eccezione: il livello di concorrenza atteso non può essere 0
9	false	3	-1	0.0001f	1	Eccezione: la dimensione iniziale della mappa non può essere negativa
10	false	0	0	0.0001f	1	Eccezione: non è possibile avere una mappa con capacità nulle
11	false	1	0	0.0001f	1	Eccezione: non è possibile avere una mappa con capacità iniziali nulle
12	false	-1	2	0.0001f	1	Eccezione: la dimensione massima deve essere maggiore o uguale della size
13	false	0	2	0.0001f	1	Eccezione: la dimensione massima deve essere maggiore o uguale della size
14	false	1	2	0.0001f	1	Eccezione: la dimensione massima deve essere maggiore o uguale della size
15	false	2	2	0.0001f	1	Valido: la mappa verrà istanziata correttamente, la dimensione della cacheMap massima è 3 e la cacheMap è vuota

Tabella 27 : Category Partition CacheMap Put

parametro	partizione	motivazione
object key	null	Verifica la robustezza del metodo a fronte di un input Nullo
	invalid	Le hashMap indicizzano le chiavi rispetto all'hashcode dell'oggetto in input. Si vuole verificare la robustezza rispetto a chiavi che hanno problematiche inerenti all'hashcode.
	Not Present	Rappresenta lo scenario standard di inserimento di una nuova associazione.
	Existing - Hard	Verifica la logica di aggiornamento (<i>Update</i>) di un elemento già residente nella cache principale.
	Existing - Soft	Test critico per la logica di promozione. Verifica che una chiave, precedentemente degradata nella SoftMap, venga correttamente ripristinata nella HardMap a seguito di un aggiornamento, prevenendone l'eviction.
object value	Existing - Pinned	Verifica il rispetto dei vincoli di persistenza. Se la cache supporta il concetto di entry pinned, l'aggiornamento del valore non deve rimuovere il flag di pinning né causare l'eviction dell'elemento, garantendone la permanenza.
	null	Verifica se la cache accetta "buchi" informativi (valori nulli associati a chiavi valide) o se impone vincoli di non-nullità sul payload.
	Valid - Immutable	Scenario nominale. Utilizza oggetti il cui stato non può cambiare (es. Stringhe o Integer).
	Valid - Mutable	Test di integrità referenziale. L'uso di oggetti mutabili serve a discriminare la strategia di memorizzazione: verifica se la put memorizza il riferimento all'oggetto (by-reference) o ne effettua una copia (deep copy). Questo è cruciale per stabilire se modifiche esterne all'oggetto influenzano retroattivamente lo stato della cache.
stato della hardmap	Under Capacity	Verifica il flusso di inserimento diretto. In questo stato, l'aggiunta non deve innescare alcun meccanismo di pulizia; serve a confermare che l'overhead dell'algoritmo di eviction non venga attivato inutilmente.
	At Capacity	Rappresenta lo stato limite che forza l'attivazione delle politiche di eviction. È la partizione fondamentale per verificare che l'inserimento del nuovo elemento N+1 causi la corretta rimozione dell'elemento vittima secondo la politica configurata.
boolean lru	LRU	Verifica il determinismo dell'algoritmo di rimozione. In stato di saturazione, il sistema deve identificare ed eliminare rigorosamente l'elemento meno recentemente utilizzato, preservando quelli più "caldi".
size della softmap	Random = 0	L'alternativa stocastica. Impostando la dimensione della cache secondaria a zero, si elimina il "cuscinetto" delle Soft Reference: ogni elemento espulso dalla HardMap viene perso definitivamente. Questo permette di asserire con certezza quale chiave è stata rimossa verificandone l'assenza (assertNull), isolando la logica di scelta della vittima dal comportamento non deterministico del Garbage Collector.
	> 0	Controlla la corretta migrazione dei dati tra i livelli gerarchici: l'elemento espulso dalla HardMap non deve andare perso, ma deve essere trasferito (downgraded) nella SoftMap, rimanendo recuperabile fino all'intervento del Garbage Collector.

Tabella 28 : Boundary Analysis CacheMap Put

parametro	partizione	boundary value	descrizione
object key	null	null	Test di robustezza per verificare la gestione di riferimenti nulli.
	invalid	hashCodeException	utilizzo di un mock per forzare un'eccezione durante il calcolo dell'hash della chiave, testando la resilienza del metodo a errori di runtime.
	Not Present	Not Present	Verifica il flusso di inserimento di una nuova entry non esistente in alcun livello della gerarchia della cache.
	Existing - Hard	Existing Hard	Verifica la logica di aggiornamento (<i>Update</i>) e il riposizionamento dei metadati per chiavi già presenti nella memoria principale.
	Existing - Soft	Existing Soft	Test della logica di promozione: verifica se l'aggiornamento di una chiave situata nella SoftMap ne provochi il ripristino nella HardMap.
object value	Existing - Pinned	Existing Pinned	Verifica che l'aggiornamento di un elemento pinned non ne alteri lo stato di protezione dall'eviction.
	null	null	Verifica la capacità della cache di gestire payload nulli associati a chiavi valide.
	Valid - Immutable	Stringa	Esempio di oggetto immutabile.
stato della hardmap	Valid - Mutable	MutableObject	Classe utility di test con metodi mutatori, serve a determinare se la cache memorizzi per riferimento o per copia.
	Under Capacity	Size = Max - 1	Punto di controllo pre-limite: verifica che l'inserimento avvenga senza innescare meccanismi di rimozione.
boolean lru	At Capacity	Size = Max	Condizione di frontiera in cui l'inserimento di un'ulteriore entry deve obbligatoriamente attivare la politica di evizione.
	LRU	Random	Logica di eviction Random.
size della softmap	Random	LRU	Logica di eviction Last Recently Used.
	= 0	0	eliminando il buffer secondario, si verifica con certezza l'eliminazione definitiva dell'elemento espulso dalla HardMap
	> 0	1	Verifica dello spostamento delle entry da hardMap a softMap.

Tabella 29 : Test Manuali CacheMap Put

#	key	value	hardmapstatus	lru	size softmap	risultato atteso
0	Not Present	Stringa	Size = Max – 1	Random	0	Valido: La coppia verrà inserita correttamente all'interno della Hard Map e non avverrà alcun eviction, il metodo put ritorna null e la dimensione aumenta di 1 senza alcuna eviction
1	Not Present	Mutable Object	Size = Max – 1	Random	0	Valido: La coppia verrà inserita correttamente all'interno della Hard Map ritornando null e non avverrà alcun eviction, il valore del campo name in Person sarà aggiornato anche alle modifiche successive
2	Not Present	Null	Size = Max – 1	Random	0	Valido: La coppia verrà inserita correttamente all'interno della Hard Map e il valore sarà null, la dimensione della mappa aumenta e non c'è nessuna eviction
3	hashCode Exception	Stringa	Size = Max – 1	Random	0	Eccezione: non verrà eseguita alcuna modifica alla mappa, la dimensione rimane identica
4	Existing Hard	Stringa	Size = Max – 1	Random	0	Valido: Il valore della coppia indicata sarà la nuova stringa, il vecchio valore sarà restituito e la dimensione della mappa rimarrà invariata
5	Existing Soft	Stringa	Size = Max – 1	LRU	1	Valido: la chiave con il nuovo valore salirà nella Hard Map e sarà eliminato dalla Soft Map, la dimensione della mappa rimarrà invariata (anche chiudendo la softMap)
6	Existing Pinned	Stringa	Size = Max – 1	Random	0	Valido: la chiave rimarrà nello stato pinned e cambierà il suo valore
7	Existing Hard	Stringa	Size = Max	Random	0	Valido: la chiave rimarrà nella Hard Map con il nuovo valore e nessuna coppia verrà evictata, il vecchio valore sarà restituito
8	Existing Pinned	Stringa	Size = Max	Random	0	Valido: la chiave rimarrà nello stato pinned e cambierà il suo valore, restituendo il vecchio valore. La dimensione della cache rimarrà invariata
9	Existing Soft	Stringa	Size = Max	LRU	1	Valido: la chiave meno recentemente utilizzata (get o put) sarà evicted mentre la chiave indicata salirà nella Hard Map con il nuovo valore. Verrà restituito il vecchio valore e la dimensione della cache rimarrà invariata. Alla chiusura forzata della SoftMap la dimensione deve scendere di una unità e la chiave scesa in SoftMap deve essere eliminata

Tabella 30 : Category Partition CacheMap Pin

parametro	partizione	motivazione
object key	null	Verifica la capacità del metodo di gestire riferimenti nulli, restituendo un esito negativo.
	invalid	Stesse considerazioni fatte anche per il metodo di Put. Controlliamo la capacità del metodo di gestire input malformati sull'hashCode.
	not present	Verifica dello scenario negativo standard. Il test accerta che il tentativo di ancorare una chiave inesistente restituisca falsee pinni la chiave con valore null.
	existing - hard	Verifica dello scenario nominale di ancoraggio. Controlla che una chiave già residente nella memoria principale venga correttamente marcata come inamovibile, sottraendola di fatto alle future passate degli algoritmi di eviction ed escludendola dal conto della size.
	existing - soft	Verifica il recupero di un oggetto dalla SoftMap e il suo reinserimento nella PinnedMap.
	existing - pinned	Verifica dell'idempotenza.
	under capacity	Condizione operativa standard. Verifica il corretto funzionamento del flag di pinning in assenza di pressione sulla memoria, isolando la logica funzionale di ancoraggio da quella di gestione della saturazione.
stato della hard map	at capacity	Boundary Condition critica. In questo stato, la mappa è piena. Il test verifica che, promuovendo una chiave da Soft a Pinned, il sistema rispetti la specifica architetturale: l'entry pinnata deve smettere di contribuire al calcolo del size, permettendo la coesistenza con le altre entry senza innescare meccanismi di eviction, tollerando di fatto un sovraccarico strutturale

Tabella 31 : Boundary Analysis CacheMap Pin

parametro	partizione	boundary value	descrizione
object key	null	null	Verifica la robustezza del sistema nel prevenire crash dovuti a riferimenti nulli, assicurando che l'input sia validato prima di accedere alle mappe interne.
	invalid	hashCode Exception	Utilizza un mock per simulare un errore durante il calcolo dell'hash della chiave. Serve a testare se il metodo pin gestisce correttamente gli errori di runtime delle dipendenze senza corrompere lo stato della cache.
	not present	Not Present	Verifica la logica di registrazione preventiva. Il test accerta che, qualora la chiave non risieda né nella HardMap né nella SoftMap, essa venga comunque inserita nella struttura di controllo delle chiavi bloccate (pinnedMap). L'esito false deve confermare l'assenza di una referenza preesistente, ma l'operazione deve garantire che l'associazione sia trattata come "pinnata" non appena verrà inserita nel sistema tramite una successiva chiamata al metodo put.
	existing - hard	Existing – Hard	Punto di controllo funzionale: valida la corretta transizione di un'entry già presente da "rimovibile" a "protetta" all'interno della memoria principale.
	existing - soft	Existing – Soft	Punto critico di Promozione: Verifica il caso in cui il pinning forzi il recupero di una chiave dalla SoftMap.
	existing - pinned	Existing - Pinned	Verifica l'idempotenza del metodo.
	under capacity	Size = Max – 1	L'ultimo stato in cui una entry può essere facilmente promossa.
stato della hard map	at capacity	Size = Max	Confine di saturazione assoluta: In questo stato, qualsiasi inserimento standard causerebbe un'eviction. Il test verifica che il pin bypassi questo limite (essendo le chiavi pinned "invisibili" al conteggio).

Tabella 32 : Test Manuali CacheMap Pin

test	key	stato map	hard	risultato atteso
0	null	Size = Max – 1		Eccezione: Nessuna entry sarà pinnata. La dimensione della pinnedMap rimarrà invariata.
1	hashCode Exception	Size = Max – 1		Eccezione: Nessuna entry sarà pinnata. La dimensione della pinnedMap rimarrà invariata.
2	Not Present	Size = Max – 1		Valido: il metodo restituisce false, poiché non ha trovato la chiave nella cacheMap ma questa chiave deve essere pinnata con valore null, la size della mappa non deve cambiare. La dimensione della pinnedMap aumenta di 1.
3	Existing Hard	Size = Max – 1		Valido: la coppia chiave valore viene pinnata. La funzione ritorna true, la dimensione della mappa rimane invariata. La dimensione della pinnedMap aumenta di 1.
4	Existing Soft	Size = Max – 1		Valido: la coppia chiave valore torna all'interno della Hard Map e viene pinnata. La funzione ritorna true e la dimensione della mappa rimane invariata. La dimensione delle pinnedKeys aumenta di 1.
5	Existing Pinned	Size = Max – 1		Valido: La funzione ritorna true perché esiste la chiave all'interno della cacheMap. La dimensione della mappa rimane invariata e anche quella della lista delle pinnedKeys.
6	Existing Soft	Size = Max		Valido: la coppia chiave valore torna all'interno della Hard Map e viene pinnata. La funzione ritorna true. Nessuna chiave dalla Hard Map viene evictata. La dimensione delle pinnedKey aumenta di 1.

Tabella 33 : Category Partition CacheMap Remove

parametro	partizione	motivazioni
object key	null	Test di robustezza volto a garantire che il metodo gestisca correttamente riferimenti nulli
	invalid	Questa categoria modella il comportamento del metodo a fronte di un fallimento strutturale della chiave. L'obiettivo è validare la robustezza della logica di rimozione, assicurando che un errore durante il calcolo dell'indice o la ricerca nelle mappe interne venga gestito correttamente dal sistema, senza compromettere l'integrità globale della cache.
	Not Present	Verifica il comportamento del sistema in assenza della risorsa. Il test accerta che l'operazione sia senza effetti collaterali e restituisca null, confermando che la struttura dati rimanga invariata.
	Existing Hard	Rappresenta lo scenario operativo standard. Valida la corretta rimozione fisica dell'associazione dalla memoria primaria (HardMap) e la restituzione dell'oggetto rimosso al chiamante.
	Existing Soft	Verifica la capacità del metodo di "pulire" la cache in profondità. Il test assicura che l'elemento venga rimosso anche se degradato nella SoftMap, garantendo che una chiave rimossa non possa essere recuperata nemmeno tramite i riferimenti deboli.
	Existing Pinned	Verifica che l'azione di remove sia distruttiva e prevalga sul vincolo di persistenza come descritto nella documentazione. Il test deve confermare la revoca automatica del pinning nella pinnedMap e la contestuale eliminazione fisica dell'entry dalla cache.

Tabella 34 : Boundary Analysis CacheMap Remove

parametro	partizione	boundary value	descrizione
object key	Null	null	Input nullo.
	Invalid	hashCodeException	La funzionalità remove deve essere resistente a chiavi utente con problemi sull'hashCode.
	Not present	Not Present	Verifichiamo che in assenza della chiave specificata non ci sia alcun effetto collaterale nella mappa.
	Existing hard	Existing Hard	L'eliminazione di una chiave presente nella hardMap deve essere definitiva. La chiave non deve essere degradata in softMap.
	Existing Soft	Existing Soft	Verifichiamo il comportamento dell'operazione all'eliminazione di una chiave a riferimento debole.
	Existing Pinned	Existing Pinned	Una chiave pinned, come descritto in documentazione, deve essere "depinnata" in fase di eliminazione.

Tabella 35 : Test Manuali CacheMap Remove

#	key	risultato atteso
0	null	Valido: il metodo deve ritornare null e la dimensione della mappa deve restare invariata
1	hashCodeException	Eccezione: l'eccezione deve essere propagata e la dimensione della mappa rimane invariata.
2	Not Present	Valido: il metodo deve ritornare null e la dimensione della mappa deve restare invariata
3	Existing – Hard	Valido: il metodo ritorna la value associato alla key eliminata e la dimensione della mappa deve essere sceso di uno.
4	Existing – Soft	Valido: il metodo ritorna il valore associato alla key eliminata e la dimensione della mappa diminuisce di uno.
5	Existing - Pinned	Valido: il metodo ritorna il valore associato alla key eliminata, la lista delle pinnedKeys non contiene più la key eliminata e la size della cache diminuisce di uno.

Tabella 36 : Estensione Test Manuali CacheMap Put

#	key	value	max	lru	size	risultato atteso
10	Not Present	Stringa	0	Random	0	Valido: l'esecuzione della funzione put restituisce null e la dimensione della cacheMap rimane invariata a 0. La cache non deve contenere la chiave aggiunta
11	Not Present	Stringa	0	Random	1	Valido: l'esecuzione della funzione put restituisce null e la dimensione della cacheMap rimane invariata a 0.
12	5 Not Present	5 inserimenti di stringhe	1	LRU	5	Verifichiamo la capacità di ridimensionamento della LRU Map. Anche se la size iniziale è 1 la mappa deve riuscire a contenere tutte le chiavi che vengono inserite.

Tabella 37 : Test Aggiuntivi CacheMap prima iterazione

configurazione	azione	risultato atteso
Inizializzazione della cachemap ed esecuzione preliminare di pin(key) su una chiave inesistente per creare una "ghost entry" (chiave presente nei pinned keys ma con valore null). Preparazione della cachemap con una "ghost entry" già presente, ovvero una chiave ghostkey pinnata tramite pin() ma ancora priva di valore (null).	Esecuzione di una seconda chiamata a pin(key) sulla stessa chiave "fantasma". Esecuzione di put(ghostKey, newValue) per trasformare la entry fantasma in una entry reale e valorizzata.	Il metodo deve ritornare false. Nonostante la chiave sia tracciata come pinnata, il sistema deve segnalare che non vi è ancora un valore reale associato da bloccare. Il metodo deve ritornare null (riferito al vecchio valore) e aggiornare contestualmente il valore in mappa a newValue. La chiave deve rimanere nella lista dei pinned keys.

Tabella 38 : Test Aggiuntivi CacheMap seconda iterazione

configurazione	azione	risultato atteso
Cachemap inizializzata con una entry presente. Preparazione di un task asincrono (completablefuture) pronto a leggere la stessa chiave.	Esecuzione del metodo pin(key) sulla chiave esistente.	Il task asincrono deve completarsi con successo entro 1 secondo. Questo conferma che il writeLock acquisito durante la pin è stato correttamente rilasciato, evitando deadlock che bloccherebbero le letture successive.
Cachemap monitorata tramite spy. Una chiave viene inserita e successivamente "pinnata" (spostata nella pinnedmap).	Aggiornamento del valore associato alla chiave pinnata tramite put(key, newValue)	Verifica tramite Spy che vengano chiamati in ordine: entryRemoved (per il vecchio valore) ed entryAdded (per il nuovo valore).
Cachemap monitorata tramite spy. Una chiave viene "pinnata" senza valore (stato ghost entry, valore null).	Inserimento del primo valore reale tramite put(key, newValue).	Verifica tramite Spy che venga chiamato solo entryAdded. Il metodo entryRemoved non deve essere invocato (poiché il valore precedente era nullo). La dimensione della mappa deve incrementare di 1.
Cachemap vuota monitorata tramite spy.	Inserimento di una nuova chiave tramite put(key,value)	Verifica tramite Spy che venga invocato unicamente entryAdded. Nessuna chiamata a entryRemoved deve avvenire.
Riempimento della hard map fino al limite di capacità per forzare l'evizione (spostamento) della prima chiave inserita nella soft map.	Esecuzione di una put sulla chiave che risiede attualmente nella Soft Map (promozione).	Verifica che venga chiamato entryRemoved (per rimuovere il riferimento Soft obsoleto) seguito da entryAdded (per reinserire il valore aggiornato nella Hard Map).
Cachemap monitorata tramite spy contenente una chiave nella hard map.	Aggiornamento del valore della chiave esistente tramite put.	Il sistema deve notificare la rimozione del vecchio valore (entryRemoved) e l'aggiunta del nuovo (entryAdded), confermando il ciclo di vita corretto durante l'update.
Cachemap inizializzata. Preparazione di un thread secondario che tenta di leggere dalla mappa.	Esecuzione di una operazione put(key, value).	Il thread secondario deve riuscire ad accedere alla mappa entro un timeout breve (1s). Ciò dimostra che il blocco di scrittura è stato rilasciato al termine dell'operazione, prevenendo il blocco indefinito delle risorse.
Cachemap monitorata tramite spy contenente una entry valida.	Rimozione della chiave tramite remove(key)	Verifica tramite Spy che il metodo entryRemoved sia stato invocato esattamente una volta con i parametri corretti (chiave e valore rimosso)
Cachemap con una entry presente. Setup di un controllo asincrono di lettura	Esecuzione di remove(key)	Il controllo asincrono deve avere successo immediato. Questo uccide i mutanti che rimuovono l'istruzione writeLock.unlock() all'interno del blocco finally della remove

4.1.4 ProxyManagerImpl

Tabella 39 : Category Partition Strumentazione ProxyManagerImpl

parametro	partizione	descrizione
Object orig	Null	Serve a testare la robustezza dell'implementazione rispetto a input nullo. Il manager non deve restituire alcun proxy, ma deve restituire null.
	Final Class	Serve a testare la robustezza dell'implementazione. Anche in questo caso, non potendo strumentare la classe, il manager deve restituire null.
	java.lang.reflect.Proxy	Serve a testare la robustezza dell'implementazione di fronte a oggetti privi di una classe concreta, assicurando che il manager gestisca correttamente i proxy dinamici restituendo null.
	AlreadyProxy	Serve a testare la robustezza dell'implementazione di fronte al passaggio degli stessi proxy da lei creati, verificando che non avvenga un annidamento dei proxy.
	Managed	Mentre il ProxyManagerImpl si occupa di rendere proxy i campi di un oggetto, la Persistence Capable rende "intelligente" l'intero oggetto. Quindi si vuole verificare che il manager non applichi un Proxy ad un oggetto managed di OpenJPA.
	Collection	L'insieme di tutte le classi che implementano l'interfaccia Collection che devono essere correttamente strumentate dal manager.
	Ordered Collection	L'insieme di tutte le classi che implementano Collection e hanno un comparatore che ordina i loro elementi. Si vuole testare che la logica comparativa rimanga anche all'interno della copia Proxy.
	Map	L'insieme di tutte le classi che implementano l'interfaccia Map che devono essere correttamente strumentate dal manager.
	Ordered Map	L'insieme di tutte le classi che implementano Map e hanno un comparatore che ordina i loro elementi. Si vuole testare che la logica comparativa rimanga anche all'interno della copia Proxy.
	Date	L'insieme di tutte le classi che implementano l'interfaccia Date che devono essere correttamente strumentate dal manager.
	Calendar	L'insieme di tutte le classi che implementano l'interfaccia Calendar che devono essere correttamente strumentate dal manager.
	Custom Bean	L'insieme di tutte le classi che non implementano le interfacce precedenti (esplicitamente dichiarate proxable) e hanno un costruttore pubblico senza argomenti e per ogni parametro possiedono un getter e un setter.
	No Bean	L'insieme di tutte le classi che non implementano le interfacce precedenti e non hanno un costruttore pubblico senza argomenti o non hanno per ogni parametro il corrispettivo getter e setter.
boolean autooff	true	
	false	
boolean trackchanges	true	
	false	
boolean assertallowedtype	true	
	false	
boolean delaycollectionloading	true	
	false	
String unproxyable	empty	Categoria in cui si definisce una stringa che non contiene caratteri o contiene solamente spazi e/o punti e virgola
	multiple list with = orig	Categoria in cui la stringa contiene la specifica classe che viene passata come parametro Object orig.
	multiple list with interface of orig	Categoria in cui la stringa contiene l'interfaccia che l'oggetto passato in input implementa.
	multiple list with no match null	Categoria in cui ci sono tutte le liste che contengono solo rumore

Tabella 40 : Boundary Analysis Strumentazione ProxyManagerImpl

parametro	partizione	boundary value	considerazioni
Object orig	Null	Null	
	Final Class	java.lang.String	
	java.lang.reflect.Proxy	Proxy che implementa java.util.List	
	AlreadyProxy	Istanza di DelayedArrayListProxy	
	Managed	mock di PersistenceCapable	
	Collection	Istanza di java.util.ArrayList e ArrayDeque	
	Ordered Collection	Istanza di java.util. TreeSet	
	Map	Istanza di java.util.HashMap	
	Ordered Map	Istanza di java.util.TreeMap	
	Date	java.sql.Timestamp	Il timestamp ha precisione al nanosecondo, si vuole verificare che questo non venga convertito in un semplice Date ma deve mantenere la precisione.
	Calendar	java.util.GregorianCalendar	
	Custom Bean	CustomBean	Implementazione di un Bean con un parametro stringa
	No Bean	CustomNoBeanConstructor CustomNoBeanParameter	e Sono due classi progettate a scopo di test, una dove non è presente il costruttore vuoto, l'altra ha un attributo int senza getter e setter.
boolean autooff	true	true	
	false	false	
boolean trackchanges	true	true	
	false	false	
boolean assertallowedtype	true	true	
	false	false	
boolean delaycollectionloading	true	true	
	false	false	
String unproxyable	empty	" " e "..."	Con questi input "cattivi" si cerca di verificare se l'implementazione preveda la gestione dei separatori vuoti
	multiple list with = orig	"NotClassOrig;ClassOrig"	Verifica che il manager sappia scorrere una lista. Se si fermasse al primo elemento perché non corrisponde, fallirebbe il test.
	multiple list with interface of orig	"NotClassOrig;java.util.Collection"	Si verifica se il manager valuta la lista unproxyable in base ad una uguaglianza esplicita o anche per interfaccia
	multiple list with no match	"NotClassOrig;AnotherNotClassOrig"	Semplice lista in cui non ci sono classi inerenti ad orig
	null	null	

Tabella 41 : Test Manuali Strumentazione ProxymanagerImpl

#	orig	Auto off	track changes	assertallowedtype	delaycollloading	unproxyable
0	null	false	true	false	false	" "
1	String	false	true	false	false	" "
2	java.lang.reflect.Proxy	false	true	false	false	" "
3	DelayedArrayListProxy	false	true	false	false	" "
4	Mock PersistenceCapable	false	true	false	false	" "
5	ArrayList	false	true	false	false	" "
6	TreeSet	false	true	false	false	" "
7	HashMap	false	true	false	false	" "
8	TreeMap	false	true	false	false	" "
9	Timestamp	false	true	false	false	" "
10	GregorianCalendar	false	true	false	false	" "
11	CostumSimpleBeam	false	true	false	false	" "
12	CustomNoBeanConstructor	false	true	false	false	" "
13	CustomNoBeanParameter	false	true	false	false	" "
14	ArrayList	false	true	false	false	"Vector;ArrayList"
15	ArrayList	false	true	false	false	"Vector;Collection"
16	ArrayList	false	true	false	false	"Vector;Stack"
17	ArrayList	false	true	false	false	"..."
18	ArrayList	false	true	false	false	" "
19	ArrayList	false	false	false	false	" "
20	ArrayList	false	true	true	false	" "
21	HashMap	false	true	true	false	" "
22	ArrayDeque	false	true	false	true	" "
23	HashMap	false	true	false	true	" "
24	TreeSet	false	true	false	true	" "
25	ArrayList	true	true	false	false	" "

Tabella 42 : Risultati Attesi Test Manuali Strumentazione ProxymanagerImpl

#	risultato atteso
0	Deve terminare immediatamente restituendo un riferimento nullo.
1	Deve restituire null.
2	Deve restituire null.
3	Il manager deve riconoscere l'oggetto come già proxato e restituire la medesima istanza per evitare ricorsione.
4	Deve restituire null.
5	Viene restituita un'istanza proxy di ArrayList contenente tutti gli elementi originali.
6	Il proxy deve essere creato preservando il Comparator originale per garantire l'ordinamento degli elementi.
7	Viene generata una ProxyMap che riflette esattamente le coppie chiave-valore della mappa sorgente.
8	Viene creato un proxy di tipo SortedMap mantenendo intatta la logica di comparazione delle chiavi.
9	Il proxy generato deve conservare la precisione dei nanosecondi specifica del tipo SQL, non limitandosi ai millisecondi della data standard.
10	Il proxy deve replicare lo stato interno del calendario, incluso il TimeZone.
11	Viene creato un proxy per il bean custom che ne preserva le proprietà.
12	Deve restituire null.
13	Deve restituire null.
14	Deve restituire null.
15	Deve restituire null.
16	Il proxy deve essere generato regolarmente poiché la lista di esclusione non riguarda l'oggetto in esame.
17	Il manager deve ignorare i separatori vuoti e procedere con la generazione del proxy standard.
18	Il manager deve ignorare gli spazi e procedere con la generazione del proxy standard.
19	La collezione proxy non deve implementare l'interfaccia org.apache.openjpa.util.ChangeTracker.
20	Il proxy deve configurare il controllo dei tipi per impedire l'inserimento di elementi non conformi ai metadati. Se proviamo ad inserire un campo non conforme il Proxy deve restituire una eccezione.
21	Il proxy deve configurare il controllo dei tipi per impedire l'inserimento di elementi non conformi ai metadati. Se proviamo ad inserire un campo non conforme il Proxy deve restituire una eccezione.
22	Il manager deve istanziare un Proxy che implementa l'interfaccia DelayedProxy.
23	Il manager deve istanziare un Proxy che implementa l'interfaccia DelayedProxy.
24	Il manager deve istanziare un Proxy che implementa l'interfaccia DelayedProxy.
25	Viene restituita un'istanza proxy di ArrayList contenente tutti gli elementi originali.

Tabella 43 : Category Partition Destrumentazione ProxyManagerImpl

parametro	partizione	descrizione
Object orig	Null	Verifica la stabilità basica. Il metodo deve restituire null senza sollevare eccezioni
	Non Proxy Cloneable	Questa categoria è composta da tutte le classi che implementano l'interfaccia Cloneable e quindi sono clonabili tramite la funzione clone().
	Non Proxy Con Copy Constructor	Questa categoria è composta da tutte le classi che implementano un costruttore che accetta l'oggetto stesso come parametro, implementando così il pattern idiomatrico per la copia delle istanze.
	Non Proxy Bean	Questa categoria è composta da tutti i Bean che possiedono un costruttore vuoto e che possono essere clonati tramite getter e setter.
	Non Proxy No Bean	Questa categoria comprende tutte le classi che non sono Bean, non implementano Cloneable e non hanno un copy constructor.
	Proxy CustomBean	Questa categoria comprende tutti i CustomBean che sono stati strumentalizzati dal Manager.
	Managed	Mentre il ProxyManagerImpl si occupa di rendere proxy i campi di un oggetto, la Persistence Capable rende "intelligente" l'intero oggetto. Quindi si vuole verificare che il manager non duplichi un oggetto managed, ma restituisca null.
	Proxy Collection	Categoria che comprende tutti i Proxy di classi che implementano l'interfaccia Collection
	Proxy Ordered Collection	Categoria che comprende tutti i Proxy di classi che implementano l'interfaccia Collection con comparatore
	Proxy Map	Categoria che comprende tutti i Proxy di classi che implementano l'interfaccia Map
	Proxy Ordered Map	Categoria che comprende tutti i Proxy di classi che implementano l'interfaccia Map con comparatore
	Proxy Date	Categoria che comprende tutti i Proxy di classi che implementano l'interfaccia Date
	Proxy Calendar	Categoria che comprende tutti i Proxy di classi che implementano l'interfaccia Calendar

Tabella 44 : Boundary Analysis Destrumentazione ProxyManagerImpl

parametro	categoria	baoundary value	motivazione
Object orig	Null	Null	Caso limite di input. Verifica che il metodo non sollevi NullPointerException (NPE).
	Non Proxy Cloneable	java.util.HashSet e CustomCloneable	È un oggetto mutabile complesso che implementa Cloneable. Verifica dell'utilizzo di clone.
	Non Proxy Con Copy Constructor	CustomCpyConstructor	Una classe con copy Cunstructor. Verifica dell'utilizzo del CopyConstructor come meccanismo di copia.
	Non Proxy Bean	CustomSimpleBean	Un POJO con costruttore pubblico e String info. Per la verifica del meccanismo di Copy reflection.
	Non Proxy No Bean	CustomNoBeanConstructor e CustomNoBeanParameter	Le stesse classi utilizzate nel meccanismo di newCustomProxy.
	Proxy CustomBean	CustomSimpleBean Proxy	Verifica che un bean utente, una volta de-strumentalizzato, torni a essere un semplice POJO senza logiche OpenJPA.
	Managed	Mock PersistenceCapable	Verifica che il manager non effettui una copia di un oggetto strumentalizzato di OpenJPA ma restituisca null.
	Proxy Collection	ArrayListProxy con 10 elementi	Verifica la corretta destrumentalizzazione della Collection
	Proxy Ordered Collection	TreeSetProxy con Comparator	Verifica la corretta destrumentalizzazione di una collection con comparator
	Proxy Map	HashMapProxy con 10 chiavi valore	Verifica la corretta destrumentalizzazione di una Map.
	Proxy Ordered Map	TreeMapProxy	Verifica che la de-strumentazione produca un TreeMap standard e non un HashMap, preservando l'ordinamento delle chiavi.
	Proxy Date	java.sql.Timestamp Proxy	È il limite della precisione: verifica che la copia mantenga i nanosecondi e non venga degradata a semplice java.util.Date.
	Proxy Calendar	GregorianCalendar proxy	Verifica la copia profonda di TimeZone e Locale. Una copia superficiale potrebbe cambiare la data in base al fuso orario del server.

Tabella 45 : Test Suite manuale Destrumentazione ProxyManagerImpl

#	orig	risultato atteso
0	Null	L'operazione restituisce null.
1	java.util.HashSet	L'operazione restituisce una nuova istanza di HashSet con tutti i dati all'interno.
2	CustomClonable	Nuova istanza di CustomClonable con tutti i dati all'interno.
3	CustomCpyConstructor	Nuova istanza di CustomCpyConstructor con i campi copiati correttamente.
4	CustomSimpleBean	Nuova istanza di CustomSimpleBean con i campi copiati correttamente.
5	CustomNoBeanConstructor	L'operazione restituisce null.
6	CustomNoBeanParameter	L'operazione restituisce null.
7	CustomSimpleBean Proxy	Istanza di CustomSimpleBean che non implementa l'interfaccia Proxy con tutti i campi copiati correttamente.
8	Mock PersistenceCapable	L'operazione restituisce null.
9	ArrayListProxy con 10 elementi	Nuova istanza di ArrayList che non implementa l'interfaccia Proxy con tutti gli elementi copiati correttamente.
10	TreeSetProxy con Comparator	Nuova istanza di TreeSet che non implementa l'interfaccia Proxy con tutti gli elementi copiati correttamente e che mantengono lo stesso ordine.
11	HashMapProxy con 10 chiavi valore	Nuova istanza di HashMap che non implementa l'interfaccia Proxy con tutti gli elementi copiati correttamente.
12	TreeMapProxy	Nuova istanza di TreeMap che non implementa l'interfaccia Proxy con tutti gli elementi copiati correttamente e che mantengono lo stesso ordine.
13	java.sql.Timestamp Proxy	Nuova istanza di TimeStamp che non implementa l'interfaccia Proxy con la copia di tutto il timestamp (anche i nanosecondi).
14	GregorianCalendar proxy	Nuova istanza di GregorianCalendar che non implementa l'interfaccia Proxy con tutti gli elementi copiati correttamente.

Tabella 46 : Jacoco ProxyManagerImpl Destrumentazione Aumento Coverage

#	input	metodo testato	risultato atteso
15	GregorianCalendar con TimeZone "GMT+5"	copyCustom	Verifica che il metodo generico riconosca un GregorianCalendar standard e ne copi correttamente anche la TimeZone (deep copy degli attributi), restituendo un oggetto standard e non un proxy.
16	java.sql.Timestamp con nanosecondi impostati (123456)	copyCustom	Verifica che la copia generica di un Timestamp SQL non perda la precisione dei nanosecondi (evitando il downgrade a semplice Date).
17	HashMap<String, Integer> con 10 elementi	copyCustom	Verifica che il metodo generico gestisca correttamente la clonazione di una HashMap standard popolata, preservando tutte le entry.
18	Null	copyMap	Verifica l'uscita anticipata (guard clause) quando l'input è nullo.
19	Null	copyDate	Verifica che la copia di una data nulla ritorni null.
20	null	copyCalendar	Verifica che la copia di un calendar nullo ritorni null.
21	Un List generato tramite newCollectionProxy (istanza di Proxy). Una Map generata tramite newMapProxy (istanza di Proxy)	copyCollection	Verifica che, se l'input è già un ProxyCollection, venga invocato il metodo copy del proxy stesso invece di usare la logica standard.
22	Una Map generata tramite newMapProxy (istanza di Proxy)	copyMap	Verifica la delega al proxy per le mappe, assicurando che il contenuto venga preservato.
23	Una java.util.Date generata tramite newDateProxy	copyDate	Verifica la delega per i tipi Date.
24	Un GregorianCalendar generato tramite newCalendarProxy	copyCalendar	Verifica la delega per i tipi Calendar.
25	Array di stringhe: new String[] { "A", "B", "C" }.	copyArray	Verifica la corretta clonazione di un array valido.
26	null	copyArray	Verifica che la copia di un array nullo ritorni null.
27	Oggetto non valido (Stringa): "Non sono un array"	copyArray	Essendo il parametro di copyArray un Object generico dobbiamo verificare che al passaggio di un oggetto che non è un array venga lanciata una eccezione

Tabella 47 : Jacoco ProxyManagerImpl Strumentazione Aumento Coverage

#	input	metodo testato	risultato atteso
26	java.util.Date con timestamp fisso	newCustomProxy	Verifica che una java.util.Date pura venga proxata correttamente senza essere confusa con un Timestamp, preservando solo i millisecondi.
27	HashSet popolato. Config: delay=true	newCustomProxy	Verifica la creazione di un DelayedHashSetProxy quando il delayed loading è attivo.
28	LinkedList popolata. Config: delay=true	newCustomProxy	Verifica la creazione di un DelayedLinkedListProxy.
29	Vector popolato. Config: delay=true	newCustomProxy	Verifica la creazione di un DelayedVectorProxy.
30	LinkedHashSet popolata. Config: delay=true	newCustomProxy	Verifica la creazione di un DelayedLinkedHashSetProxy.
31	Interfaccia SortedSet.class. Config: delay=true	newCollectionProxy	Verifica che l'interfaccia SortedSet venga mappata concretamente su DelayedTreeSetProxy.
32	PriorityQueue popolata. Config: delay=true	newCustomProxy	Verifica la creazione di un DelayedPriorityQueueProxy.
33	HashMap popolata. Config: delay=true	newCustomProxy	Verifica che tipi non supportati dal delayed loading (HashMap) vengano comunque strumentalizzati come standard proxy.
34	Istanza di FreshHashMapForTest (Custom Class)	newCustomProxy	Forza la generazione ASM per una Mappa. Poiché la classe di input non è java.util.HashMap ma una sottoclasse custom, la cache viene ignorata.
35	Istanza di FreshTreeMapForTest (Custom Class)	newCustomProxy	Forza la generazione ASM per una SortedMap custom. Verifica che il bytecode generato gestisca correttamente il passaggio del Comparator.
36	Istanza di FreshDateForTest (Custom Class)	newCustomProxy	Forza la generazione ASM per una Date custom. Verifica che il bytecode implementi correttamente l'interfaccia ProxyDate.
37	Istanza di FreshDateForTest (Custom Class)	newCustomProxy	Forza la generazione ASM per un Calendar custom. Verifica la copia profonda di Time e TimeZone nel bytecode generato ex-novo.
38	Classe DateWithOnlyLongConstructor (Custom Class)	newCustomProxy	Passando una classe custom priva di costruttore di default, verifica che il bytecode generato ne "inietti" uno sintetico per rendere il proxy istanziabile.
39	Classe DateBroken (Custom Class)	newCustomProxy	Verifica che venga lanciata UnsupportedOperationException se la classe custom non ha costruttori compatibili, impedendo la generazione del bytecode.

Tabella 48 : Test Aggiuntivi ProxyManagerImpl Seconda Iterazione

configurazione	azione	risultato atteso
Istanza di proxymanager	Invocazione del metodo copyCollection(null)	Il metodo deve restituire null.
Configurazione del proxymanager con la classe customsimplebean aggiunta esplicitamente alla lista delle classi unproxyable	Tentativo di creazione di un nuovo proxy per un'istanza di CustomSimpleBean.	Il metodo deve restituire null, confermando che il manager consulta correttamente la lista delle esclusioni e blocca la generazione del proxy per le classi configurate come non supportate.

4.2 Immagini

4.2.1 BufferedChannel

Immagine 2 : Jacoco BufferedChannel Test Manuali

BufferedChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
BufferedChannel(ByteBufAllocator, FileChannel, int)		0%		n/a	1	1	2	2	1	1
flushAndForceWrite(boolean)		0%		n/a	1	1	3	3	1	1
flushAndForceWriteIfRegularFlush(boolean)		0%		0%	2	2	3	3	1	1
clear()		0%		n/a	1	1	3	3	1	1
getFileChannelPosition()		0%		n/a	1	1	1	1	1	1
getNumOfBytesInWriteBuffer()		0%		n/a	1	1	1	1	1	1
getUnpersistedBytes()		0%		n/a	1	1	1	1	1	1
position()		0%		n/a	1	1	1	1	1	1
read(ByteBuf, long, int)		99%		90%	2	11	1	28	0	1
close()		93%		50%	1	2	1	6	0	1
write(ByteBuf)		100%		100%	0	6	0	21	0	1
BufferedChannel(ByteBufAllocator, FileChannel, int, int, long)		100%		100%	0	2	0	11	0	1
forceWrite(boolean)		100%		50%	1	2	0	7	0	1
flush()		100%		50%	1	2	0	6	0	1
BufferedChannel(ByteBufAllocator, FileChannel, int, long)		100%		n/a	0	1	0	2	0	1
Total	45 of 413	89%	7 of 40	82%	14	35	17	96	8	15

Immagine 3 : Jacoco BufferedChannel Test Manuali

```
87.     public BufferedChannel(ByteBufAllocator allocator, FileChannel fc, int writeCapacity, int readCapacity,
88.                             long unpersistedBytesBound) throws IOException {
89.         super(fc, readCapacity);
90.         this.writeCapacity = writeCapacity;
91.         this.position = fc.position();
92.         this.writeBufferStartPosition.set(position);
93.         this.writeBuffer = allocator.directBuffer(writeCapacity);
94.         this.unpersistedBytes = new AtomicLong(0);
95.         this.unpersistedBytesBound = unpersistedBytesBound;
96.         this.doRegularFlushes = unpersistedBytesBound > 0;
97.     }
98. }
```

Immagine 4 : Jacoco BufferedChannel Test Manuali

```
109. /**
110.  * Write all the data in src to the {@link FileChannel}. Note that this function can
111.  * buffer or re-order writes based on the implementation. These writes will be flushed
112.  * to the disk only when flush() is invoked.
113.  *
114.  * @param src The source ByteBuffer which contains the data to be written.
115.  * @throws IOException if a write operation fails.
116.  */
117. public void write(ByteBuf src) throws IOException {
118.     int copied = 0;
119.     boolean shouldForceWrite = false;
120.     synchronized (this) {
121.         int len = src.readableBytes();
122.         while (copied < len) {
123.             int bytesToCopy = Math.min(src.readableBytes() - copied, writeBuffer.writableBytes());
124.             writeBuffer.writeBytes(src, src.readerIndex() + copied, bytesToCopy);
125.             copied += bytesToCopy;
126.
127.             // if we have run out of buffer space, we should flush to the
128.             // file
129.             if (!writeBuffer.isWritable()) {
130.                 flush();
131.             }
132.             position += copied;
133.             if (doRegularFlushes) {
134.                 unpersistedBytes.addAndGet(copied);
135.                 if (unpersistedBytes.get() >= unpersistedBytesBound) {
136.                     flush();
137.                     shouldForceWrite = true;
138.                 }
139.             }
140.         }
141.     }
142.     if (shouldForceWrite) {
143.         forceWrite(false);
144.     }
145. }
```

Immagine 5 : Jacoco BufferedChannel Test Manuali

```

244.     @Override
245.     public synchronized int read(ByteBuf dest, long pos, int length) throws IOException {
246.         if (dest.writableBytes() < length) {
247.             throw new IllegalArgumentException("dest buffer remaining capacity is not enough"
248.                 + "(must be at least as \"length\"=" + length + ")");
249.         }
250.
251.         long prevPos = pos;
252.         while (length > 0) {
253.             // check if it is in the write buffer
254.             if (writeBuffer != null && writeBufferStartPosition.get() <= pos) {
255.                 int positionInBuffer = (int) (pos - writeBufferStartPosition.get());
256.                 int bytesToCopy = Math.min(writeBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
257.
258.                 if (bytesToCopy == 0) {
259.                     throw new IOException("Read past EOF");
260.                 }
261.
262.                 dest.writeBytes(writeBuffer, positionInBuffer, bytesToCopy);
263.                 pos += bytesToCopy;
264.                 length -= bytesToCopy;
265.             } else if (writeBuffer == null && writeBufferStartPosition.get() <= pos) {
266.                 // here we reach the end
267.                 break;
268.                 // first check if there is anything we can grab from the readBuffer
269.             } else if (readBufferStartPosition <= pos && pos < readBufferStartPosition + readBuffer.writerIndex()) {
270.                 int positionInBuffer = (int) (pos - readBufferStartPosition);
271.                 int bytesToCopy = Math.min(readBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
272.                 dest.writeBytes(readBuffer, positionInBuffer, bytesToCopy);
273.                 pos += bytesToCopy;
274.                 length -= bytesToCopy;
275.                 // let's read it
276.             } else {
277.                 readBufferStartPosition = pos;
278.
279.                 int readBytes = fileChannel.read(readBuffer.internalNioBuffer(0, readCapacity),
280.                     readBufferStartPosition);
281.                 if (readBytes <= 0) {
282.                     throw new IOException("Reading from filechannel returned a non-positive value. Short read.");
283.                 }
284.                 readBuffer.writerIndex(readBytes);
285.             }
286.         }
287.         return (int) (pos - prevPos);
288.     }

```

Immagine 6 : Jacoco BufferedChannel Test Manuali

```

207.     /**
208.      * force a sync operation so that data is persisted to the disk.
209.      * @param forceMetadata
210.      * @return
211.      * @throws IOException
212.      */
213.     public long forceWrite(boolean forceMetadata) throws IOException {
214.         // This is the point up to which we had flushed to the file system page cache
215.         // before issuing this force write hence is guaranteed to be made durable by
216.         // the force write, any flush that happens after this may or may
217.         // not be flushed
218.         long positionForceWrite = writeBufferStartPosition.get();
219.         /**
220.          * since forceWrite method is not called in synchronized block, to make
221.          * sure we are not undercounting unpersistedBytes, setting
222.          * unpersistedBytes to the current number of bytes in writeBuffer.
223.          *
224.          * since we are calling fileChannel.force, bytes which are written to
225.          * filechannel (system filecache) will be persisted to the disk. So we
226.          * dont need to consider those bytes for setting value to
227.          * unpersistedBytes.
228.          *
229.          * In this method fileChannel.force is not called in synchronized block, so
230.          * we are doing best efforts to not overcount or undercount unpersistedBytes.
231.          * Hence setting writeBuffer.readableBytes() to unpersistedBytes.
232.          */
233.         if (unpersistedBytesBound > 0) {
234.             synchronized (this) {
235.                 unpersistedBytes.set(writeBuffer.readableBytes());
236.             }
237.         }
238.     }

```

Immagine 7 : Jacoco BufferedChannel Test Manuali

```

192.     /**
193.      * Write any data in the buffer to the file and advance the writeBufferPosition.
194.      * Callers are expected to synchronize appropriately
195.      *
196.      * @throws IOException if the write fails.
197.      */
198.     public synchronized void flush() throws IOException {
199.         ByteBuffer toWrite = writeBuffer.internalNioBuffer(0, writeBuffer.writerIndex());
200.         do {
201.             fileChannel.write(toWrite);
202.         } while (toWrite.hasRemaining());
203.         writeBuffer.clear();
204.         writeBufferStartPosition.set(fileChannel.position());
205.     }

```

Immagine 8 : Jacoco BufferedChannel dopo prima iterazione

BufferedChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
BufferedChannel(ByteBufAllocator, FileChannel, int)	0%	0%	n/a	1	1	2	1
flushAndForceWrite(boolean)	0%	0%	n/a	1	1	3	1
flushAndForceWriteIfRegularFlush(boolean)	0%	0%	n/a	2	2	3	1
clear()	0%	0%	n/a	1	1	3	1
getFileChannelPosition()	0%	0%	n/a	1	1	1	1
getNumOfBytesInWriteBuffer()	0%	0%	n/a	1	1	1	1
getUnpersistedBytes()	0%	0%	n/a	1	1	1	1
close()	93%	50%	1	2	1	6	0
read(ByteBuf, long, int)	100%	100%	0	11	0	28	0
write(ByteBuf)	100%	100%	0	6	0	21	0
BufferedChannel(ByteBufAllocator, FileChannel, int, int, long)	100%	100%	0	2	0	11	0
forceWrite(boolean)	100%	100%	0	2	0	7	0
flush()	100%	100%	0	2	0	6	0
BufferedChannel(ByteBufAllocator, FileChannel, int, long)	100%	n/a	0	1	0	2	0
position()	100%	n/a	0	1	0	1	0
Total	41 of 413	90%	3 of 40	92%	9	35	15

Immagine 9 : Pit Test Prima Iterazione Mutation Coverage

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	84% 81/96	74% 45/61	83% 45/54

Immagine 10 : Pit Test BufferedChannel Prima Iterazione Inizializzazione

```

87     public BufferedChannel(ByteBufAllocator allocator, FileChannel fc, int writeCapacity, int readCapacity,
88                             long unpersistedBytesBound) throws IOException {
89         super(fc, readCapacity);
90         this.writeCapacity = writeCapacity;
91         this.position = fc.position();
92         this.writeBufferStartPosition.set(position);
93         this.writeBuffer = allocator.directBuffer(writeCapacity);
94         this.unpersistedBytes = new AtomicLong(0);
95         this.unpersistedBytesBound = unpersistedBytesBound;
96         this.doRegularFlushes = unpersistedBytesBound > 0;
97     }

```

Immagine 11 : Pit Test BufferedChannel Prima Iterazione Write

```

109     /**
110      * Write all the data in src to the {@link FileChannel}. Note that this function can
111      * buffer or re-order writes based on the implementation. These writes will be flushed
112      * to the disk only when flush() is invoked.
113      *
114      * @param src The source ByteBuffer which contains the data to be written.
115      * @throws IOException if a write operation fails.
116      */
117     public void write(ByteBuf src) throws IOException {
118         int copied = 0;
119         boolean shouldForceWrite = false;
120         synchronized (this) {
121             int len = src.readableBytes();
122             while (copied < len) {
123                 int bytesToCopy = Math.min(src.readableBytes() - copied, writeBuffer.writableBytes());
124                 writeBuffer.writeBytes(src, src.readerIndex() + copied, bytesToCopy);
125                 copied += bytesToCopy;
126             }
127             // if we have run out of buffer space, we should flush to the
128             // file
129             if (!writeBuffer.isWritable()) {
130                 flush();
131             }
132             position += copied;
133             if (doRegularFlushes) {
134                 unpersistedBytes.addAndGet(copied);
135                 if (unpersistedBytes.get() >= unpersistedBytesBound) {
136                     flush();
137                     shouldForceWrite = true;
138                 }
139             }
140         }
141         if (shouldForceWrite) {
142             forceWrite(false);
143         }
144     }

```

Immagine 12 : Pit Test BufferedChannel Prima Iterazione flush

```

192  /**
193   * Write any data in the buffer to the file and advance the writeBufferPosition.
194   * Callers are expected to synchronize appropriately
195   *
196   * @throws IOException if the write fails.
197   */
198  public synchronized void flush() throws IOException {
199      ByteBuffer toWrite = writeBuffer.internalNioBuffer(0, writeBuffer.writerIndex());
200      do {
201          fileChannel.write(toWrite);
202      } while (!toWrite.hasRemaining());
203      writeBuffer.clear();
204      writeBufferStartPosition.set(fileChannel.position());
205  }

```

Immagine 13 : Pit Test BufferedChannel Prima Iterazione forceWrite

```

213  public long forceWrite(boolean forceMetadata) throws IOException {
214      // This is the point up to which we had flushed to the file system page cache
215      // before issuing this force write hence is guaranteed to be made durable by
216      // the force write, any flush that happens after this may or may
217      // not be flushed
218      long positionForceWrite = writeBufferStartPosition.get();
219      /*
220       * since forceWrite method is not called in synchronized block, to make
221       * sure we are not undercounting unpersistedBytes, setting
222       * unpersistedBytes to the current number of bytes in writeBuffer.
223       *
224       * since we are calling fileChannel.force, bytes which are written to
225       * filechannel (system filecache) will be persisted to the disk. So we
226       * dont need to consider those bytes for setting value to
227       * unpersistedBytes.
228       *
229       * In this method fileChannel.force is not called in synchronized block, so
230       * we are doing best efforts to not overcount or undercount unpersistedBytes.
231       * Hence setting writeBuffer.readableBytes() to unpersistedBytes.
232       */
233      /*
234      if (unpersistedBytesBound > 0) {
235          synchronized (this) {
236              unpersistedBytes.set(writeBuffer.readableBytes());
237          }
238      }
239      fileChannel.force(forceMetadata);
240      return positionForceWrite;
241  }

```

Immagine 14 : Pit test BufferedChannel Prima Iterazione read

```

244  @Override
245  public synchronized int read(ByteBuf dest, long pos, int length) throws IOException {
246      if (dest.writableBytes() < length) {
247          throw new IllegalArgumentException("dest buffer remaining capacity is not enough"
248              + "(must be at least as \"length\"=" + length + ")");
249      }
250
251      long prevPos = pos;
252      while (length > 0) {
253          // check if it is in the write buffer
254          if (writeBuffer != null && writeBufferStartPosition.get() <= pos) {
255              int positionInBuffer = (int) (pos - writeBufferStartPosition.get());
256              int bytesToCopy = Math.min(writeBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
257
258              if (bytesToCopy == 0) {
259                  throw new IOException("Read past EOF");
260              }
261
262              dest.writeBytes(writeBuffer, positionInBuffer, bytesToCopy);
263              pos += bytesToCopy;
264              length -= bytesToCopy;
265          } else if (writeBuffer == null && writeBufferStartPosition.get() <= pos) {
266              // here we reach the end
267              break;
268              // first check if there is anything we can grab from the readBuffer
269          } else if (readBufferStartPosition <= pos && pos < readBufferStartPosition + readBuffer.writerIndex()) {
270              int positionInBuffer = (int) (pos - readBufferStartPosition);
271              int bytesToCopy = Math.min(readBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
272              dest.writeBytes(readBuffer, positionInBuffer, bytesToCopy);
273              pos += bytesToCopy;
274              length -= bytesToCopy;
275              // let's read it
276          } else {
277              readBufferStartPosition = pos;
278
279              int readBytes = fileChannel.read(readBuffer.internalNioBuffer(0, readCapacity),
280                  readBufferStartPosition);
281              if (readBytes <= 0) {
282                  throw new IOException("Reading from filechannel returned a non-positive value. Short read.");
283              }
284              readBuffer.writerIndex(readBytes);
285          }
286      }
287      return (int) (pos - prevPos);
288  }

```

BufferedChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● read(ByteBuf, long, int)		16%		15%	9	11	23	28	0	1
● write(ByteBuf)		74%		50%	4	6	5	21	0	1
● close()		93%		50%	1	2	1	6	0	1
● BufferedChannel(ByteBufAllocator, FileChannel, int, int, long)		100%		100%	0	2	0	11	0	1
● forceWrite(boolean)		100%		100%	0	2	0	7	0	1
● flush()		100%		50%	1	2	0	6	0	1
● BufferedChannel(ByteBufAllocator, FileChannel, int, long)		100%		n/a	0	1	0	2	0	1
● BufferedChannel(ByteBufAllocator, FileChannel, int)		100%		n/a	0	1	0	2	0	1
● flushAndForceWrite(boolean)		100%		n/a	0	1	0	3	0	1
● flushAndForceWriteIfRegularFlush(boolean)		100%		100%	0	2	0	3	0	1
● clear()		100%		n/a	0	1	0	3	0	1
● getFileChannelPosition()		100%		n/a	0	1	0	1	0	1
● getNumOfBytesInWriteBuffer()		100%		n/a	0	1	0	1	0	1
● getUnpersistedBytes()		100%		n/a	0	1	0	1	0	1
● position()		100%		n/a	0	1	0	1	0	1
Total	156 of 413	62%	24 of 40	40%	15	35	29	96	0	15

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	70% 67/96	30% 18/61	50% 18/36

BufferedChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● read(ByteBuf, long, int)		100%		100%	0	11	0	28	0	1
● write(ByteBuf)		100%		100%	0	6	0	21	0	1
● BufferedChannel(ByteBufAllocator, FileChannel, int, int, long)		100%		100%	0	2	0	11	0	1
● forceWrite(boolean)		100%		100%	0	2	0	7	0	1
● flush()		100%		100%	0	2	0	6	0	1
● close()		100%		100%	0	2	0	6	0	1
● BufferedChannel(ByteBufAllocator, FileChannel, int, long)		100%		n/a	0	1	0	2	0	1
● BufferedChannel(ByteBufAllocator, FileChannel, int)		100%		n/a	0	1	0	2	0	1
● flushAndForceWrite(boolean)		100%		n/a	0	1	0	3	0	1
● flushAndForceWriteIfRegularFlush(boolean)		100%		100%	0	2	0	3	0	1
● clear()		100%		n/a	0	1	0	3	0	1
● getFileChannelPosition()		100%		n/a	0	1	0	1	0	1
● getNumOfBytesInWriteBuffer()		100%		n/a	0	1	0	1	0	1
● getUnpersistedBytes()		100%		n/a	0	1	0	1	0	1
● position()		100%		n/a	0	1	0	1	0	1
Total	0 of 413	100%	0 of 40	100%	0	35	0	96	0	15

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	99% 95/96	92% 56/61	92% 56/61

Immagine 19 : Pit Test BufferedChannel Test suite LLM Read

```

244     @Override
245     public synchronized int read(ByteBuf dest, long pos, int length) throws IOException {
246         if (dest.writableBytes() < length) {
247             throw new IllegalArgumentException("dest buffer remaining capacity is not enough"
248                 + "(must be at least as \"length\"=" + length + ")");
249         }
250
251         long prevPos = pos;
252         while (length > 0) {
253             // check if it is in the write buffer
254             if (writeBuffer != null && writeBufferStartPosition.get() <= pos) {
255                 int positionInBuffer = (int) (pos - writeBufferStartPosition.get());
256                 int bytesToCopy = Math.min(writeBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
257
258                 if (bytesToCopy == 0) {
259                     throw new IOException("Read past EOF");
260                 }
261
262                 dest.writeBytes(writeBuffer, positionInBuffer, bytesToCopy);
263                 pos += bytesToCopy;
264                 length -= bytesToCopy;
265             } else if (writeBuffer == null && writeBufferStartPosition.get() <= pos) {
266                 // here we reach the end
267                 break;
268                 // first check if there is anything we can grab from the readBuffer
269             } else if (readBufferStartPosition <= pos && pos < readBufferStartPosition + readBuffer.writerIndex()) {
270                 int positionInBuffer = (int) (pos - readBufferStartPosition);
271                 int bytesToCopy = Math.min(readBuffer.writerIndex() - positionInBuffer, dest.writableBytes());
272                 dest.writeBytes(readBuffer, positionInBuffer, bytesToCopy);
273                 pos += bytesToCopy;
274                 length -= bytesToCopy;
275                 // let's read it
276             } else {
277                 readBufferStartPosition = pos;
278
279                 int readBytes = fileChannel.read(readBuffer.internalNioBuffer(0, readCapacity),
280                     readBufferStartPosition);
281                 if (readBytes <= 0) {
282                     throw new IOException("Reading from filechannel returned a non-positive value. Short read.");
283                 }
284                 readBuffer.writerIndex(readBytes);
285             }
286         }
287         return (int) (pos - prevPos);
288     }
289 }

```

Immagine 20 : Jacoco BufferedChannel Test all

BufferedChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
read(ByteBuf, long, int)		100%		100%	0 11	0 28	0 1
write(ByteBuf)		100%		100%	0 6	0 21	0 1
BufferedChannel(ByteBufAllocator, FileChannel, int, int, long)		100%		100%	0 2	0 11	0 1
forceWrite(boolean)		100%		100%	0 2	0 7	0 1
flush()		100%		100%	0 2	0 6	0 1
close()		100%		100%	0 2	0 6	0 1
BufferedChannel(ByteBufAllocator, FileChannel, int, long)		100%		n/a	0 1	0 2	0 1
BufferedChannel(ByteBufAllocator, FileChannel, int)		100%		n/a	0 1	0 2	0 1
flushAndForceWrite(boolean)		100%		n/a	0 1	0 3	0 1
flushAndForceWriteIfRegularFlush(boolean)		100%		100%	0 2	0 3	0 1
clear()		100%		n/a	0 1	0 3	0 1
getFileChannelPosition()		100%		n/a	0 1	0 1	0 1
getNumOfBytesInWriteBuffer()		100%		n/a	0 1	0 1	0 1
getUnpersistedBytes()		100%		n/a	0 1	0 1	0 1
position()		100%		n/a	0 1	0 1	0 1
Total	0 of 413	100%	0 of 40	100%	0 35	0 96	0 15

Immagine 21 : Pit Test BufferedChannel Test All

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	100% 96/96	98% 60/61	98% 60/61

Immagine 22 : Pit test BufferedChannel Test All clear()

```

290     @Override
291     public synchronized void clear() {
292         super.clear();
293         writeBuffer.clear();
294     }
295 }

```

4.2.2 JournalChannel

Immagine 23 : Jacoco JournalChannel Test Manuali

JournalChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
JournalChannel(File, long, long, int, int, long, boolean, int, Journal, BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)		82%		87%	4	17	10	65	0	1
preAllocNeeded(long)		0%		0%	2	2	5	5	1	1
renameJournalFile(File, File)		30%		50%	3	4	2	4	0	1
JournalChannel(File, long)		0%		n/a	1	1	2	2	1	1
JournalChannel(File, long, long, int, int, boolean, int, ServerConfiguration, FileChannelProvider)		0%		n/a	1	1	2	2	1	1
forceWrite(boolean)		66%		66%	2	4	2	9	0	1
JournalChannel(File, long, long, int, ServerConfiguration, FileChannelProvider)		0%		n/a	1	1	2	2	1	1
getBufferedChannel()		72%		50%	1	2	1	3	0	1
writeHeader(Journal, BufferedChannelBuilder, int)		100%		100%	0	2	0	13	0	1
JournalChannel(File, long, long, int, long, ServerConfiguration, FileChannelProvider)		100%		n/a	0	1	0	2	0	1
JournalChannel(File, long, long, int, int, boolean, int, Journal, BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)		100%		n/a	0	1	0	2	0	1
close()		100%		75%	1	3	0	5	0	1
read(ByteBuffer)		100%		n/a	0	1	0	1	0	1
static(...)		100%		n/a	0	1	0	1	0	1
getFormatVersion()		100%		n/a	0	1	0	1	0	1
Total	161 of 568	71%	13 of 54	75%	16	42	26	117	4	15

Immagine 24 : Jacoco JournalChannel Test Manuali Costruttore Parte 1

```
150. private JournalChannel(File journalDirectory, long logId,
151.                        long preAllocSize, int writeBufferSize, int journalAlignSize,
152.                        long position, boolean fRemoveFromPageCache,
153.                        int formatVersionToWrite, Journal.BufferedChannelBuilder bcBuilder,
154.                        ServerConfiguration conf,
155.                        FileChannelProvider provider, Long toReplaceLogId) throws IOException {
156.     this.journalAlignSize = journalAlignSize;
157.     this.zeros = ByteBuffer.allocate(journalAlignSize);
158.     this.preAllocSize = preAllocSize - preAllocSize % journalAlignSize;
159.     this.fRemoveFromPageCache = fRemoveFromPageCache;
160.     this.configuration = conf;
161.
162.     boolean reuseFile = false;
163.     File fn = new File(journalDirectory, Long.toHexString(logId) + ".txn");
164.     if (toReplaceLogId != null && logId != toReplaceLogId && provider.supportReuseFile()) {
165.         File toReplaceFile = new File(journalDirectory, Long.toHexString(toReplaceLogId) + ".txn");
166.         if (toReplaceFile.exists()) {
167.             renameJournalFile(toReplaceFile, fn);
168.             provider.notifyRename(toReplaceFile, fn);
169.             reuseFile = true;
170.         }
171.     }
172.     channel = provider.open(fn, configuration);
173.
174.     if (formatVersionToWrite < V4) {
175.         throw new IOException("Invalid journal format to write : version = " + formatVersionToWrite);
176.     }
177.
178.     LOG.info("Opening journal {}", fn);
179.     if (!channel.fileExists(fn)) { // create new journal file to write, write version
180.         if (!fn.createNewFile()) {
181.             LOG.error("Journal file {}, that shouldn't exist, already exists.",
182.                 + " is there another bookie process running?", fn);
183.             throw new IOException("File " + fn
184.                 + " suddenly appeared, is another bookie process running?");
185.         }
186.         fc = channel.getFileChannel();
187.         formatVersion = formatVersionToWrite;
188.         writeHeader(bcBuilder, writeBufferSize);
189.     } else if (reuseFile) { // Open an existing journal to write, it needs fileChannelProvider support reuse file.
190.         fc = channel.getFileChannel();
191.         formatVersion = formatVersionToWrite;
192.         writeHeader(bcBuilder, writeBufferSize);
193.     } else { // open an existing file to read.
194.         fc = channel.getFileChannel();
195.         // readonly, use fileChannel directly, no need to use BufferedChannel
196.
197.         ByteBuffer bb = ByteBuffer.allocate(VERSION_HEADER_SIZE);
198.         int c = fc.read(bb);
199.         bb.flip();
200.
201.         if (c == VERSION_HEADER_SIZE) {
202.             byte[] first4 = new byte[4];
203.             bb.get(first4);
204.
205.             if (Arrays.equals(first4, magicWord)) {
206.                 formatVersion = bb.getInt();
207.             } else {
208.                 // pre magic word journal, reset to 0;
209.                 formatVersion = V1;
210.             }
211.         } else {
212.             // no header, must be old version
213.             formatVersion = V1;
214.         }
215.     }
```


Immagine 25 : Jacoco JournalChannel Test Manuali Costruttore Parte 2

```

215.
216.     if (formatVersion < MIN_COMPAT_JOURNAL_FORMAT_VERSION
217.         || formatVersion > CURRENT_JOURNAL_FORMAT_VERSION) {
218.         String err = String.format("Invalid journal version, unable to read."
219.             + " Expected between (%d) and (%d), got (%d)",
220.             MIN_COMPAT_JOURNAL_FORMAT_VERSION, CURRENT_JOURNAL_FORMAT_VERSION,
221.             formatVersion);
222.         LOG.error(err);
223.         throw new IOException(err);
224.     }
225.
226.     try {
227.         if (position == START_OF_FILE) {
228.             if (formatVersion >= V5) {
229.                 fc.position(HEADER_SIZE);
230.             } else if (formatVersion >= V2) {
231.                 fc.position(VERSION_HEADER_SIZE);
232.             } else {
233.                 fc.position(0);
234.             }
235.         } else {
236.             fc.position(position);
237.         }
238.     } catch (IOException e) {
239.         LOG.error("Bookie journal file can seek to position : ", e);
240.         throw e;
241.     }
242.
243.     if (fRemoveFromPageCache) {
244.         this.fd = PageCacheUtil.getSysFileDescriptor(channel.getFD());
245.     } else {
246.         this.fd = -1;
247.     }
248. }
249.

```

Immagine 26 : Jacoco JournalChannel Test Manuali writeHeader

```

250.     private void writeHeader(Journal.BufferedChannelBuilder bcBuilder,
251.                             int writeBufferSize) throws IOException {
252.         int headerSize = (V4 == formatVersion) ? VERSION_HEADER_SIZE : HEADER_SIZE;
253.         ByteBuffer bb = ByteBuffer.allocate(headerSize);
254.         ZeroBuffer.put(bb);
255.         bb.clear();
256.         bb.put(magicWord);
257.         bb.putInt(formatVersion);
258.         bb.clear();
259.         fc.write(bb);
260.
261.         bc = bcBuilder.create(fc, writeBufferSize);
262.         forceWrite(true);
263.         nextPrealloc = this.preAllocSize;
264.         fc.write(zeros, nextPrealloc - journalAlignSize);
265.     }

```

Immagine 27 : Jacoco JournalChannel Test Manuali Dopo Prima Iterazione

JournalChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
preAllocNeeded(long)		0%		0%	2	2	5	5	1	1
renameJournalFile(File, File)		30%		50%	3	4	2	4	0	1
JournalChannel(File, long)		0%		n/a	1	1	2	2	1	1
JournalChannel(File, long, long, int, int, boolean, int, ServerConfiguration, FileChannelProvider)		0%		n/a	1	1	2	2	1	1
forceWrite(boolean)		66%		66%	2	4	2	9	0	1
JournalChannel(File, long, long, int, ServerConfiguration, FileChannelProvider)		0%		n/a	1	1	2	2	1	1
getBufferedChannel()		72%		50%	1	2	1	3	0	1
JournalChannel(File, long, long, int, int, long, boolean, int, Journal.BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)		100%		100%	0	17	0	65	0	1
writeHeader(Journal.BufferedChannelBuilder, int)		100%		100%	0	2	0	13	0	1
JournalChannel(File, long, long, int, long, ServerConfiguration, FileChannelProvider)		100%		n/a	0	1	0	2	0	1
JournalChannel(File, long, long, int, int, boolean, int, Journal.BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)		100%		n/a	0	1	0	2	0	1
close()		100%		75%	1	3	0	5	0	1
read(ByteBuffer)		100%		n/a	0	1	0	1	0	1
static {...}		100%		n/a	0	1	0	1	0	1
getFormatVersion()		100%		n/a	0	1	0	1	0	1
Total	107 of 568	81%	9 of 54	83%	12	42	16	117	4	15

Immagine 28 : Pit Test JournalChannel Test Manuali

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	86% 83/96	82% 50/61	89% 50/56
JournalChannel.java	86% 101/117	66% 35/53	78% 35/45

Immagine 29 : Pit Test JournalChannel Test Manuali Costruttore Parte 1

```

150     private JournalChannel(File journalDirectory, long logId,
151                           long preAllocSize, int writeBufferSize, int journalAlignSize,
152                           long position, boolean fRemoveFromPageCache,
153                           int formatVersionToWrite, Journal.BufferedChannelBuilder bcBuilder,
154                           ServerConfiguration conf,
155                           FileChannelProvider provider, Long toReplaceLogId) throws IOException {
156         this.journalAlignSize = journalAlignSize;
157         this.zeros = ByteBuffer.allocate(journalAlignSize);
158         this.preAllocSize = preAllocSize - preAllocSize % journalAlignSize;
159         this.fRemoveFromPageCache = fRemoveFromPageCache;
160         this.configuration = conf;
161
162         boolean reuseFile = false;
163         File fn = new File(journalDirectory, Long.toHexString(logId) + ".txn");
164         if (toReplaceLogId != null && logId != toReplaceLogId && provider.supportReuseFile()) {
165             File toReplaceFile = new File(journalDirectory, Long.toHexString(toReplaceLogId) + ".txn");
166             if (toReplaceFile.exists()) {
167                 renameJournalFile(toReplaceFile, fn);
168                 provider.notifyRename(toReplaceFile, fn);
169                 reuseFile = true;
170             }
171         }
172         channel = provider.open(fn, configuration);
173
174         if (formatVersionToWrite < V4) {
175             throw new IOException("Invalid journal format to write : version = " + formatVersionToWrite);
176         }
177
178         LOG.info("Opening journal {}", fn);
179         if (!channel.fileExists(fn)) { // create new journal file to write, write version
180             if (!fn.createNewFile()) {
181                 LOG.error("Journal file {}, that shouldn't exist, already exists. "
182                     + " is there another bookie process running?", fn);
183                 throw new IOException("File " + fn
184                     + " suddenly appeared, is another bookie process running?");
185             }
186             fc = channel.getFileChannel();
187             formatVersion = formatVersionToWrite;
188             writeHeader(bcBuilder, writeBufferSize);
189         } else if (reuseFile) { // Open an existing journal to write, it needs fileChannelProvider support reuse file.
190             fc = channel.getFileChannel();
191             formatVersion = formatVersionToWrite;
192             writeHeader(bcBuilder, writeBufferSize);
193         } else { // open an existing file to read.
194             fc = channel.getFileChannel();
195             // readonly, use fileChannel directly, no need to use BufferedChannel
196
197             ByteBuffer bb = ByteBuffer.allocate(VERSION_HEADER_SIZE);
198             int c = fc.read(bb);
199             bb.flip();
200

```

Immagine 30 : Pit Test JournalChannel Test Manuali Costruttore Parte 2

```

200
201         if (c == VERSION_HEADER_SIZE) {
202             byte[] first4 = new byte[4];
203             bb.get(first4);
204
205             if (Arrays.equals(first4, magicWord)) {
206                 formatVersion = bb.getInt();
207             } else {
208                 // pre magic word journal, reset to 0;
209                 formatVersion = V1;
210             }
211         } else {
212             // no header, must be old version
213             formatVersion = V1;
214         }
215
216         if (formatVersion < MIN_COMPAT_JOURNAL_FORMAT_VERSION
217             || formatVersion > CURRENT_JOURNAL_FORMAT_VERSION) {
218             String err = String.format("Invalid journal version, unable to read."
219                 + " Expected between (%d) and (%d), got (%d)",
220                 MIN_COMPAT_JOURNAL_FORMAT_VERSION, CURRENT_JOURNAL_FORMAT_VERSION,
221                 formatVersion);
222             LOG.error(err);
223             throw new IOException(err);
224         }
225
226         try {
227             if (position == START_OF_FILE) {
228                 if (formatVersion >= V5) {
229                     fc.position(HEADER_SIZE);
230                 } else if (formatVersion >= V2) {
231                     fc.position(VERSION_HEADER_SIZE);
232                 } else {
233                     fc.position(0);
234                 }
235             } else {
236                 fc.position(position);
237             }
238         } catch (IOException e) {
239             LOG.error("Bookie journal file can seek to position :", e);
240             throw e;
241         }
242     }
243     if (fRemoveFromPageCache) {
244         this.fd = PageCacheUtil.getSysFileDescriptor(channel.getFD());
245     } else {
246         this.fd = -1;
247     }
248 }

```

Immagine 31 : Pit Test JournalChannel Test Manuali writeHeader

```

250 private void writeHeader(Journal.BufferedChannelBuilder bcBuilder,
251                          int writeBufferSize) throws IOException {
252 1   int headerSize = (V4 == formatVersion) ? VERSION_HEADER_SIZE : HEADER_SIZE;
253   ByteBuffer bb = ByteBuffer.allocate(headerSize);
254 1   ZeroBuffer.put(bb);
255   bb.clear();
256   bb.put(magicWord);
257   bb.putInt(formatVersion);
258   bb.clear();
259   fc.write(bb);
260
261   bc = bcBuilder.create(fc, writeBufferSize);
262 1   forceWrite(true);
263   nextPrealloc = this.preAllocSize;
264 1   fc.write(zeros, nextPrealloc - journalAlignSize);
265 }

```

Immagine 32 : Pit Test JournalChannel Seconda Iterazione

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	86% 83/96	82% 50/61	89% 50/56
JournalChannel.java	86% 101/117	70% 37/53	82% 37/45

Immagine 33 : Pit Test JournalChannel Seconda Iterazione Costruttore

```

149  */
150 private JournalChannel(File journalDirectory, long logId,
151                       long preAllocSize, int writeBufferSize, int journalAlignSize,
152                       long position, boolean fRemoveFromPageCache,
153                       int formatVersionToWrite, Journal.BufferedChannelBuilder bcBuilder,
154                       ServerConfiguration conf,
155                       FileChannelProvider provider, Long toReplaceLogId) throws IOException {
156   this.journalAlignSize = journalAlignSize;
157   this.zeros = ByteBuffer.allocate(journalAlignSize);
158 2   this.preAllocSize = preAllocSize - preAllocSize % journalAlignSize;
159   this.fRemoveFromPageCache = fRemoveFromPageCache;
160   this.configuration = conf;
161
162   boolean reuseFile = false;
163   File fn = new File(journalDirectory, Long.toHexString(logId) + ".txn");
164 3   if (toReplaceLogId != null && logId != toReplaceLogId && provider.supportReuseFile()) {
165     File toReplaceFile = new File(journalDirectory, Long.toHexString(toReplaceLogId) + ".txn");
166 1   if (toReplaceFile.exists()) {
167     renameJournalFile(toReplaceFile, fn);
168 1   provider.notifyRename(toReplaceFile, fn);
169     reuseFile = true;
170   }
171   }
172   channel = provider.open(fn, configuration);
173
174 2   if (formatVersionToWrite < V4) {
175     throw new IOException("Invalid journal format to write : version = " + formatVersionToWrite);
176   }
177
178   LOG.info("Opening journal {}", fn);
179 1   if (!channel.fileExists(fn)) { // create new journal file to write, write version
180 1   if (!fn.createNewFile()) {
181     LOG.error("Journal file {}, that shouldn't exist, already exists. "
182             + " is there another bookie process running?", fn);
183     throw new IOException("File " + fn
184             + " suddenly appeared, is another bookie process running?");
185   }
186   fc = channel.getFileChannel();
187   formatVersion = formatVersionToWrite;
188 1   writeHeader(bcBuilder, writeBufferSize);
189 1   } else if (reuseFile) { // Open an existing journal to write, it needs fileChannelProvider support reuse file.
190     fc = channel.getFileChannel();
191     formatVersion = formatVersionToWrite;
192 1   writeHeader(bcBuilder, writeBufferSize);
193   } else { // open an existing file to read.
194     fc = channel.getFileChannel();
195     // readonly, use fileChannel directly, no need to use BufferedChannel
196
197     ByteBuffer bb = ByteBuffer.allocate(VERSION_HEADER_SIZE);
198     int c = fc.read(bb);
199     bb.flip();
200

```

Immagine 34 : Pit Test JournalChannel Seconda Iterazione writeHeader

```

250     private void writeHeader(Journal.BufferedChannelBuilder bcBuilder,
251                             int writeBufferSize) throws IOException {
252 1       int headerSize = (V4 == formatVersion) ? VERSION_HEADER_SIZE : HEADER_SIZE;
253         ByteBuffer bb = ByteBuffer.allocate(headerSize);
254 1       ZeroBuffer.put(bb);
255         bb.clear();
256         bb.put(magicWord);
257         bb.putInt(formatVersion);
258         bb.clear();
259         fc.write(bb);
260
261         bc = bcBuilder.create(fc, writeBufferSize);
262 1       forceWrite(true);
263         nextPrealloc = this.preAllocSize;
264 1       fc.write(zeros, nextPrealloc - journalAlignSize);
265     }

```

Immagine 35 : Jacoco JournalChannel Randoop

JournalChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
JournalChannel(File, long, long, int, int, long, boolean, int, Journal.BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)	52%	46%	14	17	26	65	0	1		
forceWrite(boolean)	33%	33%	3	4	5	9	0	1		
preAllocNeeded(long)	30%	50%	1	2	3	5	0	1		
JournalChannel(File, long, long, int, int, boolean, int, Journal.BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)	0%	n/a	1	1	2	2	1	1		
JournalChannel(File, long, long, int, int, boolean, int, ServerConfiguration, FileChannelProvider)	0%	n/a	1	1	2	2	1	1		
JournalChannel(File, long, long, int, ServerConfiguration, FileChannelProvider)	0%	n/a	1	1	2	2	1	1		
renameJournalFile(File, File)	76%	16%	3	4	1	4	0	1		
writeHeader(Journal.BufferedChannelBuilder, int)	96%	50%	1	2	0	13	0	1		
JournalChannel(File, long, long, int, long, ServerConfiguration, FileChannelProvider)	100%	n/a	0	1	0	2	0	1		
JournalChannel(File, long)	100%	n/a	0	1	0	2	0	1		
close()	100%	75%	1	3	0	5	0	1		
getBufferedChannel()	100%	100%	0	2	0	3	0	1		
read(ByteBuffer)	100%	n/a	0	1	0	1	0	1		
static(...)	100%	n/a	0	1	0	1	0	1		
getFormatVersion()	100%	n/a	0	1	0	1	0	1		
Total	238 of 568	58%	29 of 54	46%	26	42	41	117	3	15

Immagine 36 : Jacoco JournalChannel Randoop Costruttore Parte 1

```

149  ~/
150  private JournalChannel(File journalDirectory, long logId,
151                        long preAllocSize, int writeBufferSize, int journalAlignSize,
152                        long position, boolean fRemoveFromPageCache,
153                        int formatVersionToWrite, Journal.BufferedChannelBuilder bcBuilder,
154                        ServerConfiguration conf,
155                        FileChannelProvider provider, Long toReplaceLogId) throws IOException {
156      this.journalAlignSize = journalAlignSize;
157      this.zeros = ByteBuffer.allocate(journalAlignSize);
158      this.preAllocSize = preAllocSize - preAllocSize % journalAlignSize;
159      this.fRemoveFromPageCache = fRemoveFromPageCache;
160      this.configuration = conf;
161
162      boolean reuseFile = false;
163      File fn = new File(journalDirectory, Long.toHexString(logId) + ".txn");
164  ♦ if (toReplaceLogId != null && logId != toReplaceLogId && provider.supportReuseFile()) {
165          File toReplaceFile = new File(journalDirectory, Long.toHexString(toReplaceLogId) + ".txn");
166  ♦ if (toReplaceFile.exists()) {
167              renameJournalFile(toReplaceFile, fn);
168              provider.notifyRename(toReplaceFile, fn);
169              reuseFile = true;
170          }
171      }
172      channel = provider.open(fn, configuration);
173
174  ♦ if (formatVersionToWrite < V4) {
175          throw new IOException("Invalid journal format to write : version = " + formatVersionToWrite);
176      }
177
178      LOG.info("Opening journal {}", fn);
179  ♦ if (!channel.fileExists(fn)) { // create new journal file to write, write version
180          if (!fn.createNewFile()) {
181              LOG.error("Journal file {}, that shouldn't exist, already exists. "
182                    + " is there another bookie process running?", fn);
183              throw new IOException("File " + fn
184                    + " suddenly appeared, is another bookie process running?");
185          }
186          fc = channel.getFileChannel();
187          formatVersion = formatVersionToWrite;
188          writeHeader(bcBuilder, writeBufferSize);
189  ♦ } else if (reuseFile) { // Open an existing journal to write, it needs fileChannelProvider support reuse file.
190          fc = channel.getFileChannel();
191          formatVersion = formatVersionToWrite;
192          writeHeader(bcBuilder, writeBufferSize);
193      } else { // open an existing file to read.
194          fc = channel.getFileChannel();
195          // readonly, use fileChannel directly, no need to use BufferedChannel
196
197          ByteBuffer bb = ByteBuffer.allocate(VERSION_HEADER_SIZE);
198          int c = fc.read(bb);
199          bb.flip();

```

Immagine 37 : : Jacoco JournalChannel Randoop Costruttore Parte 2

```

200.
201.     if (c == VERSION_HEADER_SIZE) {
202.         byte[] first4 = new byte[4];
203.         bb.get(first4);
204.
205.         if (Arrays.equals(first4, magicWord)) {
206.             formatVersion = bb.getInt();
207.         } else {
208.             // pre magic word journal, reset to 0;
209.             formatVersion = V1;
210.         }
211.     } else {
212.         // no header, must be old version
213.         formatVersion = V1;
214.     }
215.
216.     if (formatVersion < MIN_COMPAT_JOURNAL_FORMAT_VERSION
217.         || formatVersion > CURRENT_JOURNAL_FORMAT_VERSION) {
218.         String err = String.format("Invalid journal version, unable to read."
219.             + " Expected between (%d) and (%d), got (%d)",
220.             MIN_COMPAT_JOURNAL_FORMAT_VERSION, CURRENT_JOURNAL_FORMAT_VERSION,
221.             formatVersion);
222.         LOG.error(err);
223.         throw new IOException(err);
224.     }
225.
226.     try {
227.         if (position == START_OF_FILE) {
228.             if (formatVersion >= V5) {
229.                 fc.position(HEADER_SIZE);
230.             } else if (formatVersion >= V2) {
231.                 fc.position(VERSION_HEADER_SIZE);
232.             } else {
233.                 fc.position(0);
234.             }
235.         } else {
236.             fc.position(position);
237.         }
238.     } catch (IOException e) {
239.         LOG.error("Bookie journal file can seek to position :", e);
240.         throw e;
241.     }
242.
243.     if (fRemoveFromPageCache) {
244.         this.fd = PageCacheUtil.getSysFileDescriptor(channel.getFD());
245.     } else {
246.         this.fd = -1;
247.     }
248. }

```

Immagine 38 : Jacoco JournalChannel Randoop writeHeader

```

250.     private void writeHeader(Journal.BufferedChannelBuilder bcBuilder,
251.                             int writeBufferSize) throws IOException {
252.         int headerSize = (V4 == formatVersion) ? VERSION_HEADER_SIZE : HEADER_SIZE;
253.         ByteBuffer bb = ByteBuffer.allocate(headerSize);
254.         ZeroBuffer.put(bb);
255.         bb.clear();
256.         bb.put(magicWord);
257.         bb.putInt(formatVersion);
258.         bb.clear();
259.         fc.write(bb);
260.
261.         bc = bcBuilder.create(fc, writeBufferSize);
262.         forceWrite(true);
263.         nextPrealloc = this.preAllocSize;
264.         fc.write(zeros, nextPrealloc - journalAlignSize);
265.     }

```

Immagine 39 : Pit Test JournalChannel Randoop

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	26% 25/96	0% 0/61	0% 0/10
JournalChannel.java	68% 79/117	32% 17/53	46% 17/37

Immagine 40: Pit Test JournalChannel Randoop Costruttore Parte 1

```

150     private JournalChannel(File journalDirectory, long logId,
151                           long preAllocSize, int writeBufferSize, int journalAlignSize,
152                           long position, boolean fRemoveFromPageCache,
153                           int formatVersionToWrite, Journal.BufferedChannelBuilder bcBuilder,
154                           ServerConfiguration conf,
155                           FileChannelProvider provider, Long toReplaceLogId) throws IOException {
156         this.journalAlignSize = journalAlignSize;
157         this.zeros = ByteBuffer.allocate(journalAlignSize);
158         this.preAllocSize = preAllocSize - preAllocSize % journalAlignSize;
159         this.fRemoveFromPageCache = fRemoveFromPageCache;
160         this.configuration = conf;
161
162         boolean reuseFile = false;
163         File fn = new File(journalDirectory, Long.toHexString(logId) + ".txn");
164         if (toReplaceLogId != null && logId != toReplaceLogId && provider.supportReuseFile()) {
165             File toReplaceFile = new File(journalDirectory, Long.toHexString(toReplaceLogId) + ".txn");
166             if (toReplaceFile.exists()) {
167                 renameJournalFile(toReplaceFile, fn);
168                 provider.notifyRename(toReplaceFile, fn);
169                 reuseFile = true;
170             }
171         }
172         channel = provider.open(fn, configuration);
173
174         if (formatVersionToWrite < V4) {
175             throw new IOException("Invalid journal format to write : version = " + formatVersionToWrite);
176         }
177
178         LOG.info("Opening journal {}", fn);
179         if (!channel.fileExists(fn)) { // create new journal file to write, write version
180             if (!fn.createNewFile()) {
181                 LOG.error("Journal file {}, that shouldn't exist, already exists. "
182                         + " is there another bookie process running?", fn);
183                 throw new IOException("File " + fn
184                         + " suddenly appeared, is another bookie process running?");
185             }
186             fc = channel.getFileChannel();
187             formatVersion = formatVersionToWrite;
188             writeHeader(bcBuilder, writeBufferSize);
189         } else if (reuseFile) { // Open an existing journal to write, it needs fileChannelProvider support reuse file.
190             fc = channel.getFileChannel();
191             formatVersion = formatVersionToWrite;
192             writeHeader(bcBuilder, writeBufferSize);
193         } else { // open an existing file to read.
194             fc = channel.getFileChannel();
195             // readonly, use fileChannel directly, no need to use BufferedChannel
196
197             ByteBuffer bb = ByteBuffer.allocate(VERSION_HEADER_SIZE);
198             int c = fc.read(bb);
199             bb.flip();
200
201             if (c == VERSION_HEADER_SIZE) {
202                 byte[] first4 = new byte[4];
203                 bb.get(first4);
204

```

Immagine 41: Pit Test JournalChannel Randoop Costruttore Parte 2

```

201         if (c == VERSION_HEADER_SIZE) {
202             byte[] first4 = new byte[4];
203             bb.get(first4);
204
205             if (Arrays.equals(first4, magicWord)) {
206                 formatVersion = bb.getInt();
207             } else {
208                 // pre magic word journal, reset to 0;
209                 formatVersion = V1;
210             }
211         } else {
212             // no header, must be old version
213             formatVersion = V1;
214         }
215
216         if (formatVersion < MIN_COMPAT_JOURNAL_FORMAT_VERSION
217             || formatVersion > CURRENT_JOURNAL_FORMAT_VERSION) {
218             String err = String.format("Invalid journal version, unable to read."
219                                     + " Expected between (%d) and (%d), got (%d)",
220                                     MIN_COMPAT_JOURNAL_FORMAT_VERSION, CURRENT_JOURNAL_FORMAT_VERSION,
221                                     formatVersion);
222             LOG.error(err);
223             throw new IOException(err);
224         }
225
226         try {
227             if (position == START_OF_FILE) {
228                 if (formatVersion >= V5) {
229                     fc.position(HEADER_SIZE);
230                 } else if (formatVersion >= V2) {
231                     fc.position(VERSION_HEADER_SIZE);
232                 } else {
233                     fc.position(0);
234                 }
235             } else {
236                 fc.position(position);
237             }
238         } catch (IOException e) {
239             LOG.error("Bookie journal file can seek to position :, e);
240             throw e;
241         }
242
243         if (fRemoveFromPageCache) {
244             this.fd = PageCacheUtil.getSysFileDescriptor(channel.getFD());
245         } else {
246             this.fd = -1;
247         }
248     }
249

```


Immagine 42 : Jacoco JournalChannel Evosuite

JournalChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
JournalChannel(File, long, long, int, int, long, boolean, int, Journal.BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)		67%		65%	9	17	21	65	0	1
forceWrite(boolean)		66%		66%	2	4	2	9	0	1
writeHeader(Journal.BufferedChannelBuilder, int)		100%		100%	0	2	0	13	0	1
preAllocNeeded(long)		100%		100%	0	2	0	5	0	1
renameJournalFile(File, File)		100%		100%	0	4	0	4	0	1
JournalChannel(File, long, long, int, long, ServerConfiguration, FileChannelProvider)		100%		n/a	0	1	0	2	0	1
JournalChannel(File, long, long, int, int, boolean, int, Journal.BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)		100%		n/a	0	1	0	2	0	1
JournalChannel(File, long)		100%		n/a	0	1	0	2	0	1
JournalChannel(File, long, long, int, int, boolean, int, ServerConfiguration, FileChannelProvider)		100%		n/a	0	1	0	2	0	1
close()		100%		75%	1	3	0	5	0	1
getBufferedChannel()		100%		100%	0	2	0	3	0	1
JournalChannel(File, long, long, int, ServerConfiguration, FileChannelProvider)		100%		n/a	0	1	0	2	0	1
read(ByteBuffer)		100%		n/a	0	1	0	1	0	1
static{...}		100%		n/a	0	1	0	1	0	1
getFormatVersion()		100%		n/a	0	1	0	1	0	1
Total	110 of 568	80%	14 of 54	74%	12	42	23	117	0	15

Immagine 43: Pit test JournalChannel Evosuite

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	30% 29/96	8% 5/61	45% 5/11
JournalChannel.java	83% 97/117	85% 45/53	100% 45/45

Immagine 44: Pit test JournalChannel Evosuite Costruttore

```

150     private JournalChannel(File journalDirectory, long logId,
151                             long preAllocSize, int writeBufferSize, int journalAlignSize,
152                             long position, boolean fRemoveFromPageCache,
153                             int formatVersionToWrite, Journal.BufferedChannelBuilder bcBuilder,
154                             ServerConfiguration conf,
155                             FileChannelProvider provider, Long toReplaceLogId) throws IOException {
156         this.journalAlignSize = journalAlignSize;
157         this.zeros = ByteBuffer.allocate(journalAlignSize);
158 2    this.preAllocSize = preAllocSize - preAllocSize % journalAlignSize;
159         this.fRemoveFromPageCache = fRemoveFromPageCache;
160         this.configuration = conf;
161
162         boolean reuseFile = false;
163         File fn = new File(journalDirectory, Long.toHexString(logId) + ".txn");
164 3    if (toReplaceLogId != null && logId != toReplaceLogId && provider.supportReuseFile()) {
165             File toReplaceFile = new File(journalDirectory, Long.toHexString(toReplaceLogId) + ".txn");
166 1    if (toReplaceFile.exists()) {
167 1    renameJournalFile(toReplaceFile, fn);
168 1    provider.notifyRename(toReplaceFile, fn);
169             reuseFile = true;
170         }
171     }
172     channel = provider.open(fn, configuration);
173
174 2    if (formatVersionToWrite < V4) {
175         throw new IOException("Invalid journal format to write : version = " + formatVersionToWrite);
176     }
177
178     LOG.info("Opening journal {}", fn);
179 1    if (!channel.fileExists(fn)) { // create new journal file to write, write version
180 1    if (!fn.createNewFile()) {
181         LOG.error("Journal file {}, that shouldn't exist, already exists. "
182                 + " is there another bookie process running?", fn);
183         throw new IOException("File " + fn
184                 + " suddenly appeared, is another bookie process running?");
185     }
186     fc = channel.getFileChannel();
187     formatVersion = formatVersionToWrite;
188 1    writeHeader(bcBuilder, writeBufferSize);
189 1    } else if (reuseFile) { // Open an existing journal to write, it needs fileChannelProvider support reuse file.
190         fc = channel.getFileChannel();
191         formatVersion = formatVersionToWrite;
192 1    writeHeader(bcBuilder, writeBufferSize);
193     } else { // open an existing file to read.
194         fc = channel.getFileChannel();
195         // readonly, use fileChannel directly, no need to use BufferedChannel
196
197         ByteBuffer bb = ByteBuffer.allocate(VERSION_HEADER_SIZE);
198         int c = fc.read(bb);
199         bb.flip();
200
201 1    if (c == VERSION_HEADER_SIZE) {
202         byte[] first4 = new byte[4];
203         bb.get(first4);
204
205 1    if (Arrays.equals(first4, magicWord)) {
206         formatVersion = bb.getInt();
207     } else {
208         // pre magic word journal, reset to 0;
209         formatVersion = V1;
210     }

```

JournalChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
• forceWrite(boolean)		33%		33%	3	4	5	9	0	1
• JournalChannel(File, long, long, int, int, long, boolean, int, Journal.BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)		95%		87%	4	17	4	65	0	1
• JournalChannel(File, Long)		0%		n/a	1	1	2	2	1	1
• writeHeader(Journal.BufferedChannelBuilder, int)		96%		50%	1	2	0	13	0	1
• preAllocNeeded(long)		100%		50%	1	2	0	5	0	1
• renameJournalFile(File, File)		100%		66%	2	4	0	4	0	1
• JournalChannel(File, long, long, int, long, ServerConfiguration, FileChannelProvider)		100%		n/a	0	1	0	2	0	1
• JournalChannel(File, long, long, int, int, boolean, int, Journal.BufferedChannelBuilder, ServerConfiguration, FileChannelProvider, Long)		100%		n/a	0	1	0	2	0	1
• JournalChannel(File, long, long, int, int, boolean, int, ServerConfiguration, FileChannelProvider)		100%		n/a	0	1	0	2	0	1
• close()		100%		75%	1	3	0	5	0	1
• getBufferedChannel()		100%		100%	0	2	0	3	0	1
• JournalChannel(File, long, long, int, long, ServerConfiguration, FileChannelProvider)		100%		n/a	0	1	0	2	0	1
• read(ByteBuffer)		100%		n/a	0	1	0	1	0	1
• static {...}		100%		n/a	0	1	0	1	0	1
• getFormatVersion()		100%		n/a	0	1	0	1	0	1
Total	54 of 568	90%	13 of 54	75%	13	42	11	117	1	15

Immagine 46: Jacoco JournalChannel LLM Costruttore Parte 1

```

150.     private JournalChannel(File journalDirectory, long logId,
151.                           long preAllocSize, int writeBufferSize, int journalAlignSize,
152.                           long position, boolean fRemoveFromPageCache,
153.                           int formatVersionToWrite, Journal.BufferedChannelBuilder bcBuilder,
154.                           ServerConfiguration conf,
155.                           FileChannelProvider provider, Long toReplaceLogId) throws IOException {
156.         this.journalAlignSize = journalAlignSize;
157.         this.zeros = ByteBuffer.allocate(journalAlignSize);
158.         this.preAllocSize = preAllocSize - preAllocSize % journalAlignSize;
159.         this.fRemoveFromPageCache = fRemoveFromPageCache;
160.         this.configuration = conf;
161.
162.         boolean reuseFile = false;
163.         File fn = new File(journalDirectory, Long.toHexString(logId) + ".txn");
164.         if (toReplaceLogId != null && logId != toReplaceLogId && provider.supportReuseFile()) {
165.             File toReplaceFile = new File(journalDirectory, Long.toHexString(toReplaceLogId) + ".txn");
166.             if (toReplaceFile.exists()) {
167.                 renameJournalFile(toReplaceFile, fn);
168.                 provider.notifyRename(toReplaceFile, fn);
169.                 reuseFile = true;
170.             }
171.         }
172.         channel = provider.open(fn, configuration);
173.
174.         if (formatVersionToWrite < V4) {
175.             throw new IOException("Invalid journal format to write : version = " + formatVersionToWrite);
176.         }
177.
178.         LOG.info("Opening journal {}", fn);
179.         if (!channel.fileExists(fn)) { // create new journal file to write, write version
180.             if (!fn.createNewFile()) {
181.                 LOG.error("Journal file {}, that shouldn't exist, already exists. "
182.                     + " is there another bookie process running?", fn);
183.                 throw new IOException("File " + fn
184.                     + " suddenly appeared, is another bookie process running?");
185.             }
186.             fc = channel.getFileChannel();
187.             formatVersion = formatVersionToWrite;
188.             writeHeader(bcBuilder, writeBufferSize);
189.         } else if (reuseFile) { // Open an existing journal to write, it needs fileChannelProvider support reuse file.
190.             fc = channel.getFileChannel();
191.             formatVersion = formatVersionToWrite;
192.             writeHeader(bcBuilder, writeBufferSize);
193.         } else { // open an existing file to read.
194.             fc = channel.getFileChannel();
195.             // readonly, use fileChannel directly, no need to use BufferedChannel
196.
197.             ByteBuffer bb = ByteBuffer.allocate(VERSION_HEADER_SIZE);
198.             int c = fc.read(bb);
199.             bb.flip();
200.

```


Immagine 47: Jacoco JournalChannel LLM Costruttore Parte 2

```

201.     if (c == VERSION_HEADER_SIZE) {
202.         byte[] first4 = new byte[4];
203.         bb.get(first4);
204.
205.         if (Arrays.equals(first4, magicWord)) {
206.             formatVersion = bb.getInt();
207.         } else {
208.             // pre magic word journal, reset to 0;
209.             formatVersion = V1;
210.         }
211.     } else {
212.         // no header, must be old version
213.         formatVersion = V1;
214.     }
215.
216.     if (formatVersion < MIN_COMPAT_JOURNAL_FORMAT_VERSION
217.         || formatVersion > CURRENT_JOURNAL_FORMAT_VERSION) {
218.         String err = String.format("Invalid journal version, unable to read."
219.             + " Expected between (%d) and (%d), got (%d)",
220.             MIN_COMPAT_JOURNAL_FORMAT_VERSION, CURRENT_JOURNAL_FORMAT_VERSION,
221.             formatVersion);
222.         LOG.error(err);
223.         throw new IOException(err);
224.     }
225.
226.     try {
227.         if (position == START_OF_FILE) {
228.             if (formatVersion >= V5) {
229.                 fc.position(HEADER_SIZE);
230.             } else if (formatVersion >= V2) {
231.                 fc.position(VERSION_HEADER_SIZE);
232.             } else {
233.                 fc.position(0);
234.             }
235.         } else {
236.             fc.position(position);
237.         }
238.     } catch (IOException e) {
239.         LOG.error("Bookie journal file can seek to position :", e);
240.         throw e;
241.     }
242. }
243.
244. if (fRemoveFromPageCache) {
245.     this.fd = PageCacheUtil.getSysFileDescriptor(channel.getFD());
246. } else {
247.     this.fd = -1;
248. }

```

Immagine 48: Jacoco JournalChannel LLM writeHeader

```

250. private void writeHeader(Journal.BufferedChannelBuilder bcBuilder,
251.     int writeBufferSize) throws IOException {
252.     int headerSize = (V4 == formatVersion) ? VERSION_HEADER_SIZE : HEADER_SIZE;
253.     ByteBuffer bb = ByteBuffer.allocate(headerSize);
254.     ZeroBuffer.put(bb);
255.     bb.clear();
256.     bb.put(magicWord);
257.     bb.putInt(formatVersion);
258.     bb.clear();
259.     fc.write(bb);
260.
261.     bc = bcBuilder.create(fc, writeBufferSize);
262.     forceWrite(true);
263.     nextPrealloc = this.preAllocSize;
264.     fc.write(zeros, nextPrealloc - journalAlignSize);
265. }

```

Immagine 49: Pit Test JournalChannel LLM

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	99% 95/96	92% 56/61	92% 56/61
JournalChannel.java	91% 106/117	60% 32/53	67% 32/48

Immagine 50: Pit Test JournalChannel LLM Costruttore Parte 1

```

150     private JournalChannel(File journalDirectory, long logId,
151                           long preAllocSize, int writeBufferSize, int journalAlignSize,
152                           long position, boolean fRemoveFromPageCache,
153                           int formatVersionToWrite, Journal.BufferedChannelBuilder bcBuilder,
154                           ServerConfiguration conf,
155                           FileChannelProvider provider, Long toReplaceLogId) throws IOException {
156         this.journalAlignSize = journalAlignSize;
157         this.zeros = ByteBuffer.allocate(journalAlignSize);
158         this.preAllocSize = preAllocSize - preAllocSize % journalAlignSize;
159         this.fRemoveFromPageCache = fRemoveFromPageCache;
160         this.configuration = conf;
161
162         boolean reuseFile = false;
163         File fn = new File(journalDirectory, Long.toHexString(logId) + ".txn");
164         if (toReplaceLogId != null && logId != toReplaceLogId && provider.supportReuseFile()) {
165             File toReplaceFile = new File(journalDirectory, Long.toHexString(toReplaceLogId) + ".txn");
166             if (toReplaceFile.exists()) {
167                 renameJournalFile(toReplaceFile, fn);
168                 provider.notifyRename(toReplaceFile, fn);
169                 reuseFile = true;
170             }
171         }
172         channel = provider.open(fn, configuration);
173
174         if (formatVersionToWrite < V4) {
175             throw new IOException("Invalid journal format to write : version = " + formatVersionToWrite);
176         }
177
178         LOG.info("Opening journal {}", fn);
179         if (!channel.fileExists(fn)) { // create new journal file to write, write version
180             if (!fn.createNewFile()) {
181                 LOG.error("Journal file {}, that shouldn't exist, already exists. "
182                     + " is there another bookie process running?", fn);
183                 throw new IOException("File " + fn
184                     + " suddenly appeared, is another bookie process running?");
185             }
186             fc = channel.getFileChannel();
187             formatVersion = formatVersionToWrite;
188             writeHeader(bcBuilder, writeBufferSize);
189         } else if (reuseFile) { // Open an existing journal to write, it needs fileChannelProvider support reuse file.
190             fc = channel.getFileChannel();
191             formatVersion = formatVersionToWrite;
192             writeHeader(bcBuilder, writeBufferSize);
193         } else { // open an existing file to read.
194             fc = channel.getFileChannel();
195             // readonly, use fileChannel directly, no need to use BufferedChannel
196
197             ByteBuffer bb = ByteBuffer.allocate(VERSION_HEADER_SIZE);
198             int c = fc.read(bb);
199             bb.flip();
200

```

Immagine 51: Pit Test JournalChannel LLM Costruttore Parte 2

```

200
201         if (c == VERSION_HEADER_SIZE) {
202             byte[] first4 = new byte[4];
203             bb.get(first4);
204
205             if (Arrays.equals(first4, magicWord)) {
206                 formatVersion = bb.getInt();
207             } else {
208                 // pre magic word journal, reset to 0;
209                 formatVersion = V1;
210             }
211         } else {
212             // no header, must be old version
213             formatVersion = V1;
214         }
215
216         if (formatVersion < MIN_COMPAT_JOURNAL_FORMAT_VERSION
217             || formatVersion > CURRENT_JOURNAL_FORMAT_VERSION) {
218             String err = String.format("Invalid journal version, unable to read."
219                 + " Expected between (%d) and (%d), got (%d)",
220                 MIN_COMPAT_JOURNAL_FORMAT_VERSION, CURRENT_JOURNAL_FORMAT_VERSION,
221                 formatVersion);
222             LOG.error(err);
223             throw new IOException(err);
224         }
225
226         try {
227             if (position == START_OF_FILE) {
228                 if (formatVersion >= V5) {
229                     fc.position(HEADER_SIZE);
230                 } else if (formatVersion >= V2) {
231                     fc.position(VERSION_HEADER_SIZE);
232                 } else {
233                     fc.position(0);
234                 }
235             } else {
236                 fc.position(position);
237             }
238         } catch (IOException e) {
239             LOG.error("Bookie journal file can seek to position :", e);
240             throw e;
241         }
242     }
243     if (fRemoveFromPageCache) {
244         this.fd = PageCacheUtil.getSysFileDescriptor(channel.getFD());
245     } else {
246         this.fd = -1;
247     }
248 }

```

Immagine 52: Pit Test JournalChannel LLM writeHeader

```
250 private void writeHeader(Journal.BufferedChannelBuilder bcBuilder,
251                          int writeBufferSize) throws IOException {
252 1   int headerSize = (V4 == formatVersion) ? VERSION_HEADER_SIZE : HEADER_SIZE;
253   ByteBuffer bb = ByteBuffer.allocate(headerSize);
254 1   ZeroBuffer.put(bb);
255   bb.clear();
256   bb.put(magicWord);
257   bb.putInt(formatVersion);
258   bb.clear();
259   fc.write(bb);
260
261   bc = bcBuilder.create(fc, writeBufferSize);
262 1   forceWrite(true);
263   nextPrealloc = this.preAllocSize;
264 1   fc.write(zeros, nextPrealloc - journalAlignSize);
265 }
---
```

Immagine 53 : Jacoco JournalChannel Tutti i Test

JournalChannel

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
forceWrite(boolean)		66%		66%	2	4	2	9	0	1
JournalChannel(File_Long_Long_Int_Int_Long_Boolean_Int_Journal.BufferedChannelBuilder_ServerConfiguration_FileChannelProvider_Long)		100%		100%	0	17	0	65	0	1
writeHeader(Journal.BufferedChannelBuilder_Int)		100%		100%	0	2	0	13	0	1
preAllocNeeded(long)		100%		100%	0	2	0	5	0	1
renameJournalFile(File_File)		100%		83%	1	4	0	4	0	1
JournalChannel(File_Long_Long_Int_Long_ServerConfiguration_FileChannelProvider)		100%		n/a	0	1	0	2	0	1
JournalChannel(File_Long_Long_Int_Int_Boolean_Int_Journal.BufferedChannelBuilder_ServerConfiguration_FileChannelProvider_Long)		100%		n/a	0	1	0	2	0	1
JournalChannel(File_Long)		100%		n/a	0	1	0	2	0	1
JournalChannel(File_Long_Long_Int_Int_Boolean_Int_ServerConfiguration_FileChannelProvider)		100%		n/a	0	1	0	2	0	1
close()		100%		75%	1	3	0	5	0	1
getBufferedChannel()		100%		100%	0	2	0	3	0	1
JournalChannel(File_Long_Long_Int_ServerConfiguration_FileChannelProvider)		100%		n/a	0	1	0	2	0	1
read(ByteBuffer)		100%		n/a	0	1	0	1	0	1
static {...}		100%		n/a	0	1	0	1	0	1
getFormatVersion()		100%		n/a	0	1	0	1	0	1
Total	12 of 568	97%	4 of 54	92%	4	42	2	117	0	15

Immagine 54 : Pit Test JournalChannel Tutti i Test

Name	Line Coverage	Mutation Coverage	Test Strength
BufferedChannel.java	100% 96/96	98% 60/61	98% 60/61
JournalChannel.java	98% 115/117	81% 43/53	84% 43/51

4.2.3 CacheMap

Immagine 55 : Jacoco CacheMap test manuali

CacheMap

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● clear()		0%		n/a	1	1	10	10	1	1
● unpin(Object)		0%		0%	2	2	8	8	1	1
● putAll(Map, boolean)		0%		0%	4	4	6	6	1	1
● containsValue(Object)		0%		0%	4	4	3	3	1	1
● remove(Object)		57%		62%	2	5	5	14	0	1
● notifyEntryRemovals(Set)		0%		0%	3	3	6	6	1	1
● toString()		0%		n/a	1	1	3	3	1	1
● put(Object, Object)		88%		90%	1	6	2	22	0	1
● getSoftReferenceSize()		0%		0%	2	2	2	2	1	1
● get(Object)		82%		66%	2	4	2	13	0	1
● softMapValueExpired(Object)		0%		n/a	1	1	2	2	1	1
● cacheMapOverflowRemoved(Object, Object)		76%		50%	1	2	1	4	0	1
● CacheMap(boolean)		0%		n/a	1	1	2	2	1	1
● putAll(Map)		0%		n/a	1	1	2	2	1	1
● keySet()		0%		n/a	1	1	1	1	1	1
● values()		0%		n/a	1	1	1	1	1	1
● entrySet()		0%		n/a	1	1	1	1	1	1
● isLRU()		0%		n/a	1	1	1	1	1	1
● setCacheSize(int)		84%		50%	1	2	0	4	0	1
● setSoftReferenceSize(int)		84%		50%	1	2	0	4	0	1
● pin(Object)		98%		87%	1	5	0	12	0	1
● isEmpty()		85%		50%	1	2	0	1	0	1
● CacheMap(boolean, int, int, float, int)		100%		100%	0	4	0	16	0	1
● containsKey(Object)		100%		100%	0	4	0	3	0	1
● size()		100%		n/a	0	1	0	3	0	1
● getCacheSize()		100%		100%	0	2	0	2	0	1
● getPinnedKeys()		100%		n/a	0	1	0	3	0	1
● CacheMap(boolean, int)		100%		n/a	0	1	0	2	0	1
● CacheMap(boolean, int, int, float)		100%		n/a	0	1	0	2	0	1
● softMapOverflowRemoved(Object, Object)		100%		n/a	0	1	0	2	0	1
● CacheMap()		100%		n/a	0	1	0	2	0	1
● put(Map, Object, Object)		100%		n/a	0	1	0	1	0	1
● remove(Map, Object)		100%		n/a	0	1	0	1	0	1
● readLock()		100%		n/a	0	1	0	2	0	1
● readUnlock()		100%		n/a	0	1	0	2	0	1
● writeLock()		100%		n/a	0	1	0	2	0	1
● writeUnlock()		100%		n/a	0	1	0	2	0	1
● entryRemoved(Object, Object, boolean)		100%		n/a	0	1	0	1	0	1
● entryAdded(Object, Object)		100%		n/a	0	1	0	1	0	1
Total	254 of 692	63%	31 of 74	58%	34	76	58	169	14	39

Immagine 56 : Jacoco CacheMap put test manuali

```

383.     @Override
384.     public Object put(Object key, Object value) {
385.         writeLock();
386.         try {
387.             // if the key is pinned, just interact directly with the pinned map
388.             Object val;
389.             if (pinnedMap.containsKey(key)) {
390.                 val = put(pinnedMap, key, value);
391.                 if (val == null) {
392.                     _pinnedSize++;
393.                     entryAdded(key, value);
394.                 } else {
395.                     entryRemoved(key, val, false);
396.                     entryAdded(key, value);
397.                 }
398.                 return val;
399.             }
400.
401.             // if no hard refs, don't put anything
402.             if (cacheMap.getMaxSize() == 0)
403.                 return null;
404.
405.             // otherwise, put the value into the map and clear it from the
406.             // soft map
407.             val = put(cacheMap, key, value);
408.             if (val == null) {
409.                 val = remove(softMap, key);
410.                 if (val == null)
411.                     entryAdded(key, value);
412.                 else {
413.                     entryRemoved(key, val, false);
414.                     entryAdded(key, value);
415.                 }
416.             } else {
417.                 entryRemoved(key, val, false);
418.                 entryAdded(key, value);
419.             }
420.             return val;
421.         } finally {
422.             writeUnlock();
423.         }
424.     }

```

Immagine 57 : Jacoco CacheMap remove Test Manuali

```

441.     /**
442.      * If <code>key</code> is pinned into the cache, the pin is
443.      * cleared and the object is removed.
444.      */
445.     @Override
446.     public Object remove(Object key) {
447.         writeLock();
448.         try {
449.             // if the key is pinned, just interact directly with the
450.             // pinned map
451.             Object val;
452.             if (pinnedMap.containsKey(key)) {
453.                 // re-put with null value; we still want key pinned
454.                 val = put(pinnedMap, key, null);
455.                 if (val != null) {
456.                     _pinnedSize--;
457.                     entryRemoved(key, val, false);
458.                 }
459.                 return val;
460.             }
461.
462.             val = remove(cacheMap, key);
463.             if (val == null)
464.                 val = softMap.remove(key);
465.             if (val != null)
466.                 entryRemoved(key, val, false);
467.
468.             return val;
469.         } finally {
470.             writeUnlock();
471.         }
472.     }

```

Immagine 58 : Jacoco CacheMap Costruttore Test Manuali

```

105.  /**
106.   * Create a cache map with the given properties.
107.   *
108.   * @since 1.1.0
109.   */
110.  public CacheMap(boolean lru, int max, int size, float load,
111.                  int concurrencyLevel) {
112.      if (size < 0)
113.          size = 500;
114.
115.      softMap = new ConcurrentReferenceHashMap(AbstractReferenceMap.ReferenceStrength.HARD,
116.          AbstractReferenceMap.ReferenceStrength.SOFT, size, load) {
117.          @Override
118.          public void overflowRemoved(Object key, Object value) {
119.              softMapOverflowRemoved(key, value);
120.          }
121.
122.          @Override
123.          public void valueExpired(Object key) {
124.              softMapValueExpired(key);
125.          }
126.      };
127.      pinnedMap = new ConcurrentHashMap();
128.
129.      if (!lru) {
130.          cacheMap = new ConcurrentHashMap(size, load) {
131.
132.              private static final long serialVersionUID = 1L;
133.
134.              @Override
135.              public void overflowRemoved(Object key, Object value) {
136.                  cacheMapOverflowRemoved(key, value);
137.              }
138.          };
139.      } else {
140.          cacheMap = new LRUMap(size, load) {
141.
142.              private static final long serialVersionUID = 1L;
143.
144.              @Override
145.              public void overflowRemoved(Object key, Object value) {
146.                  cacheMapOverflowRemoved(key, value);
147.              }
148.          };
149.      }
150.      if (max < 0)
151.          max = Integer.MAX_VALUE;
152.      cacheMap.setMaxSize(max);
153.  }

```

Immagine 59 : Jacoco CacheMap pin Test Manuali

```

281.  /**
282.   * Locks the given key and its value into the map. Objects pinned into
283.   * the map are not counted towards the maximum cache size, and are never
284.   * evicted implicitly. You may pin keys for which no value is in the map.
285.   *
286.   * @return true if the givne key's value was pinned; false if no value
287.   *         for the given key is cached
288.   */
289.  public boolean pin(Object key) {
290.      writeLock();
291.      try {
292.          // if we don't have a pinned map we need to create one; else if the
293.          // pinned map already contains the key, nothing to do
294.          if (pinnedMap.containsKey(key))
295.              return pinnedMap.get(key) != null;
296.
297.          // check other maps for key
298.          Object val = remove(cacheMap, key);
299.          if (val == null)
300.              val = remove(softMap, key);
301.
302.          // pin key
303.          put(pinnedMap, key, val);
304.          if (val != null) {
305.              _pinnedSize++;
306.              return true;
307.          }
308.          return false;
309.      } finally {
310.          writeUnlock();
311.      }
312.  }
313.
314.

```

Immagine 60 : Jacoco CacheMap Test manuali dopo prima iterazione

CacheMap

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
clear()		0%		n/a	1 1	10 10	1 1
unpin(Object)		0%		0%	2 2	8 8	1 1
putAll(Map, boolean)		0%		0%	4 4	6 6	1 1
containsValue(Object)		0%		0%	4 4	3 3	1 1
remove(Object)		57%		62%	2 5	5 14	0 1
notifyEntryRemovals(Set)		0%		0%	3 3	6 6	1 1
toString()		0%		n/a	1 1	3 3	1 1
getSoftReferenceSize()		0%		0%	2 2	2 2	1 1
get(Object)		82%		66%	2 4	2 13	0 1
softMapValueExpired(Object)		0%		n/a	1 1	2 2	1 1
cacheMapOverflowRemoved(Object, Object)		76%		50%	1 2	1 4	0 1
CacheMap(boolean)		0%		n/a	1 1	2 2	1 1
putAll(Map)		0%		n/a	1 1	2 2	1 1
keySet()		0%		n/a	1 1	1 1	1 1
values()		0%		n/a	1 1	1 1	1 1
entrySet()		0%		n/a	1 1	1 1	1 1
isLRU()		0%		n/a	1 1	1 1	1 1
setCacheSize(int)		84%		50%	1 2	0 4	0 1
setSoftReferenceSize(int)		84%		50%	1 2	0 4	0 1
isEmpty()		85%		50%	1 2	0 1	0 1
put(Object, Object)		100%		100%	0 6	0 22	0 1
CacheMap(boolean, int, int, float, int)		100%		100%	0 4	0 16	0 1
pin(Object)		100%		100%	0 5	0 12	0 1
containsKey(Object)		100%		100%	0 4	0 3	0 1
size()		100%		n/a	0 1	0 3	0 1
getCacheSize()		100%		100%	0 2	0 2	0 1
getPinnedKeys()		100%		n/a	0 1	0 3	0 1
CacheMap(boolean, int)		100%		n/a	0 1	0 2	0 1
CacheMap(boolean, int, int, float)		100%		n/a	0 1	0 2	0 1
softMapOverflowRemoved(Object, Object)		100%		n/a	0 1	0 2	0 1
CacheMap()		100%		n/a	0 1	0 2	0 1
put(Map, Object, Object)		100%		n/a	0 1	0 1	0 1
remove(Map, Object)		100%		n/a	0 1	0 1	0 1
readLock()		100%		n/a	0 1	0 2	0 1
readUnlock()		100%		n/a	0 1	0 2	0 1
writeLock()		100%		n/a	0 1	0 2	0 1
writeUnlock()		100%		n/a	0 1	0 2	0 1
entryRemoved(Object, Object, boolean)		100%		n/a	0 1	0 1	0 1
entryAdded(Object, Object)		100%		n/a	0 1	0 1	0 1
Total	242 of 692	65%	29 of 74	60%	32 76	56 169	14 39

Immagine 61 : Pit Test CacheMap Test manuali dopo prima iterazione

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	65% 118/182	50% 66/133	74% 66/89
ProxyManagerImpl.java	77% 837/1084	65% 443/684	80% 443/555

Immagine 62 : Pit Test CacheMap Costruttore Dopo prima iterazione

```

110     public CacheMap(boolean lru, int max, int size, float load,
111                     int concurrencyLevel) {
112 2       if (size < 0)
113         size = 500;
114
115         softMap = new ConcurrentReferenceHashMap(AbstractReferenceMap.ReferenceStrength.HARD,
116         AbstractReferenceMap.ReferenceStrength.SOFT, size, load) {
117             @Override
118             public void overflowRemoved(Object key, Object value) {
119                 softMapOverflowRemoved(key, value);
120             }
121
122             @Override
123             public void valueExpired(Object key) {
124                 softMapValueExpired(key);
125             }
126         };
127         pinnedMap = new ConcurrentHashMap();
128
129 1       if (!lru) {
130             cacheMap = new ConcurrentHashMap(size, load) {
131
132                 private static final long serialVersionUID = 1L;
133
134                 @Override
135                 public void overflowRemoved(Object key, Object value) {
136                     cacheMapOverflowRemoved(key, value);
137                 }
138             };
139         } else {
140             cacheMap = new LRUMap(size, load) {
141
142                 private static final long serialVersionUID = 1L;
143
144                 @Override
145                 public void overflowRemoved(Object key, Object value) {
146                     cacheMapOverflowRemoved(key, value);
147                 }
148             };
149         }
150 2       if (max < 0)
151         max = Integer.MAX_VALUE;
152 1       cacheMap.setMaxSize(max);
153     }
154

```

Immagine 63 : Pit test CacheMap pin dopo prima iterazione

```

290     public boolean pin(Object key) {
291 1       writeLock();
292         try {
293             // if we don't have a pinned map we need to create one; else if the
294             // pinned map already contains the key, nothing to do
295 1       if (pinnedMap.containsKey(key))
296 3       return pinnedMap.get(key) != null;
297
298             // check other maps for key
299             Object val = remove(cacheMap, key);
300 1       if (val == null)
301             val = remove(softMap, key);
302
303             // pin key
304             put(pinnedMap, key, val);
305 1       if (val != null) {
306 1       _pinnedSize++;
307 2       return true;
308         }
309 2       return false;
310     } finally {
311 1       writeUnlock();
312     }
313 }
314

```


Immagine 64 : Pit Test CacheMap put dopo prima iterazione

```

383  @Override
384  public Object put(Object key, Object value) {
385  1  writeLock();
386      try {
387          // if the key is pinned, just interact directly with the pinned map
388          Object val;
389  1  if (pinnedMap.containsKey(key)) {
390              val = put(pinnedMap, key, value);
391  1  if (val == null) {
392  1  _pinnedSize++;
393  1  entryAdded(key, value);
394              } else {
395  1  entryRemoved(key, val, false);
396  1  entryAdded(key, value);
397              }
398  1  return val;
399      }
400
401      // if no hard refs, don't put anything
402  1  if (cacheMap.getMaxSize() == 0)
403      return null;
404
405      // otherwise, put the value into the map and clear it from the
406      // soft map
407      val = put(cacheMap, key, value);
408  1  if (val == null) {
409          val = remove(softMap, key);
410  1  if (val == null)
411  1  entryAdded(key, value);
412      } else {
413  1  entryRemoved(key, val, false);
414  1  entryAdded(key, value);
415      }
416      } else {
417  1  entryRemoved(key, val, false);
418  1  entryAdded(key, value);
419      }
420  1  return val;
421      } finally {
422  1  writeUnlock();
423      }
424  }

```

Immagine 65 : PitTest CacheMap remove dopo prima iterazione

```

445  @Override
446  public Object remove(Object key) {
447  1  writeLock();
448      try {
449          // if the key is pinned, just interact directly with the
450          // pinned map
451          Object val;
452  1  if (pinnedMap.containsKey(key)) {
453              // re-put with null value; we still want key pinned
454              val = put(pinnedMap, key, null);
455  1  if (val != null) {
456  1  _pinnedSize--;
457  1  entryRemoved(key, val, false);
458              }
459  1  return val;
460      }
461
462      val = remove(cacheMap, key);
463  1  if (val == null)
464          val = softMap.remove(key);
465  1  if (val != null)
466  1  entryRemoved(key, val, false);
467
468  1  return val;
469      } finally {
470  1  writeUnlock();
471      }
472  }

```

Immagine 66 : Pit Test CacheMap dopo seconda iterazione

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	65% 118/182	61% 81/133	91% 81/89
ProxyManagerImpl.java	77% 837/1084	65% 443/684	80% 443/555

CacheMap

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
remove(Object)		57%		62%	2	5	5	14	0	1
softMapOverflowRemoved(Object, Object)		0%		n/a	1	1	2	2	1	1
softMapValueExpired(Object)		0%		n/a	1	1	2	2	1	1
cacheMapOverflowRemoved(Object, Object)		76%		50%	1	2	1	4	0	1
put(Object, Object)		95%		90%	1	6	1	22	0	1
CacheMap(boolean, int, int, float, int)		94%		66%	2	4	2	16	0	1
pin(Object)		98%		87%	1	5	0	12	0	1
get(Object)		100%		100%	0	4	0	13	0	1
clear()		100%		n/a	0	1	0	10	0	1
unpin(Object)		100%		100%	0	2	0	8	0	1
putAll(Map, boolean)		100%		100%	0	4	0	6	0	1
containsKey(Object)		100%		83%	1	4	0	3	0	1
containsValue(Object)		100%		83%	1	4	0	3	0	1
notifyEntryRemovals(Set)		100%		75%	1	3	0	6	0	1
size()		100%		n/a	0	1	0	3	0	1
toString()		100%		n/a	0	1	0	3	0	1
setCacheSize(int)		100%		100%	0	2	0	4	0	1
setSoftReferenceSize(int)		100%		100%	0	2	0	4	0	1
getCacheSize()		100%		100%	0	2	0	2	0	1
getSoftReferenceSize()		100%		100%	0	2	0	2	0	1
getPinnedKeys()		100%		n/a	0	1	0	3	0	1
CacheMap(boolean, int)		100%		n/a	0	1	0	2	0	1
CacheMap(boolean, int, int, float)		100%		n/a	0	1	0	2	0	1
isEmpty()		100%		100%	0	2	0	1	0	1
CacheMap()		100%		n/a	0	1	0	2	0	1
CacheMap(boolean)		100%		n/a	0	1	0	2	0	1
put(Map, Object, Object)		100%		n/a	0	1	0	1	0	1
putAll(Map)		100%		n/a	0	1	0	2	0	1
keySet()		100%		n/a	0	1	0	1	0	1
values()		100%		n/a	0	1	0	1	0	1
entrySet()		100%		n/a	0	1	0	1	0	1
remove(Map, Object)		100%		n/a	0	1	0	1	0	1
readLock()		100%		n/a	0	1	0	2	0	1
readUnlock()		100%		n/a	0	1	0	2	0	1
writeLock()		100%		n/a	0	1	0	2	0	1
writeUnlock()		100%		n/a	0	1	0	2	0	1
isLRU()		100%		n/a	0	1	0	1	0	1
entryRemoved(Object, Object, boolean)		100%		n/a	0	1	0	1	0	1
entryAdded(Object, Object)		100%		n/a	0	1	0	1	0	1
Total	50 of 692	92%	11 of 74	85%	12	76	13	169	2	39

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	87% 159/182	64% 85/133	67% 85/126
ProxyManagerImpl.java	48% 516/1084	32% 222/684	70% 222/319

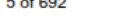
CacheMap

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
put(Object, Object)		41%		40%	5 6	12 22	0 1
pin(Object)		63%		37%	4 5	3 12	0 1
putAll(Map, boolean)		27%		16%	3 4	4 6	0 1
cacheMapOverflowRemoved(Object, Object)		0%		0%	2 2	4 4	1 1
notifyEntryRemovals(Set)		29%		25%	2 3	4 6	0 1
remove(Object)		71%		62%	3 5	3 14	0 1
unpin(Object)		51%		50%	1 2	3 8	0 1
toString()		0%		n/a	1 1	3 3	1 1
setCacheSize(int)		0%		0%	2 2	4 4	1 1
CacheMap(boolean, int, int, float, int)		85%		66%	2 4	2 16	0 1
get(Object)		82%		50%	3 4	2 13	0 1
softMapOverflowRemoved(Object, Object)		0%		n/a	1 1	2 2	1 1
softMapValueExpired(Object)		0%		n/a	1 1	2 2	1 1
isLRU()		0%		n/a	1 1	1 1	1 1
containsKey(Object)		92%		50%	3 4	0 3	0 1
containsValue(Object)		92%		50%	3 4	0 3	0 1
setSoftReferenceSize(int)		84%		50%	1 2	0 4	0 1
getCacheSize()		81%		50%	1 2	0 2	0 1
getSoftReferenceSize()		81%		50%	1 2	0 2	0 1
entryRemoved(Object, Object, boolean)		0%		n/a	1 1	1 1	1 1
clear()		100%		n/a	0 1	0 10	0 1
size()		100%		n/a	0 1	0 3	0 1
getPinnedKeys()		100%		n/a	0 1	0 3	0 1
CacheMap(boolean, int)		100%		n/a	0 1	0 2	0 1
CacheMap(boolean, int, int, float)		100%		n/a	0 1	0 2	0 1
isEmpty()		100%		100%	0 2	0 1	0 1
CacheMap()		100%		n/a	0 1	0 2	0 1
CacheMap(boolean)		100%		n/a	0 1	0 2	0 1
put(Map, Object, Object)		100%		n/a	0 1	0 1	0 1
putAll(Map)		100%		n/a	0 1	0 2	0 1
keySet()		100%		n/a	0 1	0 1	0 1
values()		100%		n/a	0 1	0 1	0 1
entrySet()		100%		n/a	0 1	0 1	0 1
remove(Map, Object)		100%		n/a	0 1	0 1	0 1
readLock()		100%		n/a	0 1	0 2	0 1
readUnlock()		100%		n/a	0 1	0 2	0 1
writeLock()		100%		n/a	0 1	0 2	0 1
writeUnlock()		100%		n/a	0 1	0 2	0 1
entryAdded(Object, Object)		100%		n/a	0 1	0 1	0 1
Total	238 of 692	65%	41 of 74	44%	41 76	50 169	7 39

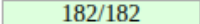
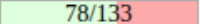
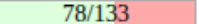
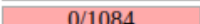
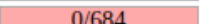
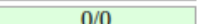
Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	66% 121/182	34% 45/133	49% 45/92
ProxyManagerImpl.java	40% 430/1084	21% 146/684	56% 146/262

CacheMap

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
cacheMapOverflowRemoved(Object, Object)		76%		50%	1	2	1	4	0	1
put(Object, Object)		100%		100%	0	6	0	22	0	1
CacheMap(boolean, int, int, float, int)		100%		100%	0	4	0	16	0	1
pin(Object)		100%		100%	0	5	0	12	0	1
remove(Object)		100%		100%	0	5	0	14	0	1
get(Object)		100%		100%	0	4	0	13	0	1
clear()		100%	n/a	n/a	0	1	0	10	0	1
unpin(Object)		100%		100%	0	2	0	8	0	1
putAll(Map, boolean)		100%		100%	0	4	0	6	0	1
containsKey(Object)		100%		100%	0	4	0	3	0	1
containsValue(Object)		100%		83%	1	4	0	3	0	1
notifyEntryRemovals(Set)		100%		100%	0	3	0	6	0	1
size()		100%	n/a	n/a	0	1	0	3	0	1
toString()		100%	n/a	n/a	0	1	0	3	0	1
setCacheSize(int)		100%		100%	0	2	0	4	0	1
setSoftReferenceSize(int)		100%		100%	0	2	0	4	0	1
getCacheSize()		100%		100%	0	2	0	2	0	1
getSoftReferenceSize()		100%		100%	0	2	0	2	0	1
getPinnedKeys()		100%	n/a	n/a	0	1	0	3	0	1
CacheMap(boolean, int)		100%	n/a	n/a	0	1	0	2	0	1
CacheMap(boolean, int, int, float)		100%	n/a	n/a	0	1	0	2	0	1
isEmpty()		100%		100%	0	2	0	1	0	1
softMapOverflowRemoved(Object, Object)		100%	n/a	n/a	0	1	0	2	0	1
softMapValueExpired(Object)		100%	n/a	n/a	0	1	0	2	0	1
CacheMap()		100%	n/a	n/a	0	1	0	2	0	1
CacheMap(boolean)		100%	n/a	n/a	0	1	0	2	0	1
put(Map, Object, Object)		100%	n/a	n/a	0	1	0	1	0	1
putAll(Map)		100%	n/a	n/a	0	1	0	2	0	1
keySet()		100%	n/a	n/a	0	1	0	1	0	1
values()		100%	n/a	n/a	0	1	0	1	0	1
entrySet()		100%	n/a	n/a	0	1	0	1	0	1
remove(Map, Object)		100%	n/a	n/a	0	1	0	1	0	1
readLock()		100%	n/a	n/a	0	1	0	2	0	1
readUnlock()		100%	n/a	n/a	0	1	0	2	0	1
writeLock()		100%	n/a	n/a	0	1	0	2	0	1
writeUnlock()		100%	n/a	n/a	0	1	0	2	0	1
isLRU()		100%	n/a	n/a	0	1	0	1	0	1
entryRemoved(Object, Object, boolean)		100%	n/a	n/a	0	1	0	1	0	1
entryAdded(Object, Object)		100%	n/a	n/a	0	1	0	1	0	1
Total	5 of 692	99%	2 of 74	97%	2	76	1	169	0	39

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	100%  182/182	59%  78/133	59%  78/133
ProxyManagerImpl.java	0%  0/1084	0%  0/684	100%  0/0

CacheMap

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
cacheMapOverflowRemoved(Object, Object)		76%		50%	1	2	1	4	0	1
put(Object, Object)		100%		100%	0	6	0	22	0	1
CacheMap(boolean, int, int, float, int)		100%		100%	0	4	0	16	0	1
pin(Object)		100%		100%	0	5	0	12	0	1
remove(Object)		100%		100%	0	5	0	14	0	1
get(Object)		100%		100%	0	4	0	13	0	1
clear()		100%		n/a	0	1	0	10	0	1
unpin(Object)		100%		100%	0	2	0	8	0	1
putAll(Map, boolean)		100%		100%	0	4	0	6	0	1
containsKey(Object)		100%		100%	0	4	0	3	0	1
containsValue(Object)		100%		83%	1	4	0	3	0	1
notifyEntryRemovals(Set)		100%		100%	0	3	0	6	0	1
size()		100%		n/a	0	1	0	3	0	1
toString()		100%		n/a	0	1	0	3	0	1
setCacheSize(int)		100%		100%	0	2	0	4	0	1
setSoftReferenceSize(int)		100%		100%	0	2	0	4	0	1
getCacheSize()		100%		100%	0	2	0	2	0	1
getSoftReferenceSize()		100%		100%	0	2	0	2	0	1
getPinnedKeys()		100%		n/a	0	1	0	3	0	1
CacheMap(boolean, int)		100%		n/a	0	1	0	2	0	1
CacheMap(boolean, int, int, float)		100%		n/a	0	1	0	2	0	1
isEmpty()		100%		100%	0	2	0	1	0	1
softMapOverflowRemoved(Object, Object)		100%		n/a	0	1	0	2	0	1
softMapValueExpired(Object)		100%		n/a	0	1	0	2	0	1
CacheMap()		100%		n/a	0	1	0	2	0	1
CacheMap(boolean)		100%		n/a	0	1	0	2	0	1
put(Map, Object, Object)		100%		n/a	0	1	0	1	0	1
putAll(Map)		100%		n/a	0	1	0	2	0	1
keySet()		100%		n/a	0	1	0	1	0	1
values()		100%		n/a	0	1	0	1	0	1
entrySet()		100%		n/a	0	1	0	1	0	1
remove(Map, Object)		100%		n/a	0	1	0	1	0	1
readLock()		100%		n/a	0	1	0	2	0	1
readUnlock()		100%		n/a	0	1	0	2	0	1
writeLock()		100%		n/a	0	1	0	2	0	1
writeUnlock()		100%		n/a	0	1	0	2	0	1
isLRU()		100%		n/a	0	1	0	1	0	1
entryRemoved(Object, Object, boolean)		100%		n/a	0	1	0	1	0	1
entryAdded(Object, Object)		100%		n/a	0	1	0	1	0	1
Total	5 of 692	99%	2 of 74	97%	2	76	1	169	0	39

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	97%	83%	84%
ProxyManagerImpl.java	89%	69%	75%

4.2.4 ProxyManagerImpl

Immagine 75 : Jacoco ProxyManagerImpl Parte 1

ProxyManagerImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
addProxyMapMethods(ClassWriterTracker_String_Class)		0%		0%	10	10	94	94	1	1
addProxyCollectionMethods(ClassWriterTracker_String_Class)		0%		0%	12	12	84	84	1	1
addProxyCalendarMethods(ClassWriterTracker_String_Class)		0%		0%	2	2	71	71	1	1
addProxyDateMethods(ClassWriterTracker_String_Class)		0%		0%	12	12	67	67	1	1
main(String[])		0%		0%	11	11	50	50	1	1
proxyBeforeAfterMethod(ClassWriterTracker_Class_Method_Class[]_Method_Method_Class[])		0%		0%	11	11	37	37	1	1
proxyRecognizedMethods(ClassWriterTracker_String_Class_Class_Class)		0%		0%	6	6	32	32	1	1
generateProxyCollectionBytecode(Class_boolean_String)		0%		0%	2	2	17	17	1	1
generateProxyMapBytecode(Class_boolean_String)		0%		0%	2	2	17	17	1	1
generateProxyDateBytecode(Class_boolean_String)		0%		0%	2	2	16	16	1	1
generateProxyCalendarBytecode(Class_boolean_String)		0%		0%	2	2	16	16	1	1
proxyOverrideMethod(ClassWriterTracker_Method_Method_Class[])		0%		0%	2	2	12	12	1	1
findGetter(Class_Method)		32%		8%	6	7	15	20	0	1
toHelperAfterParameters(Class[]_Class_Class)		0%		0%	7	7	15	15	1	1
instantiateProxy(Class_Constructor_Object[])		25%		50%	1	3	11	16	0	1
toConcreteType(Class_Map)		0%		0%	4	4	9	9	1	1
copyArray(Object)		0%		0%	2	2	10	10	1	1
toProxyableCollectionType(Class)		35%		37%	4	5	5	9	0	1
toProxyableMapType(Class)		35%		37%	4	5	5	9	0	1
copyMap(Map)		0%		0%	3	3	6	6	1	1
copyDate(Date)		0%		0%	3	3	6	6	1	1
copyCalendar(Calendar)		0%		0%	3	3	6	6	1	1
generateAndLoadProxyDate(Class_boolean_ClassLoader)		0%		0%	2	2	5	5	1	1
generateAndLoadProxyCalendar(Class_boolean_ClassLoader)		0%		0%	2	2	5	5	1	1
generateAndLoadProxyCollection(Class_boolean_ClassLoader)		0%		0%	2	2	5	5	1	1
generateAndLoadProxyMap(Class_boolean_ClassLoader)		0%		0%	2	2	5	5	1	1
toHelperParameters(Class[]_Class)		0%		n/a	1	1	4	4	1	1
findComparatorConstructor(Class)		0%		n/a	1	1	5	5	1	1
loadDelayedProxy(Class)		66%		50%	7	9	7	15	0	1
copyCustom(Object)		75%		81%	3	9	3	16	0	1
findCopyConstructor(Class)		76%		58%	3	7	3	14	0	1
assertNotFinal(Class)		0%		0%	2	2	3	3	1	1
isOrdered(Class)		0%		0%	3	3	2	2	1	1
hasConstructor(Class_Class[])		0%		0%	2	2	3	3	1	1
isProxyable(Class)		61%		50%	3	5	3	8	0	1
copyCollection(Collection)		65%		50%	2	3	2	6	0	1
allowsDuplicates(Class)		0%		0%	2	2	1	1	1	1
getFactoryProxyMap(Class)		85%		50%	2	3	1	9	0	1
getFactoryProxyDate(Class)		85%		50%	2	3	1	9	0	1
getFactoryProxyCalendar(Class)		85%		50%	2	3	1	9	0	1
getFactoryProxyCollection(Class)		85%		50%	2	3	1	9	0	1
generateProxyBeanBytecode(Class_boolean_String)		96%		83%	3	10	2	28	0	1
copyBeanProperties(MethodVisitor_Class_int)		96%		66%	4	7	3	18	0	1
getTrackChanges()		0%		n/a	1	1	1	1	1	1
getAssertAllowedType()		0%		n/a	1	1	1	1	1	1
getDelayCollectionLoading()		0%		n/a	1	1	1	1	1	1
getUnproxyable()		0%		n/a	1	1	1	1	1	1

Imagine 76 : Jacoco ProxyManagerImpl Parte 2

● toHelperParameters(Class[] ,Class)	0%	n/a	1	1	4	4	1	1		
● findComparatorConstructor(Class)	0%	n/a	1	1	5	5	1	1		
● loadDelayedProxy(Class)	66%	50%	7	9	7	15	0	1		
● copyCustom(Object)	75%	81%	3	9	3	16	0	1		
● findCopyConstructor(Class)	76%	58%	3	7	3	14	0	1		
● assertNotFinal(Class)	0%	0%	2	2	3	3	1	1		
● isOrdered(Class)	0%	0%	3	3	2	2	1	1		
● hasConstructor(Class ,Class[])	0%	0%	2	2	3	3	1	1		
● isProxyable(Class)	61%	50%	3	5	3	8	0	1		
● copyCollection(Collection)	65%	50%	2	3	2	6	0	1		
● allowsDuplicates(Class)	0%	0%	2	2	1	1	1	1		
● getFactoryProxyMap(Class)	85%	50%	2	3	1	9	0	1		
● getFactoryProxyDate(Class)	85%	50%	2	3	1	9	0	1		
● getFactoryProxyCalendar(Class)	85%	50%	2	3	1	9	0	1		
● getFactoryProxyCollection(Class)	85%	50%	2	3	1	9	0	1		
● generateProxyBeanBytecode(Class ,boolean ,String)	96%	83%	3	10	2	28	0	1		
● copyBeanProperties(MethodVisitor ,Class ,int)	96%	66%	4	7	3	18	0	1		
● getTrackChanges()	0%	n/a	1	1	1	1	1	1		
● getAssertAllowedType()	0%	n/a	1	1	1	1	1	1		
● getDelayCollectionLoading()	0%	n/a	1	1	1	1	1	1		
● getUnproxyable()	0%	n/a	1	1	1	1	1	1		
● getFactoryProxyBean(Object)	97%	70%	3	6	1	15	0	1		
● newCalendarProxy(Class ,Timezone)	90%	50%	2	3	1	7	0	1		
● isUnproxyable(Class)	89%	66%	2	4	1	5	0	1		
● proxySetter(ClassWriterTracker ,Class ,Method)	99%	75%	1	3	1	22	0	1		
● addWriteReplaceMethod(ClassWriterTracker ,String ,boolean)	98%	50%	1	2	0	12	0	1		
● addProxyMethods(ClassWriterTracker ,boolean ,String ,Class)	100%	50%	1	2	0	54	0	1		
● addProxyBeanMethods(ClassWriterTracker ,String ,Class ,Constructor)	100%	100%	0	5	0	39	0	1		
● newCustomProxy(Object ,boolean)	100%	95%	1	13	0	33	0	1		
● delegateConstructors(ClassWriterTracker ,Class ,String)	100%	100%	0	3	0	17	0	1		
● static {...}	100%	n/a	0	1	0	14	0	1		
● proxySetters(ClassWriterTracker ,String ,Class)	100%	87%	1	5	0	7	0	1		
● isSetter(Method)	100%	58%	5	7	0	6	0	1		
● ProxyManagerImpl()	100%	n/a	0	1	0	8	0	1		
● newMapProxy(Class ,Class ,Class ,Comparator ,boolean)	100%	100%	0	3	0	4	0	1		
● loadBuildTimeProxy(Class ,Class ,loader)	100%	75%	1	3	0	8	0	1		
● newCollectionProxy(Class ,Class ,Comparator ,boolean)	100%	100%	0	2	0	3	0	1		
● addInstanceVariables(ClassWriterTracker)	100%	n/a	0	1	0	5	0	1		
● generateAndLoadProxyBean(Class ,boolean ,Class ,loader)	100%	100%	0	2	0	5	0	1		
● startsWith(String ,String)	100%	66%	2	4	0	3	0	1		
● getProxyClassName(Class ,boolean)	100%	100%	0	2	0	3	0	1		
● setUnproxyable(String)	100%	100%	0	2	0	3	0	1		
● newDateProxy(Class)	100%	n/a	0	1	0	2	0	1		
● nextProxyId()	100%	n/a	0	1	0	1	0	1		
● setTrackChanges(boolean)	100%	n/a	0	1	0	2	0	1		
● setAssertAllowedType(boolean)	100%	n/a	0	1	0	2	0	1		
● setDelayCollectionLoading(boolean)	100%	n/a	0	1	0	2	0	1		
Total	3,097 of 4,896	36%	254 of 414	38%	187	280	674	1,084	32	73

Imagine 77 : Jacoco ProxyManagerImpl newCustomProxy

```

310.     @Override
311.     public Proxy newCustomProxy(Object orig, boolean autoOff) {
312.         if (orig == null)
313.             return null;
314.         if (orig instanceof Proxy)
315.             return (Proxy) orig;
316.         if (ImplHelper.isManageable(orig))
317.             return null;
318.         if (!isProxyable(orig.getClass()))
319.             return null;
320.
321.         if (orig instanceof Collection) {
322.             Comparator comp = (orig instanceof SortedSet)
323.                 ? ((SortedSet) orig).comparator() : null;
324.             Collection c = (Collection) newCollectionProxy(orig.getClass(),
325.                 null, comp, autoOff);
326.             c.addAll((Collection) orig);
327.             return (Proxy) c;
328.         }
329.         if (orig instanceof Map) {
330.             Comparator comp = (orig instanceof SortedMap)
331.                 ? ((SortedMap) orig).comparator() : null;
332.             Map m = (Map) newMapProxy(orig.getClass(), null, null, comp, autoOff);
333.             m.putAll((Map) orig);
334.             return (Proxy) m;
335.         }
336.         if (orig instanceof Date) {
337.             Date d = (Date) newDateProxy(orig.getClass());
338.             d.setTime(((Date) orig).getTime());
339.             if (orig instanceof Timestamp)
340.                 ((Timestamp) d).setNanos(((Timestamp) orig).getNanos());
341.             return (Proxy) d;
342.         }
343.         if (orig instanceof Calendar) {
344.             Calendar c = (Calendar) newCalendarProxy(orig.getClass(),
345.                 ((Calendar) orig).getTimeZone());
346.             c.setTimeInMillis(((Calendar) orig).getTimeInMillis());
347.             return (Proxy) c;
348.         }
349.
350.         ProxyBean proxy = getFactoryProxyBean(orig);
351.         return (proxy == null) ? null : proxy.newInstance(orig);
352.     }

```


Immagine 78 : Jacoco ProxyManagerImpl copyCustom

```
290.     @Override
291.     public Object copyCustom(Object orig) {
292.         ◆ if (orig == null)
293.             return null;
294.         ◆ if (orig instanceof Proxy)
295.             return ((Proxy) orig).copy(orig);
296.         ◆ if (ImplHelper.isManageable(orig))
297.             return null;
298.         ◆ if (orig instanceof Collection)
299.             return copyCollection((Collection) orig);
300.         ◆ if (orig instanceof Map)
301.             return copyMap((Map) orig);
302.         ◆ if (orig instanceof Date)
303.             return copyDate((Date) orig);
304.         ◆ if (orig instanceof Calendar)
305.             return copyCalendar((Calendar) orig);
306.         ProxyBean proxy = getFactoryProxyBean(orig);
307.         ◆ return (proxy == null) ? null : proxy.copy(orig);
308.     }
309.
```

All 2 branches covered

Immagine 79 : Jacoco ProxyManagerImpl copyCollection

```
211.     @Override
212.     public Collection copyCollection(Collection orig) {
213.         ◆ if (orig == null)
214.             return null;
215.         ◆ if (orig instanceof Proxy)
216.             return (Collection) ((Proxy) orig).copy(orig);
217.
218.         ProxyCollection proxy = getFactoryProxyCollection(orig.getClass());
219.         return (Collection) proxy.copy(orig);
220.     }
221.
222.     @Override
223.     public Proxy newCollectionProxy(Class type, Class elementType,
224.                                     Comparator compare, boolean autoOff) {
225.         type = toProxyableCollectionType(type);
226.         ProxyCollection proxy = getFactoryProxyCollection(type);
227.         ◆ return proxy.newInstance((_assertType) ? elementType : null, compare,
228.                                   _trackChanges, autoOff);
229.     }
230.
```


Immagine 80 : Jacoco ProxyManagerImpl loadDelayedProxy

```

531.     protected Class<?> loadDelayedProxy(Class<?> type) {
532.         ◆ if (type.equals(java.util.ArrayList.class)) {
533.             return DelayedArrayListProxy.class;
534.         }
535.         ◆ if (type.equals(java.util.HashSet.class)) {
536.             return DelayedHashSetProxy.class;
537.         }
538.         ◆ if (type.equals(java.util.LinkedList.class)) {
539.             return DelayedLinkedListProxy.class;
540.         }
541.         ◆ if (type.equals(java.util.Vector.class)) {
542.             return DelayedVectorProxy.class;
543.         }
544.         ◆ if (type.equals(java.util.LinkedHashSet.class)) {
545.             return DelayedLinkedHashSetProxy.class;
546.         }
547.         ◆ if (type.equals(java.util.SortedSet.class) || type.equals(java.util.TreeSet.class)) {
548.             return DelayedTreeSetProxy.class;
549.         }
550.         ◆ if (type.equals(java.util.PriorityQueue.class)) {
551.             return DelayedPriorityQueueProxy.class;
552.         }
553.         return null;
554.     }

```

Immagine 81 : Pit Test ProxyManagerImpl Adeguacy

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	64% 116/182	59% 79/133	91% 79/87
ProxyManagerImpl.java	77% 834/1084	64% 437/684	79% 437/553

Immagine 82 : Pit Test ProxyManagerImpl loadDelayedProxy Adeguacy

```

531     protected Class<?> loadDelayedProxy(Class<?> type) {
532     1 if (type.equals(java.util.ArrayList.class)) {
533     1 return DelayedArrayListProxy.class;
534     }
535     1 if (type.equals(java.util.HashSet.class)) {
536     1 return DelayedHashSetProxy.class;
537     }
538     1 if (type.equals(java.util.LinkedList.class)) {
539     1 return DelayedLinkedListProxy.class;
540     }
541     1 if (type.equals(java.util.Vector.class)) {
542     1 return DelayedVectorProxy.class;
543     }
544     1 if (type.equals(java.util.LinkedHashSet.class)) {
545     1 return DelayedLinkedHashSetProxy.class;
546     }
547     2 if (type.equals(java.util.SortedSet.class) || type.equals(java.util.TreeSet.class)) {
548     1 return DelayedTreeSetProxy.class;
549     }
550     1 if (type.equals(java.util.PriorityQueue.class)) {
551     1 return DelayedPriorityQueueProxy.class;
552     }
553     return null;
554     }

```

Immagine 83 : Pit Test ProxyManagerImpl newCustomProxy Adeguacy

```

310     @Override
311     public Proxy newCustomProxy(Object orig, boolean autoOff) {
312 1         if (orig == null)
313             return null;
314 1         if (orig instanceof Proxy)
315 1             return (Proxy) orig;
316 1         if (ImplHelper.isManageable(orig))
317             return null;
318 1         if (!isProxyable(orig.getClass()))
319             return null;
320
321 1         if (orig instanceof Collection) {
322 1             Comparator comp = (orig instanceof SortedSet)
323                 ? ((SortedSet) orig).comparator() : null;
324             Collection c = (Collection) newCollectionProxy(orig.getClass(),
325                 null, comp, autoOff);
326             c.addAll((Collection) orig);
327 1             return (Proxy) c;
328         }
329 1         if (orig instanceof Map) {
330 1             Comparator comp = (orig instanceof SortedMap)
331                 ? ((SortedMap) orig).comparator() : null;
332             Map m = (Map) newMapProxy(orig.getClass(), null, null, comp, autoOff);
333 1             m.putAll((Map) orig);
334 1             return (Proxy) m;
335         }
336 1         if (orig instanceof Date) {
337             Date d = (Date) newDateProxy(orig.getClass());
338 1             d.setTime(((Date) orig).getTime());
339 1             if (orig instanceof Timestamp)
340 1                 ((Timestamp) d).setNanos(((Timestamp) orig).getNanos());
341 1             return (Proxy) d;
342         }
343 1         if (orig instanceof Calendar) {
344             Calendar c = (Calendar) newCalendarProxy(orig.getClass(),
345                 ((Calendar) orig).getTimeZone());
346 1             c.setTimeInMillis(((Calendar) orig).getTimeInMillis());
347 1             return (Proxy) c;
348         }
349
350         ProxyBean proxy = getFactoryProxyBean(orig);
351 2         return (proxy == null) ? null : proxy.newInstance(orig);
352     }

```

Immagine 84 : Pit Test ProxyManagerImpl copyCollection Adeguacy

```

211     @Override
212     public Collection copyCollection(Collection orig) {
213 1         if (orig == null)
214 1             return null;
215 1         if (orig instanceof Proxy)
216 1             return (Collection) ((Proxy) orig).copy(orig);
217
218         ProxyCollection proxy = getFactoryProxyCollection(orig.getClass());
219 1         return (Collection) proxy.copy(orig);
220     }

```

Immagine 85 : Pi Test ProxyManagerImpl setUnproxyable Adeguacy

```

188     public void setUnproxyable(String clsNames) {
189 1         if (clsNames != null)
190             _unproxyable.addAll(Arrays.asList(StringUtil.split(clsNames, ";", 0)));
191     }
192

```

ProxyManagerImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
addProxyMapMethods(ClassWriterTracker_String_Class)		0%		0%	10	10	94	94	1	1
addProxyCalendarMethods(ClassWriterTracker_String_Class)		0%		0%	2	2	71	71	1	1
addProxyDateMethods(ClassWriterTracker_String_Class)		0%		0%	12	12	67	67	1	1
main(String[])		0%		0%	11	11	50	50	1	1
proxyBeforeAfterMethod(ClassWriterTracker_Class_Method_Class[]_Method_Method_Class[])		0%		0%	11	11	37	37	1	1
addProxyBeanMethods(ClassWriterTracker_String_Class_Constructor)		0%		0%	5	5	39	39	1	1
newCustomProxy(Object_boolean)		11%		20%	11	13	27	33	0	1
generateProxyBeanBytecode(Class_boolean_String)		4%		5%	9	10	26	28	0	1
generateProxyMapBytecode(Class_boolean_String)		0%		0%	2	2	17	17	1	1
copyBeanProperties(MethodVisitor_Class_int)		0%		0%	7	7	18	18	1	1
generateProxyDateBytecode(Class_boolean_String)		0%		0%	2	2	16	16	1	1
generateProxyCalendarBytecode(Class_boolean_String)		0%		0%	2	2	16	16	1	1
findGetter(Class_Method)		0%		0%	7	7	20	20	1	1
proxyOverrideMethod(ClassWriterTracker_Method_Method_Class[])		0%		0%	2	2	12	12	1	1
proxyRecognizedMethods(ClassWriterTracker_String_Class_Class_Class)		52%		60%	4	6	12	32	0	1
addProxyCollectionMethods(ClassWriterTracker_String_Class)		87%		40%	11	12	8	84	0	1
instantiateProxy(Class_Constructor_Object[])		11%		25%	2	3	13	16	0	1
toConcreteType(Class_Map)		0%		0%	4	4	9	9	1	1
copyCustom(Object)		47%		50%	8	9	7	16	0	1
findCopyConstructor(Class)		55%		25%	5	7	6	14	0	1
addProxyMethods(ClassWriterTracker_boolean_String_Class)		89%		50%	1	2	7	54	0	1
toProxyableCollectionType(Class)		35%		37%	4	5	5	9	0	1
toProxyableMapType(Class)		35%		37%	4	5	5	9	0	1
newMapProxy(Class_Class_Class_Comparator_boolean)		14%		0%	2	3	3	4	0	1
getFactoryProxyBean(Object)		67%		50%	5	6	3	15	0	1
getFactoryProxyMap(Class)		47%		50%	2	3	4	9	0	1
getFactoryProxyDate(Class)		47%		50%	2	3	4	9	0	1
getFactoryProxyCalendar(Class)		47%		50%	2	3	4	9	0	1
copyArray(Object)		40%		50%	1	2	6	10	0	1
copyCollection(Collection)		17%		25%	2	3	4	6	0	1
copyMap(Map)		17%		25%	2	3	4	6	0	1
copyDate(Date)		17%		25%	2	3	4	6	0	1
copyCalendar(Calendar)		17%		25%	2	3	4	6	0	1
isProxyable(Class)		30%		12%	4	5	5	8	0	1
delegateConstructors(ClassWriterTracker_Class_String)		80%		75%	1	3	2	17	0	1
newCalendarProxy(Class_TimeZone)		15%		25%	2	3	6	7	0	1
generateAndLoadProxyDate(Class_boolean_ClassLoader)		20%		0%	1	2	4	5	0	1
generateAndLoadProxyCalendar(Class_boolean_ClassLoader)		20%		0%	1	2	4	5	0	1
generateAndLoadProxyMap(Class_boolean_ClassLoader)		20%		0%	1	2	4	5	0	1
findComparatorConstructor(Class)		16%		n/a	0	1	3	5	0	1
loadDelayedProxy(Class)		70%		50%	8	9	7	15	0	1
hasConstructor(Class_Class[])		0%		0%	2	2	3	3	1	1
loadBuildTimeProxy(Class_ClassLoader)		60%		75%	1	3	2	8	0	1
newDateProxy(Class)		0%		n/a	1	1	2	2	1	1
toHelperAfterParameters(Class[], Class_Class)		88%		58%	5	7	2	15	0	1

Immagine 87 : Jacoco ProxyManagerImpl Randoop Parte 2

getFactoryProxyDate(Class)		47%		50%	2	3	4	9	0	1	
getFactoryProxyCalendar(Class)		47%		50%	2	3	4	9	0	1	
copyArray(Object)		40%		50%	1	2	6	10	0	1	
copyCollection(Collection)		17%		25%	2	3	4	6	0	1	
copyMap(Map)		17%		25%	2	3	4	6	0	1	
copyDate(Date)		17%		25%	2	3	4	6	0	1	
copyCalendar(Calendar)		17%		25%	2	3	4	6	0	1	
isProxyable(Class)		30%		12%	4	5	5	8	0	1	
delegateConstructors(ClassWriterTracker, Class, String)		80%		75%	1	3	2	17	0	1	
newCalendarProxy(Class, TimeZone)		15%		25%	2	3	6	7	0	1	
generateAndLoadProxyDate(Class, boolean, ClassLoader)		20%		0%	1	2	4	5	0	1	
generateAndLoadProxyCalendar(Class, boolean, ClassLoader)		20%		0%	1	2	4	5	0	1	
generateAndLoadProxyMap(Class, boolean, ClassLoader)		20%		0%	1	2	4	5	0	1	
findComparatorConstructor(Class)		16%		n/a	0	1	3	5	0	1	
loadDelayedProxy(Class)		70%		50%	8	9	7	15	0	1	
hasConstructor(Class, Class[])		0%		0%	2	2	3	3	1	1	
loadBuildTimeProxy(Class, ClassLoader)		60%		75%	1	3	2	8	0	1	
newDateProxy(Class)		0%		n/a	1	1	2	2	1	1	
toHelperAfterParameters(Class[], Class, Class)		88%		58%	5	7	2	15	0	1	
generateAndLoadProxyBean(Class, boolean, ClassLoader)		70%		50%	1	2	1	5	0	1	
getAssertAllowedType()		0%		n/a	1	1	1	1	1	1	
newCollectionProxy(Class, Class, Comparator, boolean)		90%		50%	1	2	0	3	0	1	
generateAndLoadProxyCollection(Class, boolean, ClassLoader)		90%		50%	1	2	1	5	0	1	
isUnproxyable(Class)		89%		66%	2	4	1	5	0	1	
isOrdered(Class)		84%		50%	2	3	0	2	0	1	
proxySetter(ClassWriterTracker, Class, Method)		99%		75%	1	3	1	22	0	1	
generateProxyCollectionBytecode(Class, boolean, String)		98%		50%	1	2	0	17	0	1	
addWriteReplaceMethod(ClassWriterTracker, String, boolean)		98%		50%	1	2	0	12	0	1	
proxySetters(ClassWriterTracker, String, Class)		97%		75%	2	5	0	7	0	1	
allowsDuplicates(Class)		87%		50%	1	2	0	1	0	1	
static {...}		100%		n/a	0	1	0	14	0	1	
getFactoryProxyCollection(Class)		100%		50%	2	3	0	9	0	1	
isSetter(Method)		100%		58%	5	7	0	6	0	1	
ProxyManagerImpl()		100%		n/a	0	1	0	8	0	1	
addInstanceVariables(ClassWriterTracker)		100%		n/a	0	1	0	5	0	1	
toHelperParameters(Class[], Class)		100%		n/a	0	1	0	4	0	1	
startsWith(String, String)		100%		66%	2	4	0	3	0	1	
getProxyClassName(Class, boolean)		100%		100%	0	2	0	3	0	1	
assertNotFinal(Class)		100%		100%	0	2	0	3	0	1	
setUnproxyable(String)		100%		50%	1	2	0	3	0	1	
nextProxyId()		100%		n/a	0	1	0	1	0	1	
setTrackChanges(boolean)		100%		n/a	0	1	0	2	0	1	
setAssertAllowedType(boolean)		100%		n/a	0	1	0	2	0	1	
setDelayCollectionLoading(boolean)		100%		n/a	0	1	0	2	0	1	
getTrackChanges()		100%		n/a	0	1	0	1	0	1	
getDelayCollectionLoading()		100%		n/a	0	1	0	1	0	1	
getUnproxyable()		100%		n/a	0	1	0	1	0	1	
Total		3,151 of 4,896	35%	296 of 414	28%	209	280	671	1,084	16	73

Immagine 88 : Pit test ProxyManagerImpl Randoop

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	66%	34%	49%
ProxyManagerImpl.java	40%	21%	56%

ProxyManagerImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
addProxyMapMethods(ClassWriterTracker,String,Class)		0%		0%	10 10	94 94	1 1
addProxyCollectionMethods(ClassWriterTracker,String,Class)		0%		0%	12 12	84 84	1 1
addProxyCalendarMethods(ClassWriterTracker,String,Class)		0%		0%	2 2	71 71	1 1
proxyBeforeAfterMethod(ClassWriterTracker,Class,Method,Class[],Method,Method,Class[])		0%		0%	11 11	37 37	1 1
proxyRecognizedMethods(ClassWriterTracker,String,Class,Class,Class)		0%		0%	6 6	32 32	1 1
addProxyDateMethods(ClassWriterTracker,String,Class)		69%		50%	11 12	18 67	0 1
generateProxyCollectionBytecode(Class,boolean,String)		0%		0%	2 2	17 17	1 1
generateProxyMapBytecode(Class,boolean,String)		0%		0%	2 2	17 17	1 1
generateProxyCalendarBytecode(Class,boolean,String)		0%		0%	2 2	16 16	1 1
proxyOverrideMethod(ClassWriterTracker,Method,Method,Class[])		0%		0%	2 2	12 12	1 1
findGetter(Class,Method)		32%		8%	6 7	15 20	0 1
toHelperAfterParameters(Class[],Class,Class)		0%		0%	7 7	15 15	1 1
instantiateProxy(Class,Constructor,Object[])		25%		50%	1 3	11 16	0 1
toConcreteType(Class,Map)		0%		0%	4 4	9 9	1 1
main(String[])		85%		65%	7 11	6 50	0 1
toProxyableCollectionType(Class)		35%		37%	4 5	5 9	0 1
toProxyableMapType(Class)		35%		37%	4 5	5 9	0 1
generateAndLoadProxyCalendar(Class,boolean,ClassLoader)		0%		0%	2 2	5 5	1 1
generateAndLoadProxyCollection(Class,boolean,ClassLoader)		0%		0%	2 2	5 5	1 1
generateAndLoadProxyMap(Class,boolean,ClassLoader)		0%		0%	2 2	5 5	1 1
toHelperParameters(Class[],Class)		0%		n/a	1 1	4 4	1 1
findComparatorConstructor(Class)		0%		n/a	1 1	5 5	1 1
newCustomProxy(Object,boolean)		89%		87%	3 13	1 33	0 1
copyCustom(Object)		77%		75%	4 9	3 16	0 1
findCopyConstructor(Class)		76%		58%	3 7	3 14	0 1
isOrdered(Class)		0%		0%	3 3	2 2	1 1
loadDelayedProxy(Class)		75%		56%	7 9	6 15	0 1
isProxyable(Class)		61%		50%	3 5	3 8	0 1
generateProxyBeanBytecode(Class,boolean,String)		93%		66%	6 10	4 28	0 1
loadBuildTimeProxy(Class,ClassLoader)		65%		25%	2 3	3 8	0 1
assertNotFinal(Class)		38%		50%	1 2	1 3	0 1
allowsDuplicates(Class)		0%		0%	2 2	1 1	1 1
getFactoryProxyBean(Object)		89%		70%	3 6	1 15	0 1
getFactoryProxyMap(Class)		85%		50%	2 3	1 9	0 1
getFactoryProxyCalendar(Class)		85%		50%	2 3	1 9	0 1
getFactoryProxyCollection(Class)		85%		50%	2 3	1 9	0 1
copyBeanProperties(MethodVisitor,Class,Int)		96%		66%	4 7	3 18	0 1
generateAndLoadProxyDate(Class,boolean,ClassLoader)		90%		50%	1 2	1 5	0 1
generateAndLoadProxyBean(Class,boolean,ClassLoader)		90%		50%	1 2	1 5	0 1
proxySetter(ClassWriterTracker,Class,Method)		99%		75%	1 3	1 22	0 1
generateProxyDateBytecode(Class,boolean,String)		98%		50%	1 2	0 16	0 1
proxySetters(ClassWriterTracker,String,Class)		97%		75%	2 5	0 7	0 1
hasConstructor(Class,Class[])		90%		50%	1 2	0 3	0 1
addProxyMethods(ClassWriterTracker,boolean,String,Class)		100%		50%	1 2	0 54	0 1
addProxyBeanMethods(ClassWriterTracker,String,Class,Constructor)		100%		100%	0 5	0 39	0 1

Imagine 90 : Jacoco ProxyManagerImpl LLM suite Parte 2

loadDelayedProxy(Class)		75%		56%	7	9	6	15	0	1	
isProxyable(Class)		61%		50%	3	5	3	8	0	1	
generateProxyBeanBytecode(Class, boolean, String)		93%		66%	6	10	4	28	0	1	
loadBuildTimeProxy(Class, ClassLoader)		65%		25%	2	3	3	8	0	1	
assertNotFinal(Class)		38%		50%	1	2	1	3	0	1	
allowsDuplicates(Class)		0%		0%	2	2	1	1	1	1	
getFactoryProxyBean(Object)		89%		70%	3	6	1	15	0	1	
getFactoryProxyMap(Class)		85%		50%	2	3	1	9	0	1	
getFactoryProxyCalendar(Class)		85%		50%	2	3	1	9	0	1	
getFactoryProxyCollection(Class)		85%		50%	2	3	1	9	0	1	
copyBeanProperties(MethodVisitor, Class, int)		96%		66%	4	7	3	18	0	1	
generateAndLoadProxyDate(Class, boolean, ClassLoader)		90%		50%	1	2	1	5	0	1	
generateAndLoadProxyBean(Class, boolean, ClassLoader)		90%		50%	1	2	1	5	0	1	
proxySetter(ClassWriterTracker, Class, Method)		99%		75%	1	3	1	22	0	1	
generateProxyDateBytecode(Class, boolean, String)		98%		50%	1	2	0	16	0	1	
proxySetters(ClassWriterTracker, String, Class)		97%		75%	2	5	0	7	0	1	
hasConstructor(Class, Class[])		90%		50%	1	2	0	3	0	1	
addProxyMethods(ClassWriterTracker, boolean, String, Class)		100%		50%	1	2	0	54	0	1	
addProxyBeanMethods(ClassWriterTracker, String, Class, Constructor)		100%		100%	0	5	0	39	0	1	
delegateConstructors(ClassWriterTracker, Class, String)		100%		100%	0	3	0	17	0	1	
addWriteReplaceMethod(ClassWriterTracker, String, boolean)		100%		100%	0	2	0	12	0	1	
static {...}		100%		n/a	0	1	0	14	0	1	
getFactoryProxyDate(Class)		100%		75%	1	3	0	9	0	1	
isSetter(Method)		100%		58%	5	7	0	6	0	1	
copyArray(Object)		100%		100%	0	2	0	10	0	1	
ProxyManagerImpl()		100%		n/a	0	1	0	8	0	1	
newMapProxy(Class, Class, Class, Comparator, boolean)		100%		100%	0	3	0	4	0	1	
copyCollection(Collection)		100%		100%	0	3	0	6	0	1	
copyMap(Map)		100%		100%	0	3	0	6	0	1	
copyDate(Date)		100%		100%	0	3	0	6	0	1	
copyCalendar(Calendar)		100%		100%	0	3	0	6	0	1	
newCollectionProxy(Class, Class, Comparator, boolean)		100%		100%	0	2	0	3	0	1	
addInstanceVariables(ClassWriterTracker)		100%		n/a	0	1	0	5	0	1	
newCalendarProxy(Class, TimeZone)		100%		100%	0	3	0	7	0	1	
isUnproxyable(Class)		100%		83%	1	4	0	5	0	1	
startsWith(String, String)		100%		66%	2	4	0	3	0	1	
getProxyClassName(Class, boolean)		100%		100%	0	2	0	3	0	1	
setUnproxyable(String)		100%		50%	1	2	0	3	0	1	
newDateProxy(Class)		100%		n/a	0	1	0	2	0	1	
nextProxyId()		100%		n/a	0	1	0	1	0	1	
setTrackChanges(boolean)		100%		n/a	0	1	0	2	0	1	
setAssertAllowedType(boolean)		100%		n/a	0	1	0	2	0	1	
setDelayCollectionLoading(boolean)		100%		n/a	0	1	0	2	0	1	
getTrackChanges()		100%		n/a	0	1	0	1	0	1	
getAssertAllowedType()		100%		n/a	0	1	0	1	0	1	
getDelayCollectionLoading()		100%		n/a	0	1	0	1	0	1	
getUnproxyable()		100%		n/a	0	1	0	1	0	1	
Total		2,445 of 4,896	50%	215 of 414	48%	166	280	525	1,084	18	73

Imagine 91 : Pit Test ProxyManagerImpl LLM suite

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	97%	83%	84%
ProxyManagerImpl.java	89%	69%	75%

ProxyManagerImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
addProxyDateMethods(ClassWriterTracker_String_Class)		71%		63%	8	12	17	67	0	1
addProxyCollectionMethods(ClassWriterTracker_String_Class)		87%		40%	11	12	8	84	0	1
findGetter(Class_Method)		32%		8%	6	7	15	20	0	1
instantiateProxy(Class_Constructor_Object[])		25%		50%	1	3	11	16	0	1
main(String[])		85%		65%	7	11	6	50	0	1
toConcreteType(Class_Map)		23%		16%	3	4	6	9	0	1
toProxyableMapType(Class)		35%		37%	4	5	5	9	0	1
toProxyableCollectionType(Class)		50%		62%	3	5	3	9	0	1
proxyRecognizedMethods(ClassWriterTracker_String_Class_Class_Class)		83%		90%	1	6	7	32	0	1
findCopyConstructor(Class)		76%		58%	3	7	3	14	0	1
isProxyable(Class)		61%		50%	3	5	3	8	0	1
addProxyMapMethods(ClassWriterTracker_String_Class)		83%		83%	3	10	1	94	0	1
findComparatorConstructor(Class)		66%		n/a	0	1	2	5	0	1
copyBeanProperties(MethodVisitor_Class_Int)		96%		66%	4	7	3	18	0	1
addProxyCalendarMethods(ClassWriterTracker_String_Class)		99%		50%	1	2	0	71	0	1
generateProxyBeanBytecode(Class_boolean_String)		98%		94%	1	10	1	28	0	1
generateAndLoadProxyDate(Class_boolean_ClassLoader)		90%		50%	1	2	1	5	0	1
generateAndLoadProxyCalendar(Class_boolean_ClassLoader)		90%		50%	1	2	1	5	0	1
generateAndLoadProxyCollection(Class_boolean_ClassLoader)		90%		50%	1	2	1	5	0	1
generateAndLoadProxyMap(Class_boolean_ClassLoader)		90%		50%	1	2	1	5	0	1
isOrdered(Class)		84%		50%	2	3	0	2	0	1
generateProxyCollectionBytecode(Class_boolean_String)		98%		50%	1	2	0	17	0	1
generateProxyMapBytecode(Class_boolean_String)		98%		50%	1	2	0	17	0	1
generateProxyDateBytecode(Class_boolean_String)		98%		50%	1	2	0	16	0	1
generateProxyCalendarBytecode(Class_boolean_String)		98%		50%	1	2	0	16	0	1
hasConstructor(Class_Class[])		90%		50%	1	2	0	3	0	1
allowsDuplicates(Class)		87%		50%	1	2	0	1	0	1
addProxyMethods(ClassWriterTracker_boolean_String_Class)		100%		100%	0	2	0	54	0	1
proxyBeforeAfterMethod(ClassWriterTracker_Class_Method_Class[], Method_Method_Class[])		100%		85%	3	11	0	37	0	1
addProxyBeanMethods(ClassWriterTracker_String_Class_Constructor)		100%		100%	0	5	0	39	0	1
newCustomProxy(Object_boolean)		100%		100%	0	13	0	33	0	1
proxySetter(ClassWriterTracker_Class_Method)		100%		100%	0	3	0	22	0	1
delegateConstructors(ClassWriterTracker_Class_String)		100%		100%	0	3	0	17	0	1
getFactoryProxyBean(Object)		100%		90%	1	6	0	15	0	1
copyCustom(Object)		100%		100%	0	9	0	16	0	1
addWriteReplaceMethod(ClassWriterTracker_String_boolean)		100%		100%	0	2	0	12	0	1
proxyOverrideMethod(ClassWriterTracker_Method_Method_Class[])		100%		100%	0	2	0	12	0	1
static {...}		100%		n/a	0	1	0	14	0	1
toHelperAfterParameters(Class[], Class_Class)		100%		75%	3	7	0	15	0	1
loadDelayedProxy(Class)		100%		93%	1	9	0	15	0	1
proxySetters(ClassWriterTracker_String_Class)		100%		100%	0	5	0	7	0	1
getFactoryProxyMap(Class)		100%		75%	1	3	0	9	0	1
getFactoryProxyDate(Class)		100%		75%	1	3	0	9	0	1
getFactoryProxyCalendar(Class)		100%		75%	1	3	0	9	0	1
getFactoryProxyCollection(Class)		100%		75%	1	3	0	9	0	1

Immagine 93 : Jacoco ProxyManagerImpl TestSuite Completa Parte 2

allowsDuplicates(Class)		87%		50%	1	2	0	1	0	1
addProxyMethods(ClassWriterTracker, boolean, String, Class)		100%		100%	0	2	0	54	0	1
proxyBeforeAfterMethod(ClassWriterTracker, Class, Method, Class[], Method, Method, Class[])		100%		85%	3	11	0	37	0	1
addProxyBeanMethods(ClassWriterTracker, String, Class, Constructor)		100%		100%	0	5	0	39	0	1
newCustomProxy(Object, boolean)		100%		100%	0	13	0	33	0	1
proxySetter(ClassWriterTracker, Class, Method)		100%		100%	0	3	0	22	0	1
delegateConstructors(ClassWriterTracker, Class, String)		100%		100%	0	3	0	17	0	1
getFactoryProxyBean(Object)		100%		90%	1	6	0	15	0	1
copyCustom(Object)		100%		100%	0	9	0	16	0	1
addWriteReplaceMethod(ClassWriterTracker, String, boolean)		100%		100%	0	2	0	12	0	1
proxyOverrideMethod(ClassWriterTracker, Method, Method, Class[])		100%		100%	0	2	0	12	0	1
static {...}		100%		n/a	0	1	0	14	0	1
toHelperAfterParameters(Class[], Class, Class)		100%		75%	3	7	0	15	0	1
loadDelayedProxy(Class)		100%		93%	1	9	0	15	0	1
proxySetters(ClassWriterTracker, String, Class)		100%		100%	0	5	0	7	0	1
getFactoryProxyMap(Class)		100%		75%	1	3	0	9	0	1
getFactoryProxyDate(Class)		100%		75%	1	3	0	9	0	1
getFactoryProxyCalendar(Class)		100%		75%	1	3	0	9	0	1
getFactoryProxyCollection(Class)		100%		75%	1	3	0	9	0	1
isSetter(Method)		100%		91%	1	7	0	6	0	1
copyArray(Object)		100%		100%	0	2	0	10	0	1
ProxyManagerImpl()		100%		n/a	0	1	0	8	0	1
newMapProxy(Class, Class, Class, Comparator, boolean)		100%		100%	0	3	0	4	0	1
copyCollection(Collection)		100%		100%	0	3	0	6	0	1
copyMap(Map)		100%		100%	0	3	0	6	0	1
copyDate(Date)		100%		100%	0	3	0	6	0	1
copyCalendar(Calendar)		100%		100%	0	3	0	6	0	1
loadBuildTimeProxy(Class, ClassLoader)		100%		100%	0	3	0	8	0	1
newCollectionProxy(Class, Class, Comparator, boolean)		100%		100%	0	2	0	3	0	1
addInstanceVariables(ClassWriterTracker)		100%		n/a	0	1	0	5	0	1
newCalendarProxy(Class, TimeZone)		100%		100%	0	3	0	7	0	1
generateAndLoadProxyBean(Class, boolean, ClassLoader)		100%		100%	0	2	0	5	0	1
isUnproxyable(Class)		100%		83%	1	4	0	5	0	1
toHelperParameters(Class[], Class)		100%		n/a	0	1	0	4	0	1
startsWith(String, String)		100%		83%	1	4	0	3	0	1
getProxyClassName(Class, boolean)		100%		100%	0	2	0	3	0	1
assertNotFinal(Class)		100%		100%	0	2	0	3	0	1
setUnproxyable(String)		100%		100%	0	2	0	3	0	1
newDateProxy(Class)		100%		n/a	0	1	0	2	0	1
nextProxyId()		100%		n/a	0	1	0	1	0	1
setTrackChanges(boolean)		100%		n/a	0	1	0	2	0	1
setAssertAllowedType(boolean)		100%		n/a	0	1	0	2	0	1
setDelayCollectionLoading(boolean)		100%		n/a	0	1	0	2	0	1
getTrackChanges()		100%		n/a	0	1	0	1	0	1
getAssertAllowedType()		100%		n/a	0	1	0	1	0	1
getDelayCollectionLoading()		100%		n/a	0	1	0	1	0	1
getUnproxyable()		100%		n/a	0	1	0	1	0	1
Total	429 of 4,896	91%	100 of 414	75%	86	280	95	1,084	0	73

Immagine 94 : PitTest ProxymanagerImpl Test Suite Completa

Name	Line Coverage	Mutation Coverage	Test Strength
CacheMap.java	97% 176/182	83% 111/133	84% 111/132
ProxyManagerImpl.java	89% 963/1084	69% 474/684	75% 474/635

4.3 Prompt LLM

Prompt LLM 1 : Guided ToT BufferedChannel Gemini PRO

Imagine three different experts in software testing who are tasked with developing comprehensive JUnit 4 test cases for the following Java class named BufferedChannel.

To reach a target of 100 distinct test cases, they must follow these steps:

1. ****Analysis****: Extract and list all public methods.
2. ****Brainstorming****: Given the source code, brainstorm potential usage scenarios. This must include:
 - Happy paths (standard usage).
 - Boundary values (empty buffer, full buffer, size 1, max int size).
 - Exception handling (null inputs, closed channel interactions).
 - Concurrency scenarios (multiple producers/consumers, thread interruption, blocking behavior).
3. ****Expansion****: Explode parameters to cover all combinations (Data-Driven approach).
4. ****Validation****: The experts review the list to ensure there are exactly 100 unique, meaningful scenarios. If duplicates exist, they replace them with new edge cases.

The Java class is:

{SOURCE_CODE}

Output Request:

Do NOT write the Java code yet.

Simply output the final ****numbered list of 100 test case descriptions**** (titles/summaries) that the experts have agreed upon.

Analysis of BufferedChannel Test Scenarios

The BufferedChannel class acts as a high-performance buffering layer over a Java NIO FileChannel. It manages a write buffer (accumulating writes before flushing to disk) and a read buffer (caching reads). It also handles data durability via "unpersisted bytes" tracking and forced writes.

To achieve 100 distinct test cases, we apply a data-driven approach, systematically covering constructors, state mutations (write/flush), complex read logic (interacting buffers), and edge cases (concurrency, IO errors).

Comprehensive Test Case List

I. Construction & Initialization (1-10)

1. **Valid Construction (Default)**: Verify instantiation with valid ByteBufAllocator, FileChannel, and capacity.
2. **Valid Construction (With Bound)**: Verify instantiation with explicit unpersistedBytesBound.
3. **Null Allocator**: Verify NullPointerException or behavior when ByteBufAllocator is null.
4. **Null FileChannel**: Verify behavior when FileChannel is null.
5. **Zero Capacity**: Verify IllegalArgumentException or behavior for capacity = 0.
6. **Negative Capacity**: Verify IllegalArgumentException for negative capacity.
7. **Negative UnpersistedBound**: Verify behavior/exception for negative unpersistedBytesBound.
8. **Initial State Verification**: Assert position(), writeBufferStartPosition, and unpersistedBytes are 0 initially.
9. **Initial Buffer State**: Verify writeBuffer is allocated but empty immediately after construction.
10. **File Position Sync**: Verify writeBufferStartPosition matches the underlying FileChannel position if the channel was not at 0.

II. Basic Write Operations (11-25)

11. **Write Single Byte**: Write a buffer with 1 byte (capacity > 1).
12. **Write Small Chunk**: Write a buffer significantly smaller than capacity.
13. **Write Exact Capacity**: Write a buffer that exactly fills the writeBuffer.
14. **Write Overflow (Trigger Flush)**: Write a buffer of capacity + 1 bytes to trigger an automatic flush.
15. **Write Zero Length**: Write a ByteBuf with 0 readable bytes (no-op).
16. **Write Null**: Verify NullPointerException when writing null.
17. **Multiple Small Writes**: Perform sequential small writes that do not fill the buffer.
18. **Accumulation to Capacity**: Perform sequential writes that eventually exactly fill the buffer.
19. **Accumulation to Overflow**: Perform sequential writes that eventually spill over capacity.
20. **Large Write (Multi-Flush)**: Write a buffer sized 3.5x capacity, verifying multiple flushes occur.
21. **Write with Offset**: Write a ByteBuf where readerIndex > 0.
22. **Write Composite Buffer**: Write a Netty CompositeByteBuf to verify handling of complex buffer types.
23. **Position Update**: Verify position() increments correctly after every write.
24. **FileChannel Position Lag**: Verify getFileChannelPosition() does *not* change during buffering (before flush).
25. **Write Buffer Reusability**: Verify writeBuffer accepts new data correctly after an automatic flush.

III. Unpersisted Bytes & Force Logic (26-40)

26. **Unpersisted Bytes Tracking**: Verify getUnpersistedBytes() increases with writes.
27. **Bound Not Reached**: Verify no force() is called when writes are below unpersistedBytesBound.
28. **Bound Reached (Exact)**: Verify force() is triggered when writes exactly hit unpersistedBytesBound.
29. **Bound Exceeded**: Verify force() is triggered when writes exceed unpersistedBytesBound.
30. **Unpersisted Reset (Auto-Force)**: Verify unpersistedBytes resets to 0 after an automatic force write.
31. **Unpersisted Retention (Flush)**: Verify unpersistedBytes is *not* reset by a standard flush() (only by force).
32. **Disabled Regular Flushes**: Test constructor with unpersistedBytesBound = 0 (feature disabled).
33. **ForceWrite Method (Metadata True)**: Verify forceWrite(true) calls FileChannel.force(true).
34. **ForceWrite Method (Metadata False)**: Verify forceWrite(false) calls FileChannel.force(false).
35. **FlushAndForceWrite**: Verify this calls flush() followed by forceWrite().
36. **FlushAndForceWriteIfRegularFlush (Enabled)**: Verify it acts like FlushAndForceWrite when bound > 0.
37. **FlushAndForceWriteIfRegularFlush (Disabled)**: Verify it is a no-op when bound = 0.
38. **Unpersisted Calculation Consistency**: Verify calculation accounts for writeBuffer readable bytes inside forceWrite.
39. **Accumulated Writes Trigger Force**: Multiple small writes accumulating to hit the bound.
40. **ForceWrite Return Value**: Verify forceWrite returns the correct writeBufferStartPosition.

IV. Explicit Flushing (41-50)

41. **Manual Flush**: Verify flush() moves bytes from writeBuffer to FileChannel.
42. **Empty Flush**: Verify flush() on an empty buffer is safe (no-op).
43. **Flush Updates Positions**: Verify writeBufferStartPosition updates to current file position after flush.
44. **Flush Clears Buffer**: Verify writeBuffer is empty (cleared) after flush.
45. **Flush Propagates IOErrors**: Mock FileChannel.write to throw exception during flush.

46. **Partial Flush Loop:** Mock FileChannel.write to write partial bytes, verifying flush loops until buffer is empty.
47. **Position Consistency:** Verify position() (absolute) matches fileChannel.position() after flush.
48. **Flush after Large Write:** Verify manual flush after a write that already triggered auto-flushes.
49. **FlushAndForce Metadata Propagation:** Verify the forceMetadata boolean is passed correctly down the chain.
50. **Flush Concurrency:** (Theoretical) Verify behavior if flush is called while buffer is empty.

V. Reading - Hot Data (Write Buffer) (51-60)

51. **Read from WriteBuffer (Full):** Read data that sits entirely in the writeBuffer (has not been flushed).
52. **Read from WriteBuffer (Partial):** Read a slice of data from the writeBuffer.
53. **Read from WriteBuffer (Offset):** Read from writeBuffer but with a pos offset.
54. **Read Exact WriteBuffer Size:** Read exactly the amount of data currently in writeBuffer.
55. **Read "Past" WriteBuffer:** Attempt to read beyond the data available in writeBuffer (expect EOF/Short read).
56. **Read Hot Data After Append:** Write A, Write B, Read B (verify access to most recent append).
57. **Read Hot Data After Seek:** Use specific pos to read earlier data still in writeBuffer.
58. **Read Buffer State Preservation:** Verify reading from writeBuffer does not modify writeBuffer's writer index.
59. **Dest Buffer Index:** Verify read updates the writerIndex of the destination ByteBuf.
60. **Dest Buffer Capacity:** Verify IllegalArgumentException if dest has insufficient writable bytes.

VI. Reading - Cold/Cached Data (File & Read Buffer) (61-75)

61. **Read from File (Flushed):** Write, Flush, Read (verify data comes from file/read cache).
62. **Read triggers ReadBuffer Fill:** Verify a read from file populates the internal readBuffer.
63. **Read from ReadBuffer (Hit):** Second read of the same area should hit the readBuffer (cache hit).
64. **Read from ReadBuffer (Partial Hit):** Read a subset of what is in readBuffer.
65. **Read Spanning File/WriteBuffer:** Read range that starts in flushed file and ends in writeBuffer.
66. **Read Spanning ReadBuffer/WriteBuffer:** Read range that starts in readBuffer and ends in writeBuffer.
67. **Read Large (Bypass ReadBuffer):** Read a chunk larger than readCapacity (verify handling).
68. **Read Sequence:** Write A, Flush, Write B. Read A (File), Read B (Buffer).
69. **Random Access Read:** Read end of file, then read beginning of file.
70. **Read Zero Bytes:** read with length 0.
71. **Read Negative Position:** Call read with negative pos (verify Exception).
72. **Read Past EOF (Empty File):** Read from empty channel.
73. **Read Past EOF (Non-Empty):** Read beyond the last written byte.
74. **FileChannel Short Read:** Mock FileChannel.read returning fewer bytes than requested.
75. **FileChannel EOF:** Mock FileChannel.read returning -1 unexpectedly.

VII. Complex Read Logic (76-85)

76. **Loop Iteration Logic:** Read a range requiring 3 steps: File -> ReadBuffer -> WriteBuffer (Construct scenario).
77. **WriteBufferStart Logic:** Verify behavior when writeBufferStartPosition < pos vs > pos.
78. **Read Buffer Invalidation:** Verify readBuffer is refreshed if read requests data outside current readBuffer range.
79. **Destination Buffer Growth:** (If applicable) or verify strict capacity check on destination.
80. **Read into Composite Buffer:** Verify reading into a Netty CompositeByteBuf.
81. **Read Does Not Alter Source:** Verify the BufferedChannel state is not corrupted by a failed read.
82. **Read Logic with Null WriteBuffer:** (Edge case) if class allows read after clear/close (or before init).
83. **Overlap Calculation:** precise test for Math.min logic inside the read loop.
84. **Read Buffer Alignment:** Verify reading works when pos is not aligned with readBufferStartPosition.
85. **Concurrent Read/Write Simulation:** (Sequential check) Write, Read, Write, Read to verify strict ordering.

VIII. Lifecycle (Clear & Close) (86-92)

86. **Clear Resets State:** Verify clear() resets writeBuffer indices.
87. **Clear Keeps Capacity:** Verify clear() does not deallocate memory, just resets indices.
88. **Close Releases Resource:** Verify close() releases the writeBuffer (ref count decrement).
89. **Close Closes FileChannel:** Verify underlying FileChannel.close() is called.
90. **Idempotent Close:** Verify calling close() twice does not throw exceptions.
91. **Operations After Close:** Verify write() throws IOException or similar after close.
92. **Getters After Close:** Verify behavior of position() after close.

IX. Exceptions & Edge Cases (93-100)

93. **Allocator OOM:** Mock ByteBufAllocator to throw OutOfMemoryError during construction.
94. **Disk Full on Flush:** Mock FileChannel.write to throw IOException (disk full).
95. **Disk Error on Force:** Mock FileChannel.force to throw IOException.
96. **Recoverable Error:** If flush fails, verify state allows retry (or fails safe).
97. **Int Overflow Position:** Test position behavior near Long.MAX_VALUE (theoretical boundary).
98. **System Clock Change:** (If time-based logic existed—none here, but standard check).
99. **Zero-Byte FileChannel:** Initialize with a FileChannel that has existing data, verify position init.
100. **Reference Counting:** Verify writeBuffer reference count is 1 after init and 0 after close

Great. Now, acting as the implementation expert, generate the JUnit 4 code for ****scenarios #{number_start} to #{number_end}**** from the list above.

- Use the class name BufferedChannelPart{RomanNumber}Test.java.
- Include the necessary setup (@Before) and teardown (@After).
- Start the output with **###Test Part {Part Number} START##** and end with **###Test Part {Part Number} END##**.
- Ensure every test method has a meaningful assertion.

Prompt LLM 2 : Zero-shot JournalChannel (Gemini PRO)

As a professional software tester who writes junit4 test methods, generate a complete java test file (JournalChannelTest.java) to comprehensively test all methods with 40 tests in the following class named JournalChannel in apache Bookkeeper project.

Your output file must start with `###Test START##` and finish with `###Test END##`.

Here is the source code:

{Codice di JournalChannel}

Prompt LLM 3 : Zero-shot CacheMap (Gemini PRO)

As a professional software tester who writes JUnit 4 test methods, generate a complete JUnit 4 test file (CacheMapTest.java) to comprehensively test all methods in the following class named CacheMap in Apache OpenJPA.

Your output file must start with `###Test START##` and finish with `###Test END##`.

Here is the source code:

(Codice sorgente della classe CacheMap)

Prompt LLM 4 : Few-Shot ProxyManagerImpl (Gemini PRO)

As a professional software tester who writes Java test methods, consider the following examples to understand the required testing style and utility usage:

`### REFERENCE UTILITY CLASS ###`

{la classe ProxyManagerImplTestUtility.java }

`### EXISTING TEST EXAMPLE ###`

{la classe ProxyManagerImplStrumentazioneTest.java}

Now, generate exactly 70 JUnit 4 test cases to comprehensively test all methods in the following class named ProxyManagerImpl of apache OpenJPA project.

Instructions:

1. Use the provided utility class or implement analogous helper methods to reach the target of 70 tests.
2. Focus on covering all public methods, including diverse input permutations, state transitions, and edge cases.
3. Ensure every test case has a clear, meaningful assertion.
4. Your output is generate a numbered list of 70 test scenarios for ProxyManagerImpl (titles only).

Here is the source code of the target class:

{la classe ProxyManagerImpl.java }

Here is the list of 70 JUnit 4 test case scenarios for ProxyManagerImpl. These tests cover configuration, array handling, collection/map/date/calendar proxying, custom bean proxying (with bytecode generation), and edge cases.

1. **testDefaultConfiguration_ShouldHaveTrackChangesTrue**: Verify that a new instance has trackChanges=true by default.
2. **testSetTrackChanges_ShouldUpdateConfiguration**: Verify setTrackChanges toggles the boolean flag correctly.
3. **testSetAssertAllowedType_ShouldUpdateConfiguration**: Verify setAssertAllowedType toggles the boolean flag correctly.
4. **testSetDelayCollectionLoading_ShouldUpdateConfiguration**: Verify setDelayCollectionLoading toggles the boolean flag correctly.
5. **testGetUnproxyable_ShouldReturnDefaultTimeZone**: Verify the default unproxyable list contains java.util.TimeZone.
6. **testSetUnproxyable_WithSemicolonSeparatedString_ShouldAddClasses**: Verify parsing of a string like "java.lang.Integer;java.lang.Double" adds them to the set.
7. **testIsUnproxyable_WithConfiguredClass_ShouldReturnTrue**: Verify a class added via setUnproxyable returns true when checked internally (via newCustomProxy).
8. **testCopyArray_WithNullInput_ShouldReturnNull**: Verify copyArray(null) returns null.
9. **testCopyArray_WithPrimitiveIntArray_ShouldReturnCopy**: Verify copying int[]{1, 2} returns a new array with same content.
10. **testCopyArray_WithObjectStringArray_ShouldReturnCopy**: Verify copying String[]{"A", "B"} returns a new array with same content.
11. **testCopyArray_WithNotArrayObject_ShouldThrowUnsupportedException**: Verify passing a standard Object to copyArray throws exception.
12. **testCopyCollection_WithNullInput_ShouldReturnNull**: Verify copyCollection(null) returns null.

13. **testCopyCollection_WithArrayList_ShouldReturnProxyCollection:** Verify copying an ArrayList returns a proxy implementing List.
14. **testCopyCollection_WithHashSet_ShouldReturnProxyCollection:** Verify copying a HashSet returns a proxy implementing Set.
15. **testCopyCollection_WithTreeSet_ShouldPreserveComparator:** Verify copying a TreeSet preserves the custom Comparator.
16. **testCopyCollection_WithLinkedList_ShouldReturnProxyQueue:** Verify copying a LinkedList returns a proxy implementing Queue and List.
17. **testCopyCollection_WithVector_ShouldReturnProxyList:** Verify copying a Vector returns a valid proxy.
18. **testCopyCollection_WithAlreadyProxiedCollection_ShouldReturnCopy:** Verify passing an existing Proxy to copyCollection returns a *new* copy, not the same instance.
19. **testCopyCollection_WithEmptyCollection_ShouldReturnEmptyProxy:** Verify copying an empty list results in an empty proxy.
20. **testNewCollectionProxy_ArrayList_ShouldCreateEmptyProxy:** Verify newCollectionProxy creates an empty instance of the correct type.
21. **testNewCollectionProxy_TreeSet_WithComparator_ShouldInjectComparator:** Verify newCollectionProxy with a Comparator applies it to the new instance.
22. **testNewCollectionProxy_WithAssertAllowedTypeTrue_ShouldSetElementType:** Enable assertAllowedType and verify the proxy knows its allowed element type.
23. **testNewCollectionProxy_WithAutoOff_ShouldInitializeTracker:** Verify the autoOff parameter behavior on the created proxy's change tracker.
24. **testCopyMap_WithNullInput_ShouldReturnNull:** Verify copyMap(null) returns null.
25. **testCopyMap_WithHashMap_ShouldReturnProxyMap:** Verify copying a HashMap returns a proxy implementing Map.
26. **testCopyMap_WithTreeMap_ShouldPreserveComparator:** Verify copying a TreeMap preserves the custom Comparator.
27. **testCopyMap_WithLinkedHashMap_ShouldReturnProxyMap:** Verify LinkedHashMap is proxied correctly to maintain insertion order.
28. **testCopyMap_WithProperties_ShouldReturnProxyMap:** Verify java.util.Properties (Hashtable subclass) is copied correctly.
29. **testCopyMap_WithAlreadyProxiedMap_ShouldReturnCopy:** Verify passing an existing Proxy to copyMap returns a new deep copy.
30. **testNewMapProxy_HashMap_ShouldCreateEmptyProxy:** Verify newMapProxy creates an empty instance.
31. **testNewMapProxy_WithAssertAllowedTypeTrue_ShouldSetKeyAndValueTypes:** Verify type checking is enabled on the proxy when configured.
32. **testCopyDate_WithNullInput_ShouldReturnNull:** Verify copyDate(null) returns null.
33. **testCopyDate_WithUtilDate_ShouldReturnProxyDate:** Verify java.util.Date is proxied and time is preserved.
34. **testCopyDate_WithSqlDate_ShouldReturnProxyDate:** Verify java.sql.Date is proxied correctly.
35. **testCopyDate_WithSqlTimestamp_ShouldPreserveNanos:** Verify java.sql.Timestamp is copied and nanoseconds are intact.
36. **testCopyDate_WithSqlTime_ShouldReturnProxyDate:** Verify java.sql.Time is proxied correctly.
37. **testCopyDate_WithAlreadyProxiedDate_ShouldReturnCopy:** Verify passing a ProxyDate returns a new copy.
38. **testNewDateProxy_WithUtilDate_ShouldReturnCurrentTime:** Verify newDateProxy creates a date initialized to current time (default constructor behavior).
39. **testCopyCalendar_WithNullInput_ShouldReturnNull:** Verify copyCalendar(null) returns null.
40. **testCopyCalendar_WithGregorianCalendar_ShouldReturnProxyCalendar:** Verify GregorianCalendar is proxied.
41. **testCopyCalendar_WithTimeZone_ShouldPreserveZone:** Verify the TimeZone is copied from the original Calendar.
42. **testCopyCalendar_WithAlreadyProxiedCalendar_ShouldReturnCopy:** Verify passing a ProxyCalendar returns a new copy.
43. **testNewCalendarProxy_WithSpecificZone_ShouldSetZone:** Verify newCalendarProxy accepts a TimeZone argument.
44. **testNewCalendarProxy_AbstractCalendar_ShouldDefaultToGregorian:** Verify requesting a proxy for java.util.Calendar (abstract) instantiates a GregorianCalendar proxy.
45. **testCopyCustom_WithNullInput_ShouldReturnNull:** Verify copyCustom(null) returns null.
46. **testCopyCustom_WithStandardBean_ShouldReturnProxyBean:** Verify a standard POJO is proxied and data copied.
47. **testCopyCustom_WithCopyConstructor_ShouldUseConstructor:** Verify a Bean with a copy constructor utilizes it during the copy process.
48. **testCopyCustom_WithCloneable_ShouldUseClone:** Verify a Bean implementing Cloneable uses the clone strategy (if supported by logic).
49. **testCopyCustom_WithCollection_ShouldDelegateToCopyCollection:** Verify calling copyCustom with an ArrayList delegates to copyCollection.
50. **testCopyCustom_WithMap_ShouldDelegateToCopyMap:** Verify calling copyCustom with a HashMap delegates to copyMap.
51. **testCopyCustom_WithDate_ShouldDelegateToCopyDate:** Verify calling copyCustom with a Date delegates to copyDate.
52. **testNewCustomProxy_WithNullInput_ShouldReturnNull:** Verify newCustomProxy(null) returns null.
53. **testNewCustomProxy_WithFinalClass_ShouldReturnNull:** Verify passing a final class (e.g., String) returns null (cannot subclass).
54. **testNewCustomProxy_WithManagedObject_ShouldReturnNull:** Verify passing an object that is already a PersistenceCapable entity returns null.
55. **testNewCustomProxy_WithAlreadyProxiedObject_ShouldReturnSameInstance:** Verify idempotency; passing a proxy returns the same proxy.
56. **testNewCustomProxy_WithCollection_ShouldDelegateToNewCollectionProxy:** Verify newCustomProxy on an ArrayList creates a Collection proxy and copies data.
57. **testNewCustomProxy_WithMap_ShouldDelegateToNewMapProxy:** Verify newCustomProxy on a HashMap creates a Map proxy and copies data.
58. **testNewCustomProxy_WithDate_ShouldDelegateToNewDateProxy:** Verify newCustomProxy on a Date creates a Date proxy.
59. **testNewCustomProxy_WithCalendar_ShouldDelegateToNewCalendarProxy:** Verify newCustomProxy on a Calendar creates a Calendar proxy.

60. **testBytecodeGen_BeanWithNoDefaultConstructor_ShouldFindAlternative**: Verify proxy generation for a bean that only has a parameterized constructor (OpenJPA attempts to find one).
61. **testBytecodeGen_BeanWithProtectedMethods_ShouldProxyCorrectly**: Verify that protected setters/getters are handled or ignored gracefully.
62. **testBytecodeGen_BeanWithFinalSetters_ShouldNotBreak**: Verify that final setters in a bean do not cause generation failure (though they won't be dirty-tracked).
63. **testBytecodeGen_Serialization_WriteReplace_ShouldBeGenerated**: Verify the generated proxy class contains the writeReplace method for serialization support.
64. **testProxyBean_GetChangeTracker_ShouldReturnNullByDefault**: Verify that for a standard custom proxy, getChangeTracker returns null if not explicitly set up for tracking.
65. **testProxyBean_Copy_ShouldCopyProperties**: Verify that the generated .copy() method on the proxy accurately transfers field values using getters/setters.
66. **testProxyBean_SetOwner_ShouldStoreStateManager**: Verify the setOwner method on the generated proxy correctly stores the OpenJPASStateManager reference.
67. **testMainMethod_WithValidClassArguments_ShouldGenerateFiles**: Test the main method (CLI) with a standard class name to ensure it runs without exception (mocking file system interactions if necessary).
68. **testMainMethod_WithUtilsOption_ShouldGenerateStandardProxies**: Test the main method with -u 5 to verify it attempts to generate standard JDK proxies.
69. **testLoadDelayedProxy_WithArrayList_ShouldReturnDelayedArrayListProxy**: Verify internal mapping of ArrayList.class to DelayedArrayListProxy.class.
70. **testLoadDelayedProxy_WithUnknownType_ShouldReturnNull**: Verify internal mapping returns null for types that do not support delayed loading.

Perfect. Now, implement the Java code for the first 35 test scenarios (#1 to #35) from the list.

- Generate complete JUnit 4 test methods.
- Your output must start with **###Test START##** and finish with **###Test PART 1 END##**.
- Use the provided utility class where appropriate.

Perfect. Now, implement the Java code for the second part of test scenarios (#36 to #70) from the list.

- Generate complete JUnit 4 test methods.
- Your output must start with **###Test START##** and finish with **###Test PART 1 END##**.
- Use the provided utility class where appropriate.